
**Desarrollo de una Plataforma Web Inteligente
para Nutricionistas**
**Development of an Intelligent Web Platform for
Nutritionists**



Trabajo de Fin de Grado
Curso 2025–2026

Autor

Víctor Martínez Blanco

Director

Alejandro Hernández Cerezo

Pablo Gutiérrez Sánchez

Doble Grado en Ingeniería Informática y ADE

Facultad de Informática

Universidad Complutense de Madrid

Desarrollo de una Plataforma Web
Inteligente para Nutricionistas
Development of an Intelligent Web
Platform for Nutritionists

Trabajo de Fin de Grado en Ingeniería Informática

Autor

Víctor Martínez Blanco

Director

Alejandro Hernández Cerezo

Pablo Gutiérrez Sánchez

Convocatoria: Septiembre 2026

Doble Grado en Ingeniería Informática y ADE

Facultad de Informática

Universidad Complutense de Madrid

DIA de MES de AÑO

Dedicatoria

Próximamente

Agradecimientos

Próximamente.

Resumen

Desarrollo de una Plataforma Web Inteligente para Nutricionistas

Este Trabajo de Fin de Grado presenta el diseño de NutriApp, una plataforma web pensada para centralizar el flujo de trabajo de los nutricionistas profesionales. La idea surgió al observar de cerca cómo trabaja un nutricionista en su día a día: su información estaba repartida entre cuadernos físicos, formularios de Google, Google Calendar, la plataforma de su clínica, notas del móvil y WhatsApp. Si se quería analizar el estado completo de un solo cliente, se tenía que cruzar datos de todas estas herramientas de forma manual; lo cual resulta ineficiente y es probable perder información por el camino.

A partir de este problema real, se propuso en un primer momento diseñar una aplicación web que reuniera todas estas funcionalidades en un único punto de acceso. La plataforma se estructuraría en dos paneles diferenciados: uno orientado al nutricionista, desde el que puede gestionar clientes, citas, planes nutricionales y la mensajería con sus clientes; y otro orientado al cliente, con una interfaz propia donde consultar su plan, registrar las comidas completadas y su peso, comunicarse con su nutricionista y hacer seguimiento de su progreso.

Al contrastar esa primera idea con más nutricionistas en activo, el rumbo del proyecto se decidió finalmente hacia su funcionalidad más innovadora: la creación de dietas personalizadas para cada cliente, que pasó a ser el elemento diferenciador de la herramienta. Esta herramienta tan específica solo es posible de llevar a cabo con éxito cuando gran parte de la información relativa al usuario está conectada, siendo posible acceder a sus alérgenos, intolerancias, objetivos nutricionales, preferencias y hábitos entre otros. Además, al mismo tiempo, son de gran relevancia las normas, preferencias y restricciones que imponga su nutricionista al programa, ya que condicionarán el plan resultante. En la versión final, el nutricionista pide lo que necesita en su propio idioma; un modelo de lenguaje, ejecutado en local para que los datos clínicos no salgan del sistema, traduce esa petición a restricciones formales; un motor de optimización construye el plan sobre los alimentos guardados y modelados en la base de datos (cada uno con sus etiquetas de alérgenos y familia de alimentos, y con sus macronutrientes y propiedades) y un validador independiente lo audita; y el plan solo llega al cliente cuando el profesional lo revisa y lo firma. El resultado es

una plataforma con dos roles, nutricionista y cliente, cuyo núcleo es ese copiloto, y su potencial está en aliviar la tarea que más tiempo consume al profesional.

Palabras clave

Plataforma web, nutrición, gestión de clientes, planes nutricionales personalizados, copiloto, programación con restricciones, CP-SAT, modelo de lenguaje, adherencia

Abstract

Development of an Intelligent Web Platform for Nutritionists

This Bachelor's Thesis presents the design of NutriApp, a web platform intended to centralise the workflow of professional nutritionists. The idea came from observing closely how a nutritionist works day to day: his information was scattered across physical notebooks, Google Forms, Google Calendar, his clinic's platform, phone notes and WhatsApp. To analyse the complete situation of a single client, data from all these tools had to be cross-referenced by hand, which is inefficient and makes it likely that information gets lost along the way.

Starting from this real problem, the initial proposal was to design a web application that would bring all these functions together in a single point of access. The platform would be structured in two separate panels: one for the nutritionist, from which they can manage clients, appointments, nutritional plans and the messaging with their clients; and one for the client, with its own interface to consult their plan, record completed meals and weight, communicate with their nutritionist and follow their progress.

When that first idea was contrasted with more practising nutritionists, the project finally moved towards its most innovative feature: the creation of personalised diets for each client, which became the differentiating element of the tool. Such a specific tool can only be carried out successfully when a large part of the information about the user is connected, making it possible to access their allergens, intolerances, nutritional goals, preferences and habits, among others. At the same time, the rules, preferences and restrictions that the nutritionist imposes on the programme are also highly relevant, since they condition the resulting plan. In the final version, the nutritionist asks for what they need in their own language; a language model, run locally so that clinical data never leave the system, translates that request into formal constraints; an optimisation engine builds the plan over the foods stored and modelled in the database (each one with its allergen and food family tags, and with its macronutrients and properties) and an independent validator audits it; and the plan only reaches the client once the professional has reviewed and signed it. The result is a platform with two roles, nutritionist and client, whose core is that copilot, and its potential lies in relieving the professional of the task

that takes them the most time.

Keywords

Web platform, nutrition, client management, personalised nutritional plans, copilot, constraint programming, CP-SAT, language model, adherence

Índice

1. Introducción	1
1.1. Motivación	1
1.2. Objetivos	2
1.3. Plan de trabajo	3
2. Estado de la Cuestión	7
2.1. La dieta como problema de optimización	7
2.2. Criterios de comparación	8
2.3. Análisis de competidores	9
2.3.1. Nutrium	9
2.3.2. Practice Better y That Clean Life	10
2.3.3. Foodzilla	10
2.3.4. Eat This Much	11
2.4. Resumen comparativo	12
2.5. Posicionamiento de NutriApp	13
3. Análisis de Requisitos	15
3.1. Reformulación del alcance	15
3.1.1. Criterio de decisión aplicado en el triaje	16
3.1.2. Bloques eliminados completos y su justificación	17
3.2. Requisitos no funcionales	18
3.2.1. Requisitos no funcionales generales	18
3.2.2. Requisitos no funcionales específicos del copiloto IA	20
3.3. Requisitos funcionales	21
3.3.1. Gestión de clientes	22
3.3.2. Calendario y citas online	22
3.3.3. Mensajería	23
3.3.4. Planes nutricionales: estructura base	23
3.3.5. Copiloto de planes nutricionales (<i>innovación clave</i>)	23
3.3.6. Check-in mínimo de adherencia	24
3.3.7. Progreso del cliente	25
3.3.8. Base de datos de alimentos	25

3.3.9. Dashboard de métricas del copiloto	25
3.4. Requisitos propuestos a trabajo futuro	26
3.5. Dependencias funcionales	27
3.6. Funcionalidades resultantes	27
4. Diseño del sistema	31
4.1. Arquitectura general	31
4.2. Capa de datos	33
4.3. Capa de seguridad	35
4.4. Capa de servicio	36
4.5. Despliegue	37
4.6. Validación y testing	38
5. El motor de generación	39
5.1. La programación con restricciones	39
5.2. Formulación del problema	40
5.2.1. El problema y su contrato	40
5.2.2. Elección de tecnología	40
5.2.3. Conjuntos y parámetros	41
5.2.4. Variables de decisión	42
5.2.5. Trabajo en enteros	42
5.3. Restricciones del modelo	43
5.3.1. Restricciones estructurales	43
5.3.2. La capa de plausibilidad del catálogo	45
5.3.3. Restricciones clínicas y restricciones de catálogo	46
5.4. El catálogo de restricciones clínicas	47
5.5. Función objetivo y resolución	48
5.6. Diagnóstico de infactibilidad	50
5.7. El validador determinista	50
6. El copiloto de IA	53
6.1. El flujo del copiloto	53
6.2. Del texto a las restricciones	54
6.3. La elección del modelo de lenguaje	55
6.4. Revisión y arbitraje	56
6.5. De la petición al plan firmado	56
7. Evaluación	59
7.1. Bancos de verificación y línea base	59
7.2. La calidad del plan	60
7.2.1. La auditoría de sinsentidos	61
7.2.2. Comparativa de versiones del modelo	62
7.2.3. Las magnitudes de la función objetivo	64
7.2.4. La jerarquía del objetivo: bandas de tolerancia	66

7.3.	La eficiencia del motor	71
7.3.1.	El tamaño del problema	71
7.3.2.	Escalado	72
7.3.3.	Rotura de simetría entre días	73
7.3.4.	Cotas de dominio ajustadas	75
7.3.5.	El motor en el alojamiento gratuito	76
7.4.	El copiloto de IA	77
7.4.1.	El estudio de formatos del mensaje	77
7.4.2.	Comportamiento de los modelos locales	80
7.5.	Prueba con nutricionistas	82
8.	Conclusiones y Trabajo Futuro	85
8.1.	Conclusiones	85
8.2.	Limitaciones	87
8.3.	Trabajo futuro	88
8.3.1.	Motor de generación	88
8.3.2.	Copiloto de IA	89
8.3.3.	Plataforma	90
8.3.4.	Evaluación	91
8.4.	Cierre	91
	Introduction	93
	Conclusions and Future Work	99
	Bibliografía	107
A.	Catálogo original de requisitos (versión V1)	111
A.1.	Requisitos no funcionales (V1)	111
A.2.	Requisitos funcionales (V1)	113
A.2.1.	Gestión de clientes	113
A.2.2.	Calendario y citas	113
A.2.3.	Mensajería	114
A.2.4.	Formularios personalizables	114
A.2.5.	Sistema de alertas inteligentes (<i>innovación clave V1</i>)	114
A.2.6.	Planes nutricionales	114
A.2.7.	Lista de la compra inteligente (<i>innovación clave V1</i>)	114
A.2.8.	Check-in diario y gamificación (<i>innovación clave V1</i>)	116
A.2.9.	Seguimiento y progreso	116
A.2.10.	Informes y análisis	116
A.2.11.	Base de datos de alimentos con reconocimiento inteligente (<i>innovación clave V1</i>)	116
A.2.12.	Marketplace de recursos educativos (<i>innovación clave V1</i>)	117
A.2.13.	Automatización de procesos (<i>innovación clave V1</i>)	117

A.3. Dependencias funcionales (V1)	118
A.4. Funcionalidades resultantes (V1)	119
A.5. Nota final sobre el alcance definitivo	121
A.6. Comparativa con la competencia en el planteamiento inicial	122
B. Formalización completa del motor de generación	123
B.1. Umbrales y pesos del modelo	123
B.2. Variables y enlace	123
B.3. Restricciones estructurales (R1 a R8)	125
B.4. Restricciones de plausibilidad (R9 a R15)	126
B.5. Los 16 tipos configurables	127
B.6. Función objetivo	130
B.7. Mapa de restricciones a código	131
C. Prompts del sistema	133
C.1. El traductor	133
C.2. Las tres comprobaciones previas	136
D. Evaluación experimental: tablas completas	139
D.1. Casos de los bancos	139
D.2. Magnitudes de la función objetivo por caso	140
D.3. Rotura de simetría: valores por caso	142
D.4. Bandas de tolerancia: desviaciones con signo	142
D.5. Estudio de formatos: detalle	143
E. Declaración de uso de inteligencia artificial	147
E.1. Herramientas y modelos	147
E.2. Tareas en las que se usaron	148
E.3. Autoría y verificación	148

Índice de figuras

2.1. Nutrium: página pública para profesionales (nutrium.com, consultada el 3 de septiembre de 2026).	9
2.2. Practice Better: página pública (practicebetter.io, consultada el 3 de septiembre de 2026).	10
2.3. Foodzilla: página pública (foodzilla.com, consultada el 3 de septiembre de 2026).	11
2.4. Eat This Much: página pública (eatthismuch.com, consultada el 3 de septiembre de 2026).	12
4.1. Arquitectura del sistema: la capa de presentación, la de datos y la de servicio, con el worker de generación como pieza que corre en un ordenador local.	32
4.2. La tabla de restricciones como interfaz: tres productores escriben filas del mismo vocabulario y un único consumidor las aplica sin distinguir su origen.	33
4.3. Modelo de datos: las tablas del núcleo con sus relaciones y el resto del esquema agrupado por dominios. Las restricciones apuntan además a su objetivo (un alimento, una etiqueta o un nutriente).	34
4.4. Dos roles sobre las mismas tablas: cada uno entra con su sesión y ve o escribe lo que sus políticas permiten, con tres tablas como ejemplo.	36
4.5. Despliegue: tres patas en planes gratuitos de nube y el worker en una máquina local, todas comunicadas a través de la base de datos o de llamadas de servidor a servidor.	37
6.1. El flujo del copiloto. La fila superior son las decisiones del profesional y la inferior el trabajo del sistema: ningún resultado avanza hacia el cliente sin pasar antes por una decisión suya.	54
7.1. Realismo de las raciones frente a coste de resolución por versión del modelo. A la derecha, media y rango del tiempo de resolución.	63
7.2. Q de cada familia en cada caso, antes y después del ajuste (escala logarítmica). La línea discontinua marca $Q = 1$; la barra vertical de cada serie es su mediana.	65

7.3.	Variedad realizada por caso y brazo, con los óptimos probados sobre cada barra.	69
7.4.	Tiempo de resolución por caso y brazo (mediana de 3 repeticiones). .	69
7.5.	Escalado por dimensión: tiempo de resolución (arriba) y brecha de optimalidad (abajo). Con el tiempo en el límite, la brecha separa las configuraciones.	73
7.6.	Tiempo de resolución por caso y brazo, media y rango de las 5 repeticiones. Donde las tres barras tocan el límite, la diferencia está en el estado y en la calidad del incumbente, que recoge la tabla 7.11. . . .	74
7.7.	Repeticiones acertadas por mensaje y formato, de 3. Diez mensajes se traducen bien en los cinco formatos; los fallos se concentran en tres.	78
7.8.	Mensajes que superan las pasadas de validación previa, por formato (una pasada por cada uno de los 65 pares formato-mensaje).	80
D.1.	Desviación diaria (puntos) y media del plan (trazo) por repetición en los casos afectados limpios. Las franjas grises son la banda diaria de ± 50 kcal y la banda de la media de ± 25 . Tomás, el caso de conflicto, queda fuera de la figura porque su desviación de $-1\ 240$ kcal aplastaría la escala.	142
D.2.	Tasa de acierto por mensaje según el formato (media de 3 repeticiones, desviación cero).	143
D.3.	Exhaustividad y precisión por restricción, por formato.	145
D.4.	Latencia por llamada al traductor, por formato.	145
D.5.	Tokens de entrada y salida por llamada, por formato.	146

Índice de tablas

2.1. Cobertura de los competidores frente a NutriApp en los criterios de la sección 2.2. ETM: Eat This Much. Practice Better se valora con That Clean Life integrado.	12
3.1. Reformulación del alcance del TFG: del catálogo original al definitivo.	16
3.2. Bloques completos eliminados del alcance tras la reformulación. . . .	18
3.3. Requisitos funcionales — Gestión de clientes.	22
3.4. Requisitos funcionales — Calendario y citas online.	22
3.5. Requisitos funcionales — Mensajería.	23
3.6. Requisitos funcionales — Planes nutricionales: estructura base. . . .	23
3.7. Requisitos funcionales — Copiloto de planes nutricionales.	24
3.8. Requisitos funcionales — Check-in mínimo de adherencia.	24
3.9. Requisitos funcionales — Progreso del cliente.	25
3.10. Requisitos funcionales — Base de datos de alimentos.	25
3.11. Requisitos funcionales — Dashboard de métricas del copiloto. . . .	26
3.12. Requisitos individuales pospuestos a trabajo futuro.	28
3.13. Dependencias funcionales de NutriApp V2. La funcionalidad marcada con * es la única innovación clave del TFG.	29
5.1. Efecto de las reglas de sentido común sobre un mismo caso (7 días, 5 comidas).	44
5.2. Catálogo de restricciones clínicas: los 16 tipos.	47
7.1. Bancos de verificación permanentes y su resultado con el motor final.	60
7.2. Línea base de tiempos del banco del solver antes de los experimentos del motor (1 de septiembre de 2026). Los casos 6 y 7 son de infactibilidad y no aparecen.	61
7.3. La auditoría de sinsentidos antes y después de la capa de plausibilidad. Antes: 12 planes con 1 382 raciones en 420 comidas, 5 comidas al día. Después: los 9 planes de demostración regenerados, 961 raciones en 252 comidas, 4 comidas al día. Las raciones que quedan en 10 o 300 g son mínimos y máximos legítimos del perfil de su alimento.	62
7.4. Tres versiones del modelo sobre el mismo catálogo y los mismos 11 casos, con 3 repeticiones por caso.	63

7.5.	Conmensurabilidad de las familias no nutricionales del objetivo, antes y después del ajuste de pesos. Mediana de los 11 casos; el detalle por caso está en el Anexo D.	65
7.6.	Jerarquía real de la función objetivo tras el reescalado de la sección 7.2.3.	67
7.7.	Bandas de tolerancia: óptimos probados de 3 repeticiones, mediana del tiempo y mediana de alimentos del catálogo sin usar, por caso y brazo. Las desviaciones con signo están en el Anexo D.	68
7.8.	Evaluación de los criterios pre-registrados, brazo a brazo.	70
7.9.	Tamaño del modelo con el catálogo de 100 alimentos: variables de decisión, restricciones y variables que quedan tras el presolve de CP-SAT.	71
7.10.	Escalado por dimensión, el resto en el caso por defecto. Medias de 3 repeticiones.	72
7.11.	Rotura de simetría entre días en sus dos codificaciones, frente al motor sin orden (5 repeticiones por caso y brazo). Los valores de objetivo están en el Anexo D.	74
7.12.	Cotas de dominio apoyadas en R2 frente a las cotas heredadas (5 repeticiones por caso). El objetivo se minimiza: menor es mejor.	75
7.13.	Estudio de formatos: acierto por mensaje, exhaustividad y precisión por restricción, y restricciones espurias y rechazadas, por formato (13 mensajes, 3 repeticiones).	78
7.14.	Matriz modelo por prompt de la comprobación de ámbito (18 casos, 3 repeticiones).	82
A.1.	Requisitos funcionales V1 — Gestión de clientes.	113
A.2.	Requisitos funcionales V1 — Calendario y citas.	113
A.3.	Requisitos funcionales V1 — Mensajería.	114
A.4.	Requisitos funcionales V1 — Formularios personalizables.	114
A.5.	Requisitos funcionales V1 — Sistema de alertas inteligentes.	115
A.6.	Requisitos funcionales V1 — Planes nutricionales.	115
A.7.	Requisitos funcionales V1 — Lista de la compra inteligente.	115
A.8.	Requisitos funcionales V1 — Check-in diario y gamificación.	116
A.9.	Requisitos funcionales V1 — Seguimiento y progreso.	116
A.10.	Requisitos funcionales V1 — Informes y análisis.	116
A.11.	Requisitos funcionales V1 — Base de datos de alimentos.	117
A.12.	Requisitos funcionales V1 — Marketplace de recursos educativos.	118
A.13.	Requisitos funcionales V1 — Automatización de procesos.	118
A.14.	Dependencias funcionales V1: funcionalidades y sus requisitos. Las funcionalidades marcadas con * son innovaciones clave del planteamiento original.	119
A.15.	Cobertura funcional prometida por la idea inicial (V1) frente a los competidores analizados en 2025.	122
B.1.	Umbrales del modelo y parámetros de resolución.	124
B.2.	Mapa a código: modelo y reglas invariantes.	131

B.3. Mapa a código: catálogo clínico, objetivo y resolución.	131
C.1. Campos de cada restricción en el esquema de salida del traductor. . .	136
D.1. Contribución de cada familia al objetivo, por caso, antes del ajuste. El término de preferencias es negativo en el objetivo (premia), y aquí se da en valor absoluto.	141
D.2. Q por caso antes del ajuste de pesos. Tras multiplicar por 1 000 los pesos no nutricionales, cada valor se multiplica por 1 000 (medianas 32, 51 y 7).	141
D.3. Cadena lexicográfica frente al motor sin rotura.	141
D.4. Variante escalar frente al motor sin orden.	142
D.5. Desviaciones del objetivo por caso afectado, brazo y repetición. La última columna es el exceso fuera de la banda diaria y fuera de la banda de la media.	143
D.6. Los cinco formatos del estudio sobre el mismo mensaje.	144
D.7. Latencia y tokens por llamada al traductor, por formato, en el equipo local.	144
D.8. Los formatos frente a las pasadas de validación previa (una pasada por par formato-mensaje).	144

Lista de acrónimos

ADE	Administración y Dirección de Empresas.
API	<i>Application Programming Interface</i> , interfaz de programación de aplicaciones.
BEDCA	Base de Datos Española de Composición de Alimentos.
BMR	<i>Basal Metabolic Rate</i> , metabolismo basal.
CP	<i>Constraint Programming</i> , programación con restricciones.
CP-SAT	Solver de programación con restricciones de OR-Tools, que combina propagación de restricciones con técnicas SAT.
CPU	<i>Central Processing Unit</i> , unidad central de procesamiento.
CSS	<i>Cascading Style Sheets</i> , hojas de estilo de la web.
HIPAA	<i>Health Insurance Portability and Accountability Act</i> , ley estadounidense de protección de datos sanitarios.
IA	Inteligencia artificial.
JSON	<i>JavaScript Object Notation</i> , formato de intercambio de datos.
JSONB	Tipo de columna de PostgreSQL que guarda documentos JSON en formato binario.
LLM	<i>Large Language Model</i> , modelo de lenguaje de gran tamaño.
MCP	<i>Model Context Protocol</i> , protocolo que conecta un asistente de IA con herramientas externas.
PDF	<i>Portable Document Format</i> .
RF	Requisito funcional.
RGPD	Reglamento General de Protección de Datos.
RLS	<i>Row Level Security</i> , seguridad a nivel de fila.
RNF	Requisito no funcional.
SAT	<i>Boolean Satisfiability</i> , problema de satisfacibilidad booleana.
SQL	<i>Structured Query Language</i> , lenguaje de consulta de bases de datos relacionales.
TFG	Trabajo de Fin de Grado.
URL	<i>Uniform Resource Locator</i> , dirección de un recurso en la web.
USDA	<i>United States Department of Agriculture</i> , referencia internacional en composición de alimentos.

Introducción

Este capítulo presenta el punto de partida del proyecto: la motivación que lo origina, los objetivos que persigue y el plan de trabajo que ha guiado su desarrollo.

1.1. Motivación

Al plantear este Trabajo de Fin de Grado, existía una premisa clara: que el resultado tuviera una aplicación real. Resolver un problema inventado para cumplir un requisito académico no me motivaba, así que busqué la forma de utilizar los conocimientos adquiridos durante la carrera y orientarlos hacia una necesidad concreta de alguien cercano. La idea era que el trabajo sirviera como ejercicio técnico y, al mismo tiempo, como solución a un problema que alguien experimentaba en su día a día profesional.

Ese alguien resultó ser un amigo que trabaja como nutricionista. En una conversación sobre su rutina diaria, encontré un problema que él mismo no percibía como tal, porque lo había normalizado: su flujo de trabajo estaba fragmentado entre cinco o más herramientas que no se comunicaban entre sí. Utilizaba formularios de Google para recoger información de sus clientes, Google Calendar para gestionar las citas, WhatsApp como canal de comunicación directa, un cuaderno físico para sus notas privadas sobre cada caso, y la plataforma de gestión de su clínica para el resto. Para hacerse una idea completa de la situación de un solo cliente, tenía que consultar todas estas fuentes manualmente y reconstruir la información por su cuenta. Además, gran parte de su tiempo se iba en responder consultas repetitivas como: si se podía sustituir un ingrediente, cuándo debería ser la próxima cita, y miles de dudas varias sobre cómo arreglar el plan tras haberse saltado/cambiado comidas; cuestiones casi triviales que podrían resolverse sin su intervención directa.

A partir de esa conversación surgió la idea central de este proyecto: diseñar una plataforma que centralizara todas esas herramientas dispersas en un único punto. Si toda la información de los clientes (formularios, citas, mensajes, notas, planes nutricionales, registros diarios) viviera en una misma base de datos, se eliminaría la división de fuentes y se abrirían nuevas posibilidades como detectar patrones de forma automática, generar alertas cuando un cliente muestra señales de abandono, o construir un perfil completo sin tener que cruzar datos a mano. En definitiva,

que el nutricionista pudiera dedicar su tiempo a lo que realmente importa: diseñar dietas, acompañar a sus clientes y lograr que los tratamientos funcionen en lugar de invertirlo en tareas administrativas que una aplicación bien diseñada puede resolver.

Esa visión inicial, una plataforma que centralizara todo el flujo de trabajo, guió la primera versión del proyecto: se catalogó todo lo que una herramienta así podría ofrecer y se diseñó un prototipo completo con ello, desde formularios personalizables hasta un marketplace de recursos educativos. Al presentar la propuesta a más nutricionistas en activo, sus inquietudes ayudaron a repensar el enfoque. Lo que de verdad demandaban no era tenerlo todo en un mismo sitio, que obviamente también sería de gran ayuda, sino que también encontraban un cuello de botella con la tarea que más tiempo les consume como nutricionistas: elaborar cada dieta a medida. La ficha del cliente (sus datos, mediciones y restricciones) les importaba como apoyo de esa tarea; la comunicación con el cliente la veían como un complemento, no como el núcleo; y la centralización del resto de sus procesos, en la práctica, depende de la clínica en la que trabajan, no de la herramienta que ellos elijan.

Ese contraste con la demanda real me llevó a reformular el proyecto que se detalla en el capítulo 3. El centro dejó de ser la plataforma total y pasó a ser un copiloto de generación de planes nutricionales a medida: el nutricionista describe lo que necesita en su propio idioma, el sistema construye un borrador que cumple restricciones verificables, y el profesional lo revisa, lo ajusta y lo firma antes de que llegue al cliente. La plataforma sigue existiendo alrededor (clientes, citas, mensajes, seguimiento) pero como soporte de ese núcleo, no como fin en sí misma.

1.2. Objetivos

Este Trabajo de Fin de Grado se desarrolla en el marco del Doble Grado en Ingeniería Informática y Administración y Dirección de Empresas, lo cual permite abordarlo desde dos perspectivas complementarias: la técnica, que abarca desde el diseño de la base de datos hasta la integración de un motor de optimización y modelos de lenguaje; y la de negocio, aplicando herramientas de análisis de mercado y viabilidad de producto propias de la formación en ADE. El objetivo no es simplemente cumplir un requisito académico, sino aprovechar esta doble formación para construir un proyecto totalmente funcional con aplicación real.

El objetivo general de este trabajo es diseñar, construir y evaluar una plataforma web para nutricionistas cuyo núcleo es un copiloto de generación de planes nutricionales: el profesional expresa en su propio idioma las necesidades de un cliente, la inteligencia artificial las traduce a restricciones formales, un motor de optimización construye un borrador que las satisface y un validador independiente lo audita, quedando siempre la decisión final en manos del profesional. Se busca recorrer el proceso completo de un producto real, refinando la idea con profesionales y con el objetivo de despliegue público y la evaluación empírica, para que el resultado sea técnicamente sólido y también defendible como herramienta real.

Para alcanzar este objetivo general, se plantean los siguientes objetivos específicos:

1. Analizar el flujo de trabajo real de un nutricionista profesional para extraer

los requisitos funcionales del sistema. Esto implica comprender cómo trabaja en su día a día, qué herramientas utiliza actualmente, qué problemas enfrenta y qué necesita realmente. De esta forma, los requisitos no se inventan, sino que se extraen de la observación directa de un caso real, para que el diseño responda a necesidades concretas y no a suposiciones.

2. Realizar un estudio de mercado y análisis de la competencia en el ámbito de las plataformas de nutrición, aplicando conocimientos propios de la formación en ADE, para identificar los principales competidores, sus fortalezas y carencias, y posicionar NutriApp con una propuesta diferenciada.
3. Diseñar la arquitectura del sistema y su modelo de datos: dos paneles (nutricionista y cliente) sobre una misma base de datos, con el aislamiento entre profesionales garantizado en la propia capa de datos, y con una tabla de restricciones que actúa como interfaz entre quien las produce (el profesional o la IA) y el motor que las consume.
4. Construir un motor de generación de planes basado en programación con restricciones, acompañado de un validador clínico determinista e independiente, capaz de funcionar por sí solo, sin depender de ningún componente de inteligencia artificial.
5. Integrar la capa de IA que hace usable ese motor en lenguaje natural: traducción de las peticiones del profesional a restricciones formales, validación previa de los mensajes y generación asíncrona manteniendo al profesional como árbitro de cada decisión.

1.3. Plan de trabajo

El desarrollo de este proyecto ha seguido un proceso iterativo en el que cada fase se apoya en los resultados de la anterior. A continuación se describen las fases, desde la investigación inicial con profesionales hasta la evaluación del sistema final.

Fase 1. Investigación con nutricionistas reales (de mayo de 2025 a febrero de 2026)

El punto de partida fue entender en profundidad el problema que se quería resolver. Mantuve una serie de conversaciones con el nutricionista cuyo caso motivó el proyecto para conocer a fondo su método de trabajo: qué herramientas utilizaba, en qué puntos perdía más tiempo y qué información se le escapaba por tenerla repartida entre plataformas desconectadas.

Tras consultar con los tutores, me recomendaron contactar a más nutricionistas para no basar el producto en la experiencia de una sola persona. Por ello contacté a través de LinkedIn con otros profesionales del sector y les pregunté lo mismo. Esta ronda de consultas me permitió contrastar ideas para construir una visión del producto más amplia y sólida.

Fase 2. Análisis de competencia y mercado (febrero y marzo de 2026)

Una vez definidas las necesidades desde el lado del usuario, era el momento de investigar el mercado. Obviamente, ya existían soluciones muy similares, así que convenía saber quiénes eran los competidores, qué ofrecían y dónde dejaban huecos sin cubrir. Analicé las principales plataformas del sector (Nutrium (Nutrium, 2026), Indya (INDYA, 2026), Dietopro (DietoPro, 2026), Nutrimeal (Nutrimeal, 2026) y Practice Better (Practice Better, 2026)) estudiando sus funcionalidades, modelos de negocio, puntos fuertes y carencias, para detectar qué aspectos se habían quedado fuera del radar en la fase anterior y si las necesidades que yo había recogido estaban ya cubiertas o existía una oportunidad real. Al final de esta fase, la idea del producto estaba mucho más definida, con una propuesta de valor más clara. Aquel primer análisis, hecho para el alcance inicial, se conserva en el Anexo A; la comparativa definitiva, ya centrada en el copiloto, es la del capítulo 2.

Fase 3. Diseño visual del prototipo (marzo de 2026)

Con los requisitos extraídos, los traduje en una interfaz concreta: los mockups de la aplicación. El objetivo de las pantallas era doble; que el nutricionista pudiera gestionar todo su trabajo desde un único punto de acceso (clientes, citas, formularios, planes, mensajes y notas) y que el cliente contara con herramientas para consultar su plan, registrar sus hábitos y seguir su progreso. Cada pantalla cubría casos de uso identificados en las fases anteriores.

El conjunto de pantallas seguía el mismo sistema de diseño para mantener la coherencia visual. Se revisó lo ya diseñado para incorporar mejoras detectadas durante el análisis de requisitos, como las vistas diaria, semanal y mensual del calendario o la pantalla de ajustes del cliente. El resultado fue un primer prototipo navegable con los dos paneles interconectados y todos los requisitos funcionales iniciales presentes.

Fase 4. Segunda iteración: competencia en profundidad y nuevas funcionalidades (abril de 2026)

Tras completar el primer prototipo, analicé por segunda vez la competencia, esta vez contrastando NutriApp con siete plataformas de referencia. De ese análisis surgieron cuatro incorporaciones al diseño: un motor de automatizaciones configurable, un marketplace de recursos educativos, una base de datos de alimentos con reconocimiento inteligente y las videollamadas integradas en el chat; además de refinamientos varios, como la ficha detallada del cliente. El resultado fue un prototipo navegable completo de veinticuatro pantallas, con los sesenta y un requisitos funcionales representados y la navegación cableada en Figma (Figma, Inc., 2026a), de forma que un evaluador externo podía recorrerlo como si fuera una aplicación real.

Fase 5. Validación con usuarios reales (abril de 2026)

Con el prototipo completo, preparé dos guías de prueba de usabilidad (nutricionista y cliente), cada una con doce flujos de navegación formulados como pequeños retos (“quieres consultar la adherencia media de tus clientes, ¿a qué sección del menú lateral irías?”). El evaluador debía simular el flujo de navegación en cada caso y marcar si le resultó obvia, confusa o si directamente no sabría cómo llegar.

Fase 6. Reformulación del alcance (abril de 2026)

Inesperadamente, el prototipo completo “final” (para el plan que había en ese momento) resultó ser la mejor herramienta para contrastar la idea de implementación. Al presentarlo a más nutricionistas y hablar más con ellos para recibir feedback, quedó claro que el valor no estaba en abarcar todas las funcionalidades posibles, sino en automatizar la elaboración de dietas a medida, la tarea que más tiempo les consume.

Además, los tutores cuando probaron el prototipo, me hicieron ver la complejidad y cantidad de funcionalidades que quería programar yo solo. Esto me hizo ver que debía enfocarme en cubrir la necesidad primaria de los nutricionistas y centrar mis esfuerzos en esa dirección. Así que decidí acotar el alcance drásticamente a lo que sostiene esa tarea y concentrar el esfuerzo técnico en el copiloto de generación de planes, que es donde se encontraba el valor real. El capítulo 3 documenta el triaje completo y sus criterios.

Fase 7. Diseño e implementación de la base de datos (mayo de 2026)

Siguiendo el orden que me recomendaron mis directores del TFG (primero la base de datos, después su conexión y por último las pantallas), diseñé e implementé el modelo de datos completo sobre PostgreSQL gestionado: las tablas de catálogos, personas, restricciones y planes; las políticas de seguridad a nivel de fila que garantizan el aislamiento entre profesionales; y los datos de arranque necesarios para trabajar con casos realistas desde el primer día.

Fase 8. Construcción de la aplicación web (mayo y junio de 2026)

Sobre esa base construí la aplicación en dos pasos. Primero, la conexión de las dos piezas que hablan con la base de datos: el backend en Python y el frontend en Next.js (Vercel, Inc., 2026), cada uno verificado con lecturas y escrituras reales antes de dibujar una sola pantalla. Después, el panel del nutricionista sección a sección (clientes, planes, alimentos, calendario, mensajería y ajustes), siempre sobre datos reales y sin simular funcionalidad que no existiera: si una pantalla mostraba algo, había una tabla detrás que lo sostenía.

La aplicación se desplegó públicamente en cuanto tuvo sus primeras secciones, siguiendo la recomendación de los tutores, lo que permitió enseñarla y probarla

en condiciones reales lo antes posible. Al cerrar esta fase comparé el prototipo de Figma con lo implementado y revisé las diferencias, para no olvidarme de elementos y revisar de nuevo si algunos de ellos no eran necesarios, justificando la divergencia.

Fase 9. Motor de generación de planes (junio de 2026)

El corazón técnico del proyecto: la formalización de la dieta como un problema de satisfacción de restricciones. Tras consultar con uno de mis directores cuál sería el mejor enfoque, me explicó que su implementación debería consistir en un solver de programación con restricciones, un validador determinista independiente que audita cada plan antes de darlo por bueno, y me indicó que podría añadir también un diagnóstico de infactibilidad cuando las peticiones son incompatibles entre sí. El motor funciona sin ningún componente de inteligencia artificial, lo que permitió verificarlo primero con un banco de casos y después desde la propia web (creando un apartado para modificar los parámetros que se usan para generar el plan con el motor).

Fase 10. Capa de inteligencia artificial (julio de 2026)

Con el motor verificado se añadió la capa que lo hace usable en lenguaje natural: el traductor que convierte las peticiones del nutricionista en restricciones formales, las validaciones previas que examinan cada mensaje antes de traducirlo, la generación asíncrona y el estilo clínico que cada profesional define una vez y se aplica a todos sus planes. Todo esto se implementó mediante consultas a un modelo de lenguaje abierto, qwen2.5, ejecutado en local con Ollama (Ollama, 2026).

Fase 11. Panel del cliente (julio de 2026)

Después desarrollé el segundo rol de la aplicación: el panel del cliente. El cliente accede a su plan una vez firmado, marca las comidas que ha completado, registra su peso y solicita citas; toda esa información vuelve al panel del nutricionista como métricas de seguimiento. Con esta fase queda cerrado el ciclo completo entre profesional y cliente y toda la información que debía ser compartida entre ambos roles ya estaba implementada.

Fase 12. Evaluación (de julio a septiembre de 2026)

La evaluación acompañó a todo el desarrollo. Incluye un estudio pre-registrado sobre el efecto del formato de los mensajes en la traducción (con un resultado negativo que evitó construir un módulo innecesario), mediciones de rendimiento del motor en el entorno de producción al crecer el catálogo de alimentos, y una auditoría sistemática de la calidad nutricional de los planes generados, cuyo antes y después justificó una capa entera de reglas nuevas del motor. Y se cerró con una prueba guiada con dos nutricionistas sobre la aplicación desplegada, que completó el ciclo que empezó en la Fase 1: del profesional al prototipo, del prototipo al producto, y del producto de vuelta al profesional.

Capítulo 2

Estado de la Cuestión

NutriApp no nace en un mercado vacío. Existen plataformas comerciales que ya centralizan el trabajo de un nutricionista, y algunas de ellas generan planes de forma automática. Antes de diseñar una propuesta propia convenía saber qué se ofrece ya, qué funciona bien y dónde quedan huecos.

Este capítulo recoge ese análisis en dos partes. La primera sitúa el problema de generar una dieta en la literatura académica, desde la programación lineal de los años cuarenta hasta los trabajos recientes con programación con restricciones y modelos de lenguaje. La segunda estudia cuatro plataformas comerciales con un mismo conjunto de criterios, prestando atención a cómo generan planes las que lo hacen. La comparativa se hace sobre lo que NutriApp entrega; la que acompañó a la idea inicial del proyecto, con más funcionalidades y otros competidores, se conserva en el Anexo A.

2.1. La dieta como problema de optimización

Formular una dieta como un problema de optimización es anterior a los ordenadores. En 1945 Stigler se preguntó cuál era la dieta más barata que cubría los requerimientos nutricionales de un adulto, con 77 alimentos y 9 nutrientes, y la resolvió a mano por aproximaciones (Stigler, 1945). Dos años después el mismo problema sirvió como primera prueba a gran escala del método simplex, que encontró el óptimo exacto (Dantzig, 1990). La formulación es programación lineal pura: una cantidad continua de cada alimento, un coste a minimizar y una desigualdad por nutriente.

Dantzig contaba lo que ocurrió al aplicar el modelo a su dieta: el óptimo matemático proponía cantidades absurdas de unos pocos ingredientes. Ese es el problema de fondo que arrastra el enfoque desde entonces: cubrir los nutrientes al mínimo coste no produce comidas que alguien quiera comer. Posteriormente se corrigió añadiendo restricciones de aceptabilidad, y la revisión de Gazan et al. (2018) sobre los modelos de dieta actuales sigue manifestando esa tensión entre adecuación nutricional y aceptabilidad como el eje del campo.

El paso de la dieta agregada al menú concreto lo dio Balintfy (1964) con el

primer planificador de menús por ordenador, pensado para hospitales: días, comidas y platos, con restricciones de nutrientes y de variedad entre días. Petot et al. (1998) añadieron la preferencia individual con un sistema experto que combinaba reglas y casos anteriores, y Kahraman y Seven (2005) trataron el problema con un algoritmo genético de dos objetivos, el coste y la valoración de los platos. Las carencias que estos trabajos señalan sobre la formulación clásica son las mismas que aparecen en una consulta real: aceptabilidad de las combinaciones, variedad a lo largo de los días y preferencias de la persona.

Todos estos enfoques comparten un rasgo: el problema mezcla cantidades con decisiones lógicas y de conteo (si un alimento entra o no en una comida, cuántos alimentos distintos hay, cuántas veces se repite uno en la semana), y esas decisiones no se expresan bien con desigualdades continuas. Los algoritmos evolutivos las admiten, pero no garantizan que se cumpla una restricción obligatoria ni pueden afirmar que no existe solución. La programación con restricciones trabaja directamente con variables enteras y booleanas y con restricciones lógicas, y sus solvers actuales, como CP-SAT, combinan la propagación de restricciones con técnicas SAT y de programación lineal. Para este problema tiene dos ventajas concretas: el catálogo cerrado de alimentos hace que el espacio de búsqueda sea finito, y cuando las restricciones son incompatibles el solver puede señalar un subconjunto de ellas en conflicto, que es la base de la explicación que recibe el profesional. El capítulo 5 presenta el paradigma (sección 5.1) y la formulación de NutriApp (sección 5.2).

Los modelos de lenguaje han abierto otra vía: pedir la dieta directamente a un modelo generativo. Los trabajos recientes son cautos con ella. Valmeekam et al. (2023) midieron la capacidad de estos modelos para planificar y la encontraron muy limitada, y Kambhampati et al. (2024) proponen usarlos como fuente de conocimiento aproximado dentro de un marco en el que la verificación la hace un componente formal externo. En nutrición, Papastratis et al. (2024) combinan un modelo generativo con un optimizador que ajusta las raciones para cuadrar la energía, y en programación con restricciones Michailidis et al. (2024) estudian el uso de modelos de lenguaje para traducir descripciones en texto a modelos de restricciones ejecutables. NutriApp sigue esa misma línea: el modelo de lenguaje traduce lo que escribe el profesional a restricciones formales y el plan lo construye el solver, como se describe en el capítulo 6.

2.2. Criterios de comparación

Las plataformas se han revisado en sus webs públicas, todas consultadas el 3 de septiembre de 2026, bajo los mismos criterios:

- Gestión de clientes (perfil, mediciones, notas del profesional).
- Calendario y citas.
- Mensajería con el cliente.
- Base de datos de alimentos, con alimentos propios del profesional.

- Generación automática de planes: qué recibe como entrada, si explica al profesional por qué no hay solución y si el plan pasa por una revisión antes de llegar al cliente.
- Registro del cliente (comidas completadas, peso).
- Idioma de la interfaz y mercado al que se dirige.

A estos criterios se añade una valoración cualitativa de la interfaz y del público de cada plataforma. Cuando una web no documenta un aspecto, se dice; no se ha inferido nada de las zonas privadas de ningún producto.

2.3. Análisis de competidores

2.3.1. Nutrium

Nutrium (Nutrium, 2026) es la plataforma integral más extendida entre nutricionistas: opera en unos noventa países y su interfaz está disponible en seis idiomas, castellano incluido. La figura 2.1 muestra su página para profesionales.



Figura 2.1: Nutrium: página pública para profesionales (nutrium.com, consultada el 3 de septiembre de 2026).

Cubre con solvencia la gestión de la consulta: agenda con recordatorios, videoconsulta, mensajería, y una aplicación del paciente con diario de comidas y seguimiento del progreso. El plan lo construye el profesional a mano sobre la base de datos de alimentos, con plantillas y una biblioteca de recetas de otros profesionales, y el cálculo nutricional se actualiza al ajustar las raciones. Su única función de inteligencia artificial son las notas clínicas: genera un resumen estructurado de la videoconsulta.

Sus puntos fuertes son la madurez del producto y la interfaz, limpia y bien organizada, que tomé como referencia visual para el panel del nutricionista. Sus

limitaciones para este trabajo son dos: no genera planes (el tiempo de elaborar cada dieta sigue siendo del profesional) y su catálogo de funcionalidades es amplio para una consulta pequeña.

2.3.2. Practice Better y That Clean Life

Practice Better (Practice Better, 2026) es una plataforma canadiense de gestión para profesionales de la salud, con más de cincuenta mil usuarios, cumplimiento de HIPAA y RGPD, y solo en inglés (figura 2.2). Ofrece agenda, telesalud, mensajería, formularios de admisión, historia clínica, programas para clientes y automatizaciones de flujo, además de una aplicación móvil en la que el cliente registra comida y hábitos. Su inteligencia artificial se dedica a la documentación: resúmenes de sesiones y notas clínicas que, según su propia web, el profesional revisa y firma.

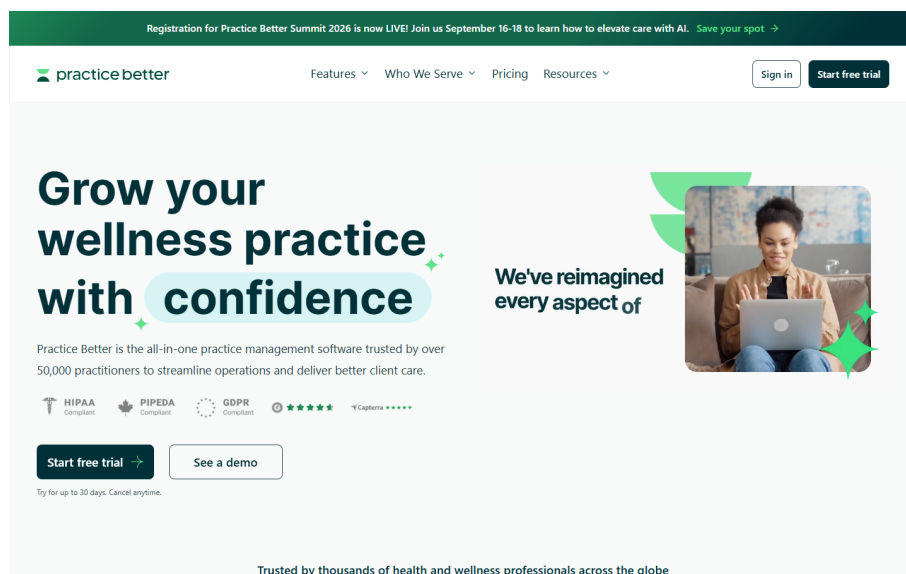


Figura 2.2: Practice Better: página pública (practicebetter.io, consultada el 3 de septiembre de 2026).

La planificación de dietas no es nativa: llega por la integración con That Clean Life (That Clean Life, 2026), una herramienta de planes con más de ocho mil recetas y ciento cincuenta plantillas por patologías. Su generador “crea planes en segundos a partir de calorías, macros y tipo de dieta” y se presenta como “un punto de partida” que el profesional ajusta. La web no describe cómo se eligen las recetas ni qué ocurre cuando los objetivos no se pueden cumplir.

2.3.3. Foodzilla

Foodzilla (Foodzilla, 2026) es una plataforma para dietistas, nutricionistas y entrenadores, con presencia sobre todo en Estados Unidos, Reino Unido, Australia y Nueva Zelanda (figura 2.3). Combina la gestión de la consulta (portal de clientes, chat cifrado, citas y videoconsulta) con una aplicación de marca propia para el cliente, en la que este registra sus comidas con reconocimiento por foto y código de

barras. Su base de alimentos supera el medio millón de entradas y su recetario las cien mil, de las que unas mil quinientas están revisadas por dietistas.



Figura 2.3: Foodzilla: página pública (foodzilla.com, consultada el 3 de septiembre de 2026).

Es la plataforma profesional con la generación automática más visible. El profesional fija calorías, reparto de macronutrientes, exclusiones y número de comidas al día, y el sistema “genera el plan en menos de 60 segundos” ensamblando recetas de su biblioteca y escalando las raciones; después se ajusta arrastrando recetas. La web insiste en que las recetas “nunca son generadas por IA”, pero no publica cómo se seleccionan ni si el resultado se verifica contra los objetivos, y no hay un paso de revisión obligatorio antes de que el plan llegue al cliente.

2.3.4. Eat This Much

Eat This Much (Eat This Much, 2026) es un generador automático de planes pensado para el consumidor final, con una versión para profesionales (figura 2.4). El usuario indica calorías, reparto de macronutrientes, tipo de dieta, alimentos a evitar y lo que tiene en la despensa, y la aplicación “crea una semana completa de comidas, recetas y lista de la compra con un clic”. En la versión profesional esos parámetros se fijan por cliente desde un panel, y el cliente puede usar la aplicación como registro de lo que come.

Es el producto más cercano al motor de NutriApp en lo que hace: ajusta cada comida para acercarse a los objetivos del día, reparte la variedad entre días y tiene en cuenta el desperdicio al componer la lista de la compra. Sobre el cómo, la web solo habla de “algoritmos” y de combinar “estrategias nutricionales establecidas con potencia de cálculo”. No hay gestión de consulta (citas, mensajería), la interfaz está en inglés y no se describe una revisión profesional antes de entregar el plan.

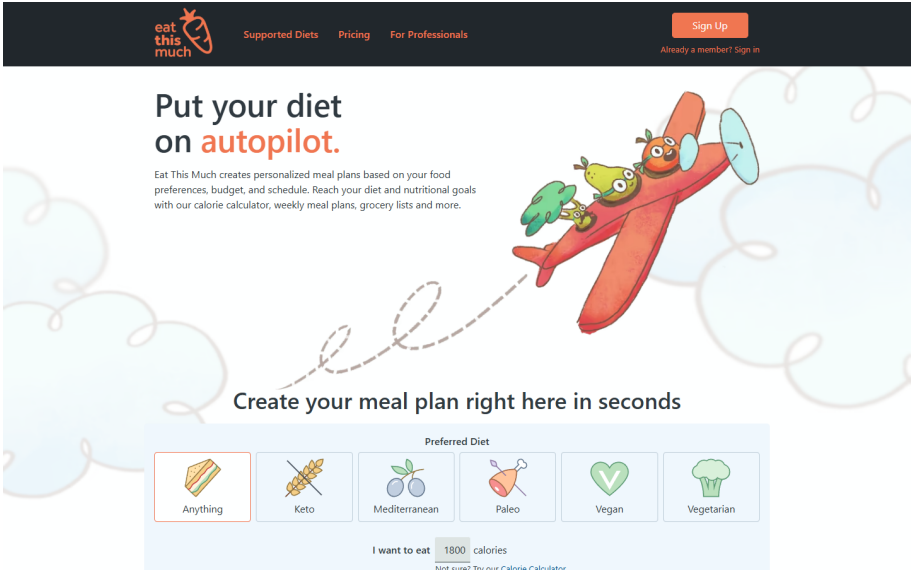


Figura 2.4: Eat This Much: página pública (eatthismuch.com, consultada el 3 de septiembre de 2026).

Criterio	NutriApp	Nutrium	P. Better	Foodzilla	ETM
Gestión de clientes	Sí	Sí	Sí	Sí	Parc.
Calendario y citas	Sí	Sí	Sí	Sí	No
Mensajería con el cliente	Sí	Sí	Sí	Sí	No
Planes contruidos a mano	No	Sí	Sí	Sí	No
Alimentos propios del profesional	Sí	Sí	Parc.	Sí	n.d.
Generación automática	Sí	No	Sí	Sí	Sí
Petición en lenguaje natural	Sí	No	No	No	No
Explicación si no hay solución	Sí	n.a.	n.d.	n.d.	n.d.
Revisión profesional previa	Sí	Sí	Parc.	Parc.	n.d.
Registro del cliente (comidas, peso)	Sí	Sí	Sí	Sí	Sí
Interfaz en castellano	No	Sí	No	Parc.	No

Tabla 2.1: Cobertura de los competidores frente a NutriApp en los criterios de la sección 2.2. ETM: Eat This Much. Practice Better se valora con That Clean Life integrado.

2.4. Resumen comparativo

La tabla 2.1 resume la cobertura de cada plataforma frente a NutriApp. Se usa “Sí” cuando la funcionalidad está presente, “Parc.” cuando existe de forma limitada, “No” cuando no está, “n.d.” cuando la web pública no lo documenta y “n.a.” cuando no aplica.

La tabla deja ver dos cosas. Las plataformas integrales cubren la gestión de la consulta mejor que NutriApp, que no tiene constructor manual de planes ni interfaz

en castellano (la petición al copiloto sí se escribe en castellano; la interfaz está en inglés). Y las tres que generan planes lo hacen como una caja cerrada: reciben calorías, macronutrientes y exclusiones y devuelven un plan, sin explicar qué pasa cuando los objetivos son incompatibles ni describir una revisión profesional obligatoria.

2.5. Posicionamiento de NutriApp

NutriApp no compite con estas plataformas en amplitud. Toma de Nutrium la organización del panel y el lenguaje visual, y de todas ellas el patrón de una aplicación del cliente con su plan y su registro diario. Su diferencia está en la generación.

Foodzilla, That Clean Life y Eat This Much generan a partir de números: calorías, macronutrientes, exclusiones. En NutriApp el profesional escribe lo que necesita en su idioma, un modelo de lenguaje lo traduce a restricciones formales de un vocabulario cerrado y el plan lo construye un solver de programación con restricciones sobre ese modelo (capítulo 6). La distinción importa por dos motivos. El plan cumple las restricciones obligatorias por construcción, y cuando no existe ninguno que las cumpla el solver señala qué restricciones chocan, en vez de devolver un plan que las incumple en silencio (sección 5.6). Y el modelo de lenguaje nunca decide qué se come: propone restricciones que el profesional acepta o descarta una a una, y el plan solo llega al cliente cuando el profesional lo firma.

También hay una diferencia académica. Ninguno de los tres generadores publica cómo elige las comidas ni cómo trata los objetivos incompatibles, algo esperable en un producto comercial. El capítulo 5 documenta el modelo completo de NutriApp, con sus variables, sus restricciones y su función objetivo, y el capítulo 7 mide su comportamiento; es lo que permite discutir el generador y no solo usarlo.

Por último, la plataforma no depende de servicios externos de datos: el catálogo de alimentos es propio y vive en la base de datos del sistema (capítulo 4), y el único componente externo es el modelo de lenguaje, que se ejecuta en local (sección 6.3). Integraciones como wearables, pasarelas de pago o videollamada, habituales en las plataformas analizadas, quedan fuera de este trabajo.

Capítulo 3

Análisis de Requisitos

A partir de la investigación con nutricionistas reales y del análisis de mercado descritos en el plan de trabajo, se extrajeron los requisitos que debe cumplir la plataforma. Este capítulo recoge tanto las restricciones de calidad del sistema (requisitos no funcionales) como las funcionalidades concretas que debe ofrecer (requisitos funcionales), sus dependencias, las funcionalidades resultantes y las decisiones tecnológicas que lo sostienen.

3.1. Reformulación del alcance

El catálogo inicial de requisitos recogía 61 requisitos funcionales y 7 no funcionales, estructurados alrededor de ocho innovaciones clave, y respondía a una pregunta demasiado ambiciosa: todo lo que una plataforma de gestión para nutricionistas podría llegar a ofrecer. Este catálogo se conserva en el Anexo A como registro del punto de partida del proyecto.

Como ya comenté, las necesidades más importantes de los profesionales fueron lo que obligaron a matizarlo. Tras una reunión con mis directores del TFG en abril de 2026, acordamos una reducción a unos 44 requisitos de primer nivel, ya que era inviable abarcarlo todo en un desarrollo en solitario.

Al descubrir la demanda real de los nutricionistas, automatizar la elaboración de cada dieta a medida (la tarea que más tiempo les quita); clasifiqué la importancia de las funcionalidades relacionadas: la ficha del cliente como apoyo de esa tarea es útil; la comunicación con el cliente es un complemento, no el núcleo; y la centralización del resto de sus procesos depende de las clínicas en las que trabajan, no de la herramienta.

Por ello, la decisión resultante no fue un recorte por falta de tiempo, sino más bien una elección informada por ese feedback: concentrar todo el esfuerzo técnico en una única funcionalidad diferencial de alta complejidad, un *copiloto* de generación de planes nutricionales en el que la inteligencia artificial propone, y el profesional revisa, ajusta y firma cada plan antes de entregarlo.

Este giro obliga a reformular los requisitos en dos direcciones complementarias:

- **Recorte en amplitud.** Se elimina por completo la implementación de bloques

que no contribuyen al núcleo del copiloto o a su evaluación: formularios personalizables, sistema de alertas inteligentes, lista de la compra, marketplace de recursos educativos, motor de automatizaciones configurable, reconocimiento de alimentos por imagen, videollamadas y chat entre clientes. Varios bloques adicionales se conservan en forma reducida (calendario, check-in, progreso, informes).

- **Ampliación en profundidad.** Se añaden ocho nuevos requisitos funcionales que describen el copiloto con el detalle necesario para su implementación (traducción de lenguaje natural a restricciones formales, validación previa del mensaje, arbitraje del profesional sobre cada propuesta, generación mediante un motor de optimización, validación clínica automatizada, explicación de infactibilidades, estilo clínico del profesional y seguimiento de generaciones asíncronas), así como cinco nuevos requisitos no funcionales específicos de la parte de inteligencia artificial (latencia, validación clínica obligatoria, trazabilidad, protección de datos de salud y explicabilidad).

El resultado es un catálogo final de 27 requisitos funcionales (19 conservados del catálogo inicial, 8 nuevos del copiloto) y 14 requisitos no funcionales (7 generales, 7 específicos de IA). La reducción global de requisitos funcionales respecto al catálogo original es del 56 %, mientras que la carga de trabajo real no disminuye en la misma proporción ya que cada uno de los ocho requisitos nuevos del copiloto arrastra consigo una arquitectura técnica compleja.

De esta forma se redefine el cómputo de innovaciones clave; pasan de ser ocho inicialmente a centrar el foco en una única innovación principal de mucha mayor ambición técnica: el copiloto de planes nutricionales per-nutricionista. Las otras siete quedan fuera del alcance (formularios con reglas de alerta, lista de la compra inteligente, *check-in* gamificado, base de alimentos con reconocimiento por IA, *marketplace*, motor de automatizaciones y comunidad entre clientes).

La tabla 3.1 resume el antes y el después del alcance del TFG.

Categoría	V1 (original)	V2 (final)	Variación
Requisitos funcionales	61	27	−56 %
Requisitos no funcionales	7	14	+7 (+8 nuevos, −1 eliminado)
Innovaciones clave	8	1 (copiloto IA)	−7 a trabajo futuro
Bloques de funcionalidad	13	8 (parcialmente reducidos)	−5 bloques eliminados

Tabla 3.1: Reformulación del alcance del TFG: del catálogo original al definitivo.

3.1.1. Criterio de decisión aplicado en el triaje

Con la solicitud por parte de los nutricionistas como referencia, cada requisito del catálogo original se ha clasificado aplicando el siguiente criterio, en este orden de prioridad:

1. **¿Contribuye al funcionamiento del copiloto IA?** Requisitos como el perfil del cliente con mediciones (RF-01, RF-04, RF-05), la base de datos de alimentos (RF-25, RF-45, RF-48) o la estructura básica de planes (RF-24, RF-29) se mantienen porque alimentan directamente el *pipeline* de generación.
2. **¿Contribuye a la evaluación experimental del copiloto?** El *check-in* del cliente se conserva en versión reducida (sólo marcar qué comidas del plan se han completado) porque alimenta la métrica de adherencia, que es una de las variables clave para demostrar empíricamente en la defensa que los planes generados funcionan. Del mismo modo, la gráfica de peso se mantiene por el mismo motivo.
3. **¿Es imprescindible para que el producto tenga sentido como aplicación?** La mensajería mínima (chat RF-12, RF-13) y el calendario reducido (RF-06, RF-09, RF-10, todas las citas en modalidad *online*) se conservan porque sin ellas la plataforma no funciona como herramienta de gestión profesional coherente, aunque no contribuyan directamente al copiloto.
4. Todo lo demás pasa a trabajo futuro. Los requisitos que no cumplen ninguno de los tres criterios anteriores, por valiosos que sean como producto, se recogen en el capítulo 8.

A continuación, se presentan los requisitos finales agrupados por bloque temático en secciones, manteniendo la estructura del catálogo original del Anexo A para facilitar la trazabilidad.

3.1.2. Bloques eliminados completos y su justificación

Cinco bloques completos del catálogo original se eliminan de la implementación del TFG y se recogen en el capítulo 8. La tabla 3.2 los recoge con la razón concreta que justifica su exclusión.

Adicionalmente, se eliminan requisitos individuales de otros bloques: videollamadas y llamadas de voz (RF-43, RF-44), chat entre clientes (RF-61), multi-clínica y colores por tipo de cita (RF-07, RF-08, RF-11), plantillas reutilizables de planes (RF-28), sustitución de comida por parte del cliente (RF-59), gamificación del *check-in* (RF-34, RF-35), *heat map* y fotos de progreso (RF-37, RF-38), exportación a PDF (RF-40), escáner de códigos de barras y reconocimiento fotográfico (RF-46, RF-47), registrar alimento consumido (RF-49) y comparativa con objetivos diarios (RF-50). Todos se justifican individualmente en las secciones de su bloque correspondiente o en el capítulo de trabajo futuro.

La reformulación también operó en sentido inverso. El prototipo original, pese a su amplitud, tenía huecos que la implementación tuvo que cubrir: no existía pantalla de inicio de sesión (la navegación arrancaba directamente en el panel) ni catálogo de alimentos propio del nutricionista, dos piezas imprescindibles en cuanto la aplicación dejó de ser un diseño navegable y pasó a ser un producto con datos reales. Ambas se incorporaron al alcance durante el desarrollo.

Con esta reformulación cerrada, el diseño e implementación del TFG se guiará por las siguientes secciones de requisitos.

Bloque eliminado	RFs afectados	Razón de exclusión
Formularios personalizables	RF-14 a RF-17	Editor visual <i>drag-and-drop</i> con plantillas y asignación programada. Alto coste de desarrollo, sin contribución al copiloto.
Alertas inteligentes	RF-18 a RF-23	Motor de reglas sobre formularios con prioridades y <i>dashboard</i> dedicado. Su concepto se absorbe parcialmente en RF-66 (validación clínica del copiloto).
Lista de la compra	RF-30 a RF-32	Generación desde el plan con sustituciones y marcado. Derivable trivialmente del plan generado cuando sea necesario; no aporta al diferencial IA.
<i>Marketplace</i> de recursos	RF-51 a RF-55	Catálogo, publicación, gestión de ventas y biblioteca personal. Implica un subsistema completo de comercio electrónico sin aporte académico al núcleo IA.
Motor de automatizaciones	RF-41, RF-42, RF-56 a RF-58	Motor de reglas configurable con disparadores y acciones. Alta complejidad; la automatización básica se suprime también al desaparecer los formularios y las alertas que la disparaban.

Tabla 3.2: Bloques completos eliminados del alcance tras la reformulación.

3.2. Requisitos no funcionales

Los requisitos no funcionales definen las restricciones y condiciones de calidad que debe cumplir el sistema, independientemente de las funcionalidades concretas que ofrezca. Respecto al catálogo V1 (Anexo A), cinco se conservan sin cambios, uno se reformula y uno se elimina al quedar fuera del alcance las clínicas como entidad. A ellos se suman ocho nuevos: cinco específicos del copiloto que nacen con la reformulación, y tres más (uno general y dos de IA) que se consolidaron durante el desarrollo, cuando decisiones tomadas en las reuniones de dirección y en la revisión externa del diseño demostraron ser en realidad requisitos del sistema.

3.2.1. Requisitos no funcionales generales

RNF-01 — Interfaz diseñada para escritorio. Durante las entrevistas con nutricionistas se constató que tanto los profesionales como sus clientes trabajan desde un portátil durante las consultas. Ninguno de los nutricionistas consultados realizaba gestiones desde el móvil en su entorno laboral. Por este motivo, toda la

interfaz se ha diseñado para una resolución de escritorio (1280 px de ancho mínimo), siguiendo un enfoque *desktop-first*. Esto no excluye el acceso desde otros dispositivos, pero la experiencia principal está optimizada para portátil.

RNF-02 — Aplicación web accesible desde navegador. Al tratarse de una aplicación web, no requiere instalación ni configuración local. Esto es especialmente relevante porque muchos nutricionistas trabajan en varias clínicas distintas y no siempre utilizan el mismo equipo. Acceder desde cualquier navegador garantiza disponibilidad sin dependencia del hardware.

RNF-03 — Interfaz moderna, limpia y profesional. La aplicación va a ser utilizada por profesionales de la salud en su entorno de trabajo, por lo que debe transmitir confianza y profesionalidad. Se tomó como referencia visual Nutrium, por su interfaz limpia y orientada a productividad. El objetivo es que el nutricionista perciba la herramienta como un instrumento profesional, no como un producto genérico.

RNF-04 — Sidebar permanente adaptable. Al tratarse de una herramienta de uso diario, el nutricionista necesita acceder rápidamente a cualquier sección sin perder el contexto de lo que está haciendo. NutriApp ofrece un *sidebar* fijo y siempre visible, con dos modos de visualización que el usuario puede cambiar: un modo expandido en el que cada entrada incluye icono y etiqueta de texto, y un modo compacto en el que solo se muestran los iconos para aprovechar más espacio horizontal en pantalla. Este enfoque, inspirado en aplicaciones como Notion, permite conservar la navegación permanente sin penalizar el espacio disponible para el contenido principal.

RNF-05 — Separación clara entre panel de nutricionista y panel de cliente. Ambos perfiles de usuario tienen necesidades completamente distintas: el nutricionista gestiona múltiples clientes, citas y planes; el cliente interactúa con sus propios datos, su plan y su progreso. Mezclar ambas experiencias en una sola interfaz generaría confusión. Por ello, cada perfil dispone de su propio panel con navegación, funcionalidades y diseño adaptados, aunque ambos paneles comparten la misma base de datos.

RNF-07 — Seguridad y privacidad de datos. La plataforma maneja datos de salud de los clientes, que son especialmente sensibles. El sistema debe contemplar autenticación segura, roles diferenciados y acceso restringido a la información según el perfil del usuario. Por ejemplo, las notas privadas que el nutricionista escribe sobre un cliente no deben ser visibles para dicho cliente en ningún caso. Este requisito se complementa con RNF-IA-4 para el tratamiento específico de datos de salud empleados en la generación de planes por IA.

RNF-08 — Aislamiento multi-tenant verificado. Cada nutricionista solo puede ver y modificar sus propios datos, y cada cliente los suyos. Este aislamiento

no puede depender de que la interfaz no muestre lo ajeno: debe estar garantizado en la propia capa de datos, de forma que manipular una URL o llamar directamente al API devuelva cero resultados. El requisito exige además verificación empírica: el aislamiento se comprueba con baterías de pruebas automatizadas que intentan lecturas y escrituras cruzadas entre cuentas y confirman que ninguna alcanza datos ajenos. Este requisito se consolidó durante el desarrollo a petición de los tutores del trabajo.

3.2.2. Requisitos no funcionales específicos del copiloto IA

Los siete requisitos siguientes recogen restricciones inherentes a la incorporación del copiloto inteligente en el sistema. Los cinco primeros nacen con la reformulación V2; los dos últimos se consolidaron durante el desarrollo, uno a propuesta de la revisión externa del diseño y otro como decisión de las reuniones de dirección. Todos ellos deben cumplirse antes de que cualquier borrador de plan generado por la IA llegue al nutricionista o, por extensión, al cliente.

RNF-IA-1 — Generación acotada en tiempo y sin bloqueo. La generación de un plan no puede dejar la aplicación inutilizable mientras se calcula. El requisito se concreta en dos garantías: la interfaz responde de inmediato (la generación se ejecuta como tarea en segundo plano y el profesional puede seguir trabajando mientras consulta su estado) y el motor de optimización trabaja con un límite de tiempo explícito, calibrado contra los recursos del entorno de producción, transcurrido el cual devuelve la mejor solución encontrada o un diagnóstico de por qué no existe ninguna.

RNF-IA-2 — Validación clínica obligatoria. Toda generación debe pasar por un validador clínico determinista antes de ser entregada al nutricionista. Ningún borrador que incumpla restricciones duras (alergias declaradas del cliente, alimentos prohibidos, rangos de calorías y macronutrientes fuera de tolerancia, incompatibilidades conocidas con patologías registradas) puede llegar a presentarse. Este requisito es una garantía de seguridad clínica y no es negociable: su cumplimiento es condición previa a cualquier mejora de calidad estética o estilística del plan.

RNF-IA-3 — Trazabilidad completa de cada generación. Cada generación queda trazada extremo a extremo: *prompt* enviado, modelo utilizado y su versión concreta, tokens consumidos, latencia y coste de la llamada, restricciones resultantes de la traducción y veredicto del validador clínico. La motivación es doble: auditoría posterior en caso de incidencia clínica y recogida de datos para la evaluación experimental del sistema.

RNF-IA-4 — Protección de datos de salud conforme al RGPD. El tratamiento de los datos de salud de los clientes debe cumplir lo establecido por el Reglamento General de Protección de Datos para las categorías especiales definidas en su artículo 9. Esto incluye consentimiento explícito para el uso de los datos en la

generación por IA, cifrado en tránsito y en reposo, ausencia de retención por parte de los proveedores externos de modelos de lenguaje, y la posibilidad efectiva de ejercer los derechos de acceso, rectificación y supresión.

RNF-IA-5 — Explicabilidad mínima. El sistema debe permitir al nutricionista entender por qué se le propone lo que se le propone. Esto se concreta en tres frentes: las restricciones que gobiernan cada generación son visibles y editables (nada se aplica sin que el profesional lo haya aceptado), cada traducción de lenguaje natural queda registrada con su entrada y su salida, y cuando un plan resulta imposible el sistema señala qué restricciones entran en conflicto en lugar de fallar sin explicación. La motivación académica es garantizar que el copiloto no se comporta como una caja negra, la motivación práctica es que el nutricionista pueda intervenir en el punto exacto en el que el sistema se equivoca.

RNF-IA-6 — Firma profesional obligatoria. Ningún plan generado llega al cliente sin la firma explícita del nutricionista. El sistema puede producir y almacenar tantos borradores como haga falta, pero solo pasará a ser un plan entregable al cliente si el profesional lo autoriza y firma, esa excepción se aplica en la propia capa de datos: para el cliente, los borradores no existen. Este requisito es el que convierte al sistema en un asistente del profesional colegiado y no en un generador autónomo de dietas.

RNF-IA-7 — Independencia del motor respecto al LLM. El motor de generación debe funcionar sin el componente de inteligencia artificial: las restricciones pueden introducirse manualmente por el profesional y el plan resultante es exactamente el mismo. Esta decisión, acordada con los tutores, ordena toda la arquitectura: el modelo de lenguaje es una capa de conveniencia sobre una interfaz estable de restricciones, no una pieza imprescindible, y garantiza que la plataforma siga siendo útil si el modelo no está disponible o su calidad resulta insuficiente para un caso concreto.

3.3. Requisitos funcionales

Los requisitos funcionales describen las acciones concretas que el sistema debe permitir realizar. Se organizan por bloques temáticos, indicando para cada requisito el actor principal que lo ejecuta: el nutricionista (N), el cliente (C) o el propio sistema de forma automática (S). El copiloto IA constituye la única innovación clave del TFG tras la reformulación y se describe en su propio bloque. Los demás bloques se presentan reducidos al subconjunto de requisitos efectivamente implementados; los requisitos pospuestos a trabajo futuro se listan al final del capítulo con referencia al capítulo 8.

3.3.1. Gestión de clientes

La gestión del perfil del cliente es el punto de entrada de información al sistema y, en particular, al copiloto IA: los datos personales, las mediciones antropométricas y las restricciones clínicas y dietéticas registradas aquí son precisamente las variables que condicionan la generación de un plan personalizado. Las notas privadas del nutricionista se mantienen en el alcance porque pueden incluir observaciones sobre el plan del cliente sin añadir complejidad técnica al sistema.

ID	Requisito	Actor
RF-01	Crear, editar y consultar perfiles de clientes con historial clínico, datos personales, objetivos y mediciones.	N
RF-02	Buscar y filtrar clientes por nombre y por estado (activo, inactivo, nuevo).	N
RF-03	Escribir notas privadas asociadas a cada cliente, visibles únicamente para el nutricionista. Pueden incluir observaciones sobre el plan o el seguimiento.	N
RF-04	Editar datos personales y registrar mediciones corporales.	C
RF-05	Consultar las mediciones y la evolución de cada uno de sus clientes.	N
RF-60	Acceder a una ficha detallada por cliente que consolide datos básicos, notas privadas, plan actual, próxima sesión y adherencia reciente en una sola pantalla.	N

Tabla 3.3: Requisitos funcionales — Gestión de clientes.

3.3.2. Calendario y citas online

El módulo de calendario se reduce respecto a V1 para eliminar el modelo de multi-clínica y los recordatorios automáticos. Todas las citas son en modalidad *online*: el nutricionista configura su disponibilidad y el cliente puede solicitar, cancelar o reprogramar sus propias citas. La justificación de este recorte es doble: por un lado, al desaparecer la entidad *clínica* se simplifica drásticamente el modelo de datos; por otro, los recordatorios automáticos dependían de un motor de notificaciones programadas cuyo único otro uso (RF-41, RF-42) también sale del alcance.

ID	Requisito	Actor
RF-06	Gestionar una agenda de citas con vista diaria, semanal y mensual.	N
RF-09	Configurar la disponibilidad horaria para que los clientes puedan solicitar citas.	N
RF-10	Ver citas programadas, solicitar nuevas según disponibilidad, y cancelar o reprogramar citas existentes.	C

Tabla 3.4: Requisitos funcionales — Calendario y citas online.

3.3.3. Mensajería

La mensajería queda reducida al chat directo entre nutricionista y cliente con soporte de adjuntos. Las videollamadas y llamadas de voz, así como el chat entre clientes del mismo nutricionista, pasan a trabajo futuro. El chat es imprescindible para la coherencia de la plataforma como herramienta de seguimiento profesional, pero no contribuye directamente al copiloto IA.

ID	Requisito	Actor
RF-12	Ofrecer un chat de mensajería directa entre nutricionista y cliente, con historial completo de conversaciones.	N/C
RF-13	Permitir adjuntar archivos (documentos, imágenes, PDFs) en los mensajes.	N/C

Tabla 3.5: Requisitos funcionales — Mensajería.

3.3.4. Planes nutricionales: estructura base

Este bloque recoge la infraestructura básica de planes nutricionales sobre la que opera el copiloto IA del bloque siguiente. Un plan es una estructura de comidas organizada por día, compuesta por alimentos de la base de datos con su cantidad en gramos. El cliente consulta su plan en formato semanal. No se incluyen en V2 las plantillas reutilizables (RF-28), dado que el copiloto IA genera planes a medida por cliente y el uso de plantillas manuales deja de tener sentido; tampoco la sustitución por parte del cliente (RF-59), pospuesta para evitar complicar el flujo y su impacto en el análisis de adherencia. Las recetas compuestas (RF-26) y las sustituciones por equivalencia (RF-27), que el catálogo inicial situaba en este bloque, pasan también a trabajo futuro: el motor de generación trabaja con alimentos sueltos (capítulo 5) y ningún componente del sistema las usa.

ID	Requisito	Actor
RF-24	Crear planes nutricionales completos organizados por día y por comida (desayuno, comida, cena, <i>snacks</i>).	N
RF-25	Gestionar una base de datos de alimentos con información nutricional (calorías, macronutrientes, categoría).	N
RF-29	Consultar el plan nutricional actual con vista semanal, desglose por comidas e información nutricional.	C

Tabla 3.6: Requisitos funcionales — Planes nutricionales: estructura base.

3.3.5. Copiloto de planes nutricionales (*innovación clave*)

Este bloque constituye la única innovación clave de NutriApp tras la reformulación. El copiloto se describe sobre un principio rector: el sistema propone y el profesional decide. El modelo de lenguaje no genera dietas; traduce las peticiones del nutricionista a restricciones formales de un vocabulario cerrado, y es un motor de

optimización determinista quien construye el plan a partir de ellas, de modo que las garantías clínicas nunca dependen de un componente probabilístico. La arquitectura técnica completa se describe en el capítulo 4; los requisitos no funcionales asociados son RNF-IA-1 a RNF-IA-5 (sección 3.2).

ID	Requisito	Actor
RF-62	Describir en lenguaje natural las necesidades dietéticas de un cliente y obtener su traducción a restricciones formales del catálogo del sistema.	N
RF-63	Validar cada mensaje antes de traducirlo (idioma, pertinencia nutricional, completitud y contradicciones) y devolver avisos accionables en lugar de fallos opacos.	S
RF-64	Revisar cada restricción propuesta por la IA y decidir su destino: incorporarla al perfil del cliente, aplicarla solo a la generación en curso o descartarla.	N
RF-65	Generar borradores de plan que satisfagan las restricciones activas mediante un motor de optimización determinista, operativo también sin el componente de IA.	S
RF-66	Auditar cada borrador con un validador clínico determinista e independiente del motor antes de presentarlo al profesional.	S
RF-67	Explicar las generaciones imposibles señalando el subconjunto de restricciones en conflicto, expresado en lenguaje llano con los nombres reales del catálogo.	S
RF-68	Definir el estilo clínico propio del nutricionista como reglas declarativas que se aplican a la generación de todos sus planes.	N
RF-69	Ejecutar las generaciones de forma asíncrona y seguir su estado desde la interfaz, con las tareas fallidas visibles junto a su explicación.	S

Tabla 3.7: Requisitos funcionales — Copiloto de planes nutricionales.

3.3.6. Check-in mínimo de adherencia

El *check-in* diario, planteado en V1 como innovación clave con gamificación, racha de días y hábitos personalizables, se reduce en V2 a un único requisito funcional: el cliente marca diariamente qué comidas del plan ha completado. La razón de mantener esta versión mínima no es estética sino experimental: el porcentaje de adherencia resultante es una de las variables más relevantes para evaluar empíricamente la calidad de los planes generados por el copiloto IA y demostrar ante el tribunal que el sistema produce planes que los clientes efectivamente siguen.

ID	Requisito	Actor
RF-33	Registrar diariamente qué comidas del plan se han completado, para calcular automáticamente el porcentaje de adherencia.	C

Tabla 3.8: Requisitos funcionales — Check-in mínimo de adherencia.

3.3.7. Progreso del cliente

El bloque de seguimiento se reduce a una única vista gráfica que combina la evolución del peso registrado por el cliente y el porcentaje de adherencia acumulado. El *heat map* tipo GitHub y las fotos de progreso se posponen a trabajo futuro: el primero por depender del *check-in* en su versión gamificada, y las segundas por no aportar valor al copiloto IA.

ID	Requisito	Actor
RF-36	Visualizar la evolución del peso del cliente y del porcentaje de adherencia al plan a lo largo del tiempo.	C

Tabla 3.9: Requisitos funcionales — Progreso del cliente.

3.3.8. Base de datos de alimentos

La base de datos de alimentos se reduce a la búsqueda por nombre y a la posibilidad de registrar alimentos personalizados por parte del nutricionista. Se elimina la integración con Open Food Facts (Open Food Facts, 2025). La fuente prevista en su lugar era BEDCA (Agencia Española de Seguridad Alimentaria y Nutrición, 2026) (Base de Datos Española de Composición de Alimentos, de referencia académica en España); finalmente el catálogo se construyó como un conjunto propio de alimentos con sus nutrientes (capítulo 4) y la carga de BEDCA queda como trabajo futuro (capítulo 8). Se posponen también el escaneo de códigos de barras (RF-46), el reconocimiento de alimentos por imagen (RF-47), el registro directo de consumo (RF-49) y la comparativa con objetivos diarios (RF-50). Los alimentos personalizados se conservan porque permiten al nutricionista cubrir los huecos del catálogo con preparaciones o productos locales, y porque el copiloto IA necesita un catálogo completo cuando genera un plan.

ID	Requisito	Actor
RF-45	Consultar una base de datos de alimentos con buscador por nombre, sobre un catálogo propio de alimentos con sus nutrientes.	N/C
RF-48	Añadir alimentos personalizados a la base de datos (recetas propias o productos locales) con sus macronutrientes.	N

Tabla 3.10: Requisitos funcionales — Base de datos de alimentos.

3.3.9. Dashboard de métricas del copiloto

El *dashboard* general del nutricionista original (RF-39 en V1) se reemplaza por un *dashboard* centrado en las métricas del copiloto IA: tasa de aprobación de los borradores al primer intento, *edit distance* media entre el borrador y la versión final entregada, coste acumulado de llamadas a modelos de lenguaje en el mes en curso, número de planes generados y comparativa de adherencia entre clientes con

plan generado por IA y clientes con plan elaborado manualmente. Este cambio de enfoque es coherente con la nueva centralidad del copiloto y aporta, por primera vez en el sector, información cuantitativa sobre el rendimiento del sistema de IA frente a la práctica manual.

ID	Requisito	Actor
RF-39	Visualizar un <i>dashboard</i> con métricas del copiloto IA: tasa de aprobación al primer intento, <i>edit distance</i> media, coste acumulado, planes generados y adherencia IA <i>vs.</i> manual.	N

Tabla 3.11: Requisitos funcionales — Dashboard de métricas del copiloto.

3.4. Requisitos pospuestos a trabajo futuro

Los requisitos del catálogo original que no forman parte del alcance implementado se posponen a trabajo futuro y se recogen en el capítulo 8. A continuación se enumeran agrupados por el bloque al que pertenecen originalmente, con una breve razón de posposición.

Formularios personalizables (RF-14 a RF-17). Editor visual *drag-and-drop* con plantillas y asignación manual o programada. Se posponen al sustituir el concepto de diagnóstico basado en formularios por la variable más directa que alimenta al copiloto: las mediciones y restricciones clínicas registradas en el perfil del cliente.

Sistema de alertas inteligentes (RF-18 a RF-23). Motor de reglas condicionales sobre las respuestas de los formularios, *dashboard* de gestión con prioridades y respuestas rápidas desde la alerta. Aunque este bloque era una de las innovaciones clave en V1, su núcleo conceptual (evaluación automática y bloqueo ante situaciones anómalas) queda absorbido en el validador clínico del copiloto IA (RF-66), que aplica reglas deterministas sobre cada plan generado.

Lista de la compra (RF-30 a RF-32). Generación automática de lista agregada, sustituciones sobre ingredientes no disponibles y marcado de compras. Su implementación es trivialmente derivable de los planes generados por el copiloto, pero no aporta al núcleo IA y se deja fuera del TFG.

Marketplace de recursos educativos (RF-51 a RF-55). Catálogo, publicación, gestión de ventas y biblioteca personal del cliente. Implica un subsistema completo de comercio electrónico (pagos, inventario, reseñas) sin aporte académico al núcleo IA.

Motor de automatizaciones (RF-41, RF-42, RF-56 a RF-58). Envío programado de formularios, notificaciones genéricas y motor de reglas configurables con disparadores y acciones. Su alta complejidad, unida al hecho de que los bloques de

formularios y alertas que disparaban la mayor parte de sus acciones también salen del alcance, justifica su posposición completa.

Ampliación clínica de la ficha del cliente. La revisión externa del diseño con perspectiva clínica propuso ampliar la ficha del cliente con más de veinticinco variables adicionales (patologías codificadas, medicación, bioquímica, hábitos de vida), que permitirían afinar tanto la generación como la validación de los planes. Se registra aquí como requisito identificado y se pospone de forma consciente: el modelo de datos actual cubre las variables que el motor consume hoy, y ampliar la ficha sin que ninguna pieza del sistema explote esos datos añadiría coste sin aportar valor.

Requisitos individuales de otros bloques. La tabla 3.12 recoge los requisitos sueltos propuestos y la razón concreta de cada uno.

3.5. Dependencias funcionales

Los requisitos del alcance V2 no actúan de forma aislada. La tabla 3.13 recoge las funcionalidades finales de NutriApp y los requisitos implicados en cada una. La dependencia más relevante es la del copiloto IA, que articula a su alrededor la mayoría del resto de requisitos funcionales mantenidos.

La lectura cualitativa de la tabla es directa: todos los bloques del alcance convergen en el copiloto IA, ya sea como fuente de datos (perfil, mediciones, base de alimentos, plan base), como mecanismo de entrega (estructura de plan, vista del cliente) o como instrumento de medición de su calidad (adherencia, *dashboard*). La estructura aplanada respecto a V1 es consecuencia directa de haber eliminado las funcionalidades autónomas que no contribuían al copiloto.

3.6. Funcionalidades resultantes

A partir de los requisitos V2 definidos y sus dependencias, las funcionalidades finales de NutriApp se agrupan en las siguientes áreas. La única etiqueta de *innovación clave* se reserva para el copiloto IA; el resto son funcionalidades base necesarias para la coherencia de la plataforma.

Gestión integral del cliente (RF-01 a RF-05, RF-60). Perfiles con historial clínico, mediciones y objetivos; búsqueda y filtrado por nombre y estado; notas privadas del nutricionista; consulta de la evolución; ficha detallada por cliente. Sustituye la combinación de cuaderno físico, notas del móvil y plataforma de la clínica.

Agenda de citas online (RF-06, RF-09, RF-10). Vistas diaria, semanal y mensual; configuración de disponibilidad horaria por el nutricionista; solicitud, cancelación y reprogramación de citas por parte del cliente. Todas las citas son en modalidad *online*.

ID	Requisito	Razón de posposición
RF-07	Colores por tipo de cita.	No aplica: todas las citas son <i>online</i> .
RF-08	Multi-clínica.	Se elimina la clínica como entidad; arquitectura <i>multi-tenant</i> por nutricionista.
RF-11	Recordatorios automáticos de citas.	Depende del motor de automatizaciones, fuera del alcance.
RF-26	Recetas compuestas por ingredientes.	El motor trabaja con alimentos sueltos; las tablas de receta existen en el modelo de datos, sin interfaz ni uso.
RF-27	Sustituciones por equivalencia de macronutrientes.	Ningún componente las modela; el cliente tampoco sustituye comidas (RF-59).
RF-28	Plantillas de plan reutilizables.	El copiloto IA sustituye el uso manual de plantillas.
RF-34	Racha de <i>check-in</i> gamificado.	La versión mínima de <i>check-in</i> mide adherencia, no gamifica.
RF-35	Hábitos personalizables por el nutricionista.	Idem.
RF-37	<i>Heat map</i> de cumplimiento.	Depende del <i>check-in</i> completo.
RF-38	Fotos de progreso.	No contribuye al copiloto.
RF-40	Exportación a PDF de informes.	No contribuye al copiloto.
RF-43, RF-44	Videollamadas y llamadas de voz.	Requieren integración con proveedores externos; alto coste por bajo valor añadido al TFG.
RF-46	Escáner de códigos de barras.	Integración externa sin aporte al copiloto.
RF-47	Reconocimiento de alimentos por foto con IA.	Línea de IA paralela fuera del foco del TFG.
RF-49	Registrar alimento consumido.	Depende del <i>check-in</i> completo.
RF-50	Comparar alimento con objetivos diarios.	Idem.
RF-59	Sustitución de comida por el cliente.	Complica el flujo y contamina la métrica de adherencia.
RF-61	Chat entre clientes del mismo nutricionista.	Subsistema de mensajería peer-to-peer fuera del alcance.

Tabla 3.12: Requisitos individuales pospuestos a trabajo futuro.

Funcionalidad	Requisitos involucrados
Gestión integral del cliente	RF-01, RF-02, RF-03, RF-04, RF-05, RF-60
Agenda de citas <i>online</i>	RF-06, RF-09, RF-10
Mensajería profesional	RF-12, RF-13
Estructura base de planes nutricionales	RF-24, RF-25, RF-29
Base de datos de alimentos (catálogo propio + personalizada)	RF-25, RF-45, RF-48
Copiloto IA de planes*	RF-01 + RF-24 + RF-25 + RF-45 + RF-48 + RF-62 a RF-69
Seguimiento de adherencia	RF-29, RF-33, RF-36
<i>Dashboard</i> del nutricionista	RF-39

Tabla 3.13: Dependencias funcionales de NutriApp V2. La funcionalidad marcada con * es la única innovación clave del TFG.

Mensajería profesional (RF-12, RF-13). Chat directo nutricionista-cliente con historial completo y adjuntos.

Estructura base de planes nutricionales (RF-24, RF-25, RF-29). Planes organizados por día y por comida, base de alimentos con información nutricional y vista semanal del plan para el cliente. Este bloque define la estructura de datos sobre la que opera el copiloto IA.

Copiloto de planes nutricionales (RF-62 a RF-69). *Innovación clave*. Sistema de generación semi-automática de planes: el nutricionista describe lo que necesita en su idioma, la inteligencia artificial lo traduce a restricciones formales, un motor de optimización construye el borrador y un validador clínico independiente lo audita. El profesional arbitra cada restricción propuesta y siempre aprueba el plan antes de entregarlo; su estilo clínico, declarado una vez, se aplica a todas sus generaciones. La arquitectura técnica se describe en el capítulo 4.

Base de datos de alimentos (RF-45, RF-48). Búsqueda por nombre sobre el catálogo propio y extensión con alimentos personalizados por el nutricionista (recetas propias, productos locales). Sirve de catálogo tanto para el constructor manual de planes como para el copiloto IA.

Check-in mínimo de adherencia (RF-33). El cliente marca qué comidas del plan ha completado cada día. El sistema calcula automáticamente el porcentaje de adherencia, que es la métrica principal con la que se evaluará la calidad de los planes generados por el copiloto.

Progreso del cliente (RF-36). Gráfico de evolución combinado de peso y porcentaje de adherencia a lo largo del tiempo.

Dashboard de métricas del copiloto (RF-39). Panel del nutricionista con las métricas de rendimiento del copiloto: tasa de aprobación al primer intento, *edit distance* media entre borrador y versión final, coste económico de las llamadas a modelos de lenguaje, número de planes generados y comparativa de adherencia de los clientes con plan IA frente a los clientes con plan manual.

Capítulo 4

Diseño del sistema

Este capítulo describe el diseño del sistema: la arquitectura que conecta sus piezas, el modelo de datos, la seguridad que aísla a cada usuario, la capa de servicio que persiste y firma los planes, el despliegue y la manera de verificar cada parte. Los dos componentes con más peso propio, el motor de generación y el copiloto de IA, tienen sus capítulos aparte (5 y 6); aquí se explica la aplicación sobre la que trabajan. Cada tecnología se justifica con los motivos por los que se eligió en la capa donde se usa.

La sección 4.1 muestra la arquitectura y la decisión que ordena todo el diseño: una tabla de restricciones como interfaz entre quien las produce y quien las consume. Las siguientes cuatro secciones explican las capas una a una, desde los datos al despliegue, y la sección 4.6 muestra cómo se verifica cada bloque de trabajo. Las pruebas y sus resultados se enseñan en el capítulo 7.

4.1. Arquitectura general

La aplicación tiene tres piezas principales: un frontend web con Next.js (Vercel, Inc., 2026), una base de datos PostgreSQL (The PostgreSQL Global Development Group, 2026) gestionada a través de Supabase (Supabase, Inc., 2026) y un backend propio en Python. Además, se suma el worker de generación, el proceso que ejecuta los trabajos de IA. Durante el desarrollo corre en mi propio ordenador, porque ahí vive el modelo de lenguaje local; la sección 4.5 explica ese reparto y sus alternativas. La figura 4.1 enseña el conjunto y cómo se comunican las piezas.

El reparto lo acordé con los tutores antes de empezar a programar, con un criterio y orden: cada pieza debía ser la opción más simple que soportara los requisitos del capítulo 3, y la construcción iría de los datos hacia la interfaz (primero la base de datos, después su conexión y luego las pantallas), para que cada capa se apoyara sobre una anterior ya verificada.

El backend de generación es un servicio propio con FastAPI (Ramírez, 2026). La razón principal es que tanto el solver del capítulo 5 como el ecosistema de acceso a modelos de lenguaje viven de forma natural en Python; y también porque el solver necesita un proceso con límites de tiempo largos, lo cual encaja mal en las funciones

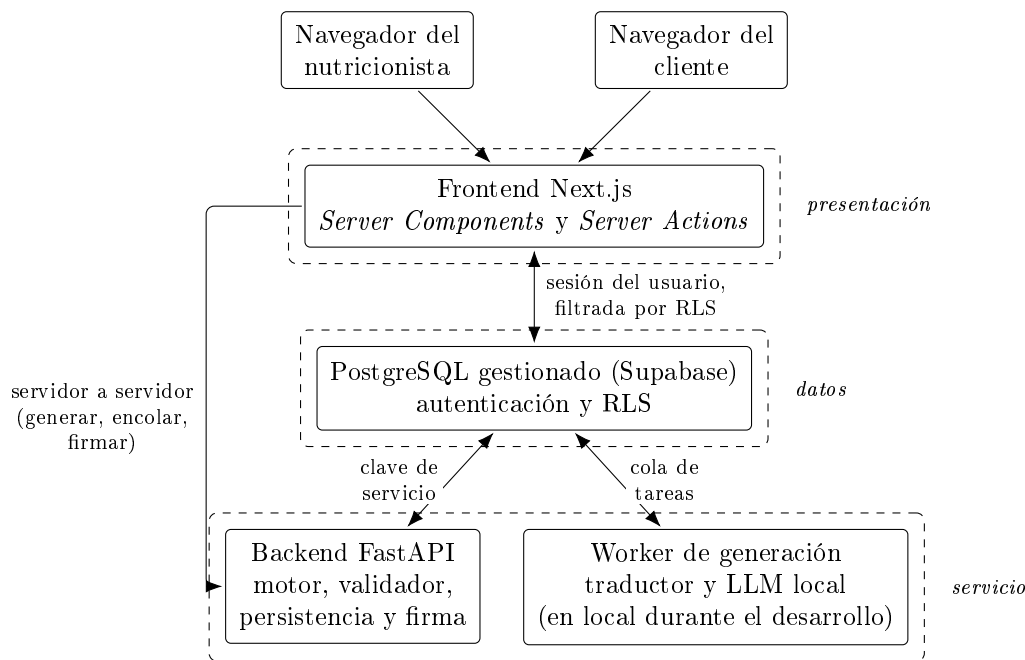


Figura 4.1: Arquitectura del sistema: la capa de presentación, la de datos y la de servicio, con el worker de generación como pieza que corre en un ordenador local.

serverless de las plataformas donde se despliega el frontend. Además, este servicio es el único que guarda la clave privilegiada de la base de datos, que nunca viaja al navegador (sección 4.3).

La capa de presentación usa Next.js con *Server Components*: las páginas se montan en el servidor y leen directamente de la base de datos bajo la sesión del usuario, con las políticas de seguridad de la sección 4.3 como filtro. Así desaparece la capa de API intermedia para casi toda la aplicación. Las escrituras sencillas (dar de alta un alimento, añadir una restricción) siguen el mismo camino con *Server Actions*, funciones que corren en el servidor y validan la entrada antes de tocar los datos. Las operaciones que exigen más garantías como generar, encolar y firmar planes se delegan en el backend con una llamada de servidor a servidor autenticada por un secreto compartido, como se detalla en la sección 4.4.

Para el sistema de diseño seguí la recomendación de uno de mis directores: Radix UI (WorkOS, 2026), como base de componentes accesibles sin estilos propios, y Tailwind CSS (Tailwind Labs, 2026) para aplicarles la paleta y tipografía del prototipo de Figma que servía de referencia.

Por encima del reparto en capas hay una decisión que ordena todo el sistema: las restricciones dietéticas viven en una tabla propia, **diet_constraint**, con un vocabulario cerrado de 16 tipos, y esa tabla es la interfaz entre quien produce las restricciones (el formulario de la ficha del cliente, el formulario del estilo clínico del nutricionista o el traductor de texto libre del capítulo 6) y quien las consume (el motor de generación). La figura 4.2 recoge el esquema.

Este vocabulario se concluyó antes de integrar ningún modelo de lenguaje: primero se construyó y verificó el motor con restricciones escritas a mano, y luego se

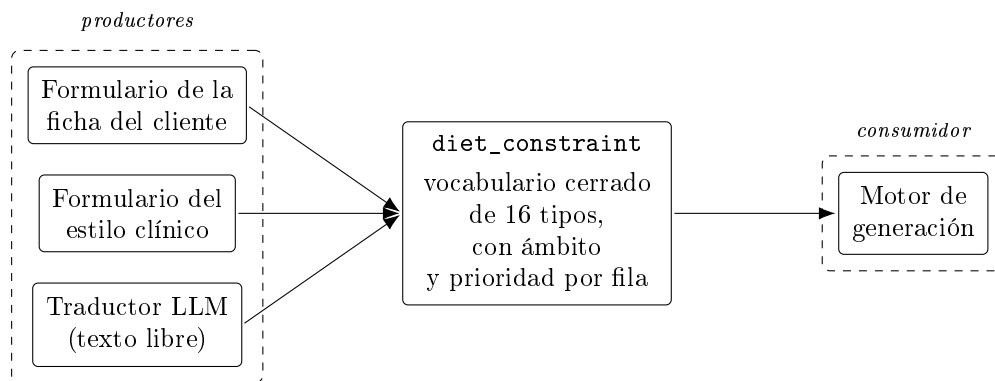


Figura 4.2: La tabla de restricciones como interfaz: tres productores escriben filas del mismo vocabulario y un único consumidor las aplica sin distinguir su origen.

integró la capa de IA. La vía manual quedó como respaldo: si el traductor falla o el profesional prefiere escribir la restricción él mismo, el formulario alimenta al motor exactamente igual. Los 16 tipos del vocabulario se describen en la sección 5.4.

Del resto de decisiones de la capa de presentación, las que más condicionaron el desarrollo se resumen en cuatro:

- Un único inicio de sesión para los dos roles: la aplicación lleva a cada usuario a su panel, y el panel del cliente vive bajo su propio prefijo de rutas, con guardas equivalentes en cada sección.
- Los 13 tipos de restricción se pueden dar de alta desde la interfaz mediante un único formulario que sirve en tres contextos distintos (la ficha del cliente, el estilo clínico y la pantalla de generación) cambiando sus propiedades.
- Las pantallas que esperan datos nuevos se actualizan por sondeo (cada 15 segundos en la mensajería, cada 4 durante una generación), no es necesario gastar recursos en mantener una conexión abierta, esos tiempos son bastante aceptables.
- Los totales nutricionales de un plan se calculan en la presentación, agregando el modelo alimento-nutriente de la sección 4.2 y avisando cuando falta algún dato de composición.

4.2. Capa de datos

La base de datos es PostgreSQL gestionado a través de Supabase. Fue la recomendación que me dieron mis tutores, y además cumplía con las tres necesidades del catálogo de requisitos: autenticación integrada para resolver los dos roles de la aplicación (RNF-05), políticas de seguridad a nivel de fila para aislar los datos de cada nutricionista en la propia base de datos (RNF-08), y alojamiento en la Unión Europea (la instancia está en Fráncfort), coherente con la protección de datos de salud (RNF-IA-4).

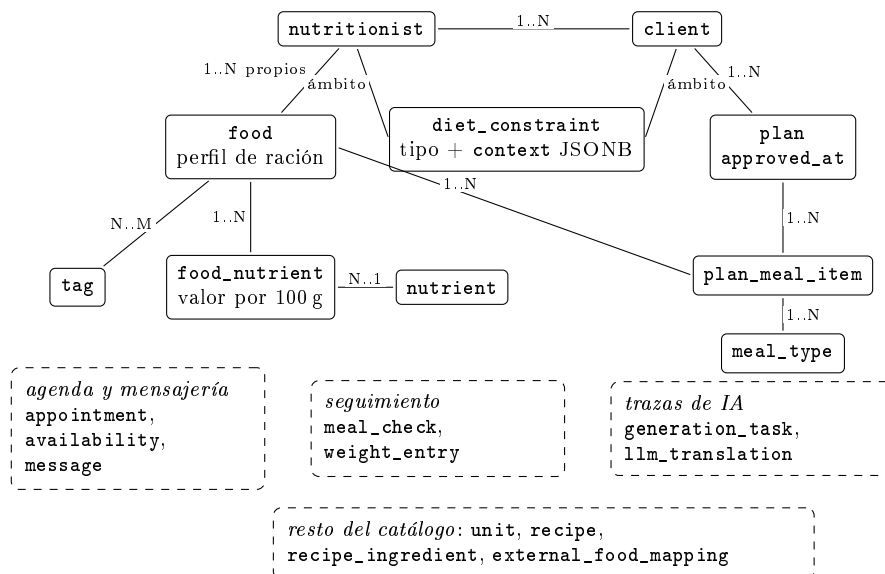


Figura 4.3: Modelo de datos: las tablas del núcleo con sus relaciones y el resto del esquema agrupado por dominios. Las restricciones apuntan además a su objetivo (un alimento, una etiqueta o un nutriente).

El esquema tiene 22 tablas. La figura 4.3 enseña las que soportan el núcleo de la aplicación, con el resto agrupado por dominios: la agenda y la mensajería, el seguimiento que registra el cliente y las trazas de los trabajos de IA. El esquema describe el diseño final (el actual), tras todas las decisiones que fueron moldeando el proyecto.

La composición nutricional vive en una tabla propia, **food_nutrient**, con una fila por alimento y nutriente (el valor por cada 100 g), en vez de en columnas planas de la tabla de alimentos. Así, añadir un nutriente nuevo es una fila más en **nutrient**, sin tocar el esquema, a cambio de agregar al leer los totales.

Los planes son dos tablas: **plan** y **plan_meal_item**, con una fila por alimento en cada comida de cada día. No hay tabla intermedia de comidas: la jerarquía días-comidas-alimentos se reconstruye al presentar. Tampoco hay columna de estado: un plan es borrador hasta que tiene fecha de firma (**approved_at**), y esa fecha la escribe solo la capa de servicio (sección 4.4).

El catálogo global y los alimentos propios de cada nutricionista comparten la tabla **food**: un alimento con dueño es privado y uno sin dueño es de todos. Y los argumentos heterogéneos de las restricciones (la cantidad y unidad de un límite, la etiqueta de una exclusión) van en una columna JSONB dentro del esquema fijo de **diet_constraint**: los 16 tipos comparten tabla y difieren en el contenido de esa columna, que la aplicación valida antes de escribir.

Del resto de decisiones de modelado, las específicas de esta aplicación:

- Claves primarias naturales compuestas donde el propio esquema debe impedir duplicados: una marca de comida por plan, día y comida, y un peso por cliente y día.

- La adherencia no se guarda: se calcula a partir de las marcas al leerla, con el mismo criterio en los dos paneles.
- El vínculo entre la ficha del cliente y su cuenta de acceso admite nulos y se rompe sin borrar la ficha si la cuenta desaparece; un disparador crea la fila de perfil que toca al registrarse una cuenta, según su rol.
- Las familias de alimentos son etiquetas, con el dato ausente permitido: un alimento sin clasificar no rompe nada, simplemente no participa en las reglas que dependen de su familia.

El capítulo 5 distingue dos familias de restricciones, y cada una vive en un sitio distinto de la base de datos. El vocabulario clínico son las filas de `diet_constraint` y su enumeración de 16 tipos: crece con una migración del esquema. La plausibilidad alimentaria son datos del catálogo: las etiquetas de cada alimento (familias, alérgenos, franjas) y su perfil de ración (gramos mínimos y máximos por aparición, gramos por pieza). Dar de alta un alimento con su perfil y sus etiquetas amplía lo que el motor sabe hacer sin tocar ni el esquema ni el código.

La capa de acceso desde el frontend impone dos límites que condicionaron el diseño. Las mutaciones desde el navegador van fila a fila, sin transacción que las agrupe: el alta de un alimento con su composición se hace en dos pasos y, si el segundo falla, el primero se deshace con un borrado de compensación. Y las lecturas cortan a un número máximo de filas por consulta, así que los totales de macros se agregan plan a plan en vez de pedirse de golpe. Las operaciones que no toleran estos límites se llevan al backend, que escribe en transacción (sección 4.4).

4.3. Capa de seguridad

La seguridad tiene tres líneas. La sesión viaja en una cookie e identifica al usuario en cada petición. Las guardas de cada sección comprueban el rol y redirigen si no corresponde. Y la última palabra la tiene la base de datos: cada tabla lleva políticas de fila (RLS) que filtran por el usuario de la sesión, así que forzar la URL de un cliente ajeno devuelve cero filas; el filtro lo aplica la propia base de datos, no la pantalla.

Los dos roles conviven sobre las mismas tablas solo con políticas. Como las políticas permisivas se combinan con O lógico, el panel del cliente se añadió escribiendo sus políticas de lectura sin tocar las del nutricionista. Cuando una operación mezcla columnas del actor con columnas que no le pertenecen (el cliente marca una comida, pero el plan es del nutricionista), la política no basta: la escritura pasa por una función con privilegios del sistema que valida la fila y escribe solo lo permitido.

Dos criterios atraviesan todas las políticas. El primero es qué dato puede autorizar: nunca uno que el propio usuario pueda editar; el vínculo entre cuenta y ficha vive en una tabla que solo escribe el servidor. El segundo es que la política también encierra reglas de negocio: el cliente no puede marcar comidas de días futuros ni de comidas que no están en su plan, y los campos que dicen quién envió un mensaje los fija el servidor.

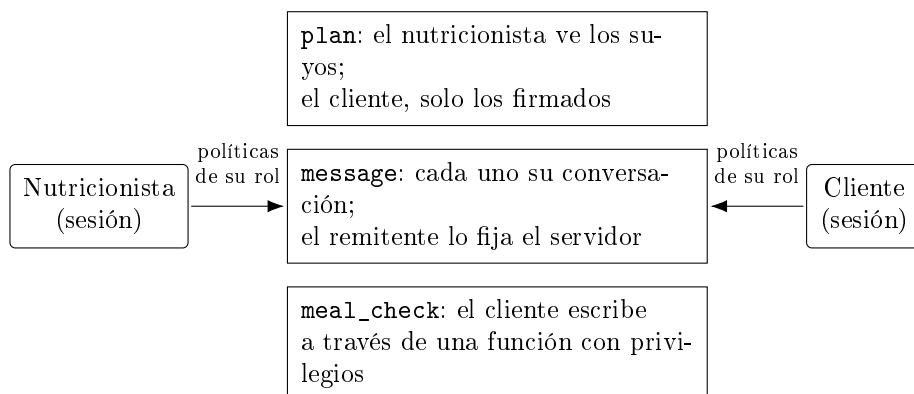


Figura 4.4: Dos roles sobre las mismas tablas: cada uno entra con su sesión y ve o escribe lo que sus políticas permiten, con tres tablas como ejemplo.

La firma es además una frontera de visibilidad: para el cliente solo existen los planes firmados. Esa regla vive en la política de lectura de **plan**, así que un borrador es invisible para el cliente aunque ninguna pantalla lo compruebe. La figura 4.4 resume el reparto.

Las claves siguen el mismo reparto: la clave pública que viaja al navegador solo puede lo que la RLS permite, y la clave de servicio, que se salta las políticas, no sale del backend. Aun operando con privilegios, el backend vuelve a comprobar que el cliente pertenece al nutricionista antes de generar o firmar. Cada bloque de desarrollo que tocó la seguridad cerró con una tanda de comprobaciones multi-actor (leer y escribir como un nutricionista, como otro y como un cliente), acumuladas en un banco que se ejecuta con un comando; sus resultados se enseñan en el capítulo 7.

4.4. Capa de servicio

La capa de servicio son el backend FastAPI y el worker de generación. La API es pequeña: generar un plan, encolar un trabajo de IA y firmar. Cada endpoint es una capa fina sobre las funciones puras de los capítulos 5 y 6: lee los datos del cliente, llama a la función y persiste el resultado. La respuesta distingue los dos casos del motor (un plan con sus métricas, o el conflicto explicado), y el frontend la enseña tal cual.

La persistencia es el motivo de que estas operaciones no usen la vía del navegador. Guardar un plan generado son decenas de filas en dos tablas, y aquí sí hay transacción: o se escriben todas o ninguna, mientras que las mutaciones del frontend van fila a fila (sección 4.2).

Firmar es la operación con más peso de diseño y la más simple de código: poner la fecha en **approved_at**. Antes de hacerlo, el endpoint vuelve a validar el plan y la propiedad del cliente; como esa fecha solo la escribe el backend, un plan firmado siempre pasó por la re-validación. Con la política de visibilidad de la sección 4.3, firmar es el acto con el que el profesional convierte su borrador en el plan que el cliente ve: la regla de que el sistema propone y el profesional decide (capítulo 3)

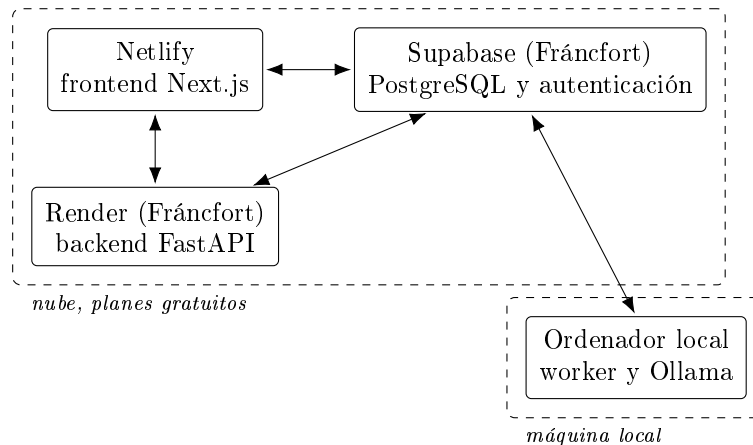


Figura 4.5: Despliegue: tres patas en planes gratuitos de nube y el worker en una máquina local, todas comunicadas a través de la base de datos o de llamadas de servidor a servidor.

queda escrita en la capa de datos.

Los trabajos de IA no se ejecutan durante la petición: la API deja una fila en la cola `generation_task` y responde al momento con su identificador. El worker reclama tareas pendientes con un bloqueo que impide que dos procesos cojan la misma (en SQL, `FOR UPDATE SKIP LOCKED`), ejecuta el mismo pipeline que el camino síncrono y escribe el resultado en la propia fila; el frontend consulta el estado por sondeo bajo RLS. Se usó la base de datos como cola para evitar una pieza de infraestructura nueva, y el estado de cada tarea (en cola, en curso, terminada o fallida) se consulta como cualquier otro dato.

4.5. Despliegue

El sistema desplegado tiene cuatro patas: el frontend en Netlify (Netlify, Inc., 2026), la base de datos en Supabase (Fráncfort), el backend en Render (Render, 2026) (también Fráncfort) y el worker de generación en un ordenador local. Todo el alojamiento es gratuito. La figura 4.5 enseña el reparto.

El worker corre en local por el modelo de lenguaje. La decisión de privacidad del capítulo 6 fija que el dato clínico no sale hacia ningún proveedor de IA, así que el modelo se ejecuta con Ollama (Ollama, 2026) en una máquina propia; y como el alojamiento gratuito no puede correr un modelo de lenguaje, esa máquina es, durante el desarrollo y la evaluación, mi propio ordenador, que hace como si fuese el ordenador del profesional. Si el worker está apagado, la aplicación funciona entera y las tareas de IA esperan en la cola. En un despliegue real la misma pieza podría correr en un servidor de la clínica o cambiar a un motor remoto con garantías contractuales; queda como trabajo futuro en el capítulo 8.

Los límites del alojamiento gratuito están asumidos y medidos. El plan gratuito de Render suspende el backend tras unos minutos sin tráfico: no afecta a una sesión de trabajo normal, pero la primera petición tras una pausa larga paga el arranque

en frío. Por su parte, la base de datos gratuita de Supabase se pausa si el proyecto pasa una semana entera sin uso, y hay que reactivarla desde su panel. El límite de tiempo del solver se calibró contra la CPU de producción, bastante más lenta que la de desarrollo. Y el paralelismo del solver se configura por entorno, porque la memoria del plan gratuito no soporta la configuración de desarrollo (ocho hilos en local, dos en producción). Las medidas se enseñan en el capítulo 7.

Como el prototipo es público, lleva mitigaciones acordes a su carácter académico: un aviso de uso académico en la propia interfaz, datos de demostración con seudónimos y las credenciales fuera del repositorio. El repositorio es privado y sin protección de rama, una limitación asumida para un proyecto de una persona.

4.6. Validación y testing

El desarrollo fue iterativo por bloques (la base de datos, su conexión, el frontend, el motor, la capa de IA, el panel del cliente), y cada bloque cerró con su propia verificación. Esas verificaciones quedan como bancos de pruebas permanentes que se re-ejecutan al cerrar los bloques siguientes, para detectar regresiones al seguir construyendo.

Hay tres familias de verificación. Los bancos deterministas del motor y del validador, con casos contruidos a mano y veredictos esperados. Los runners de seguridad multi-actor, que entran como dos nutricionistas y un cliente y comprueban lectura y escritura tabla a tabla (sección 4.3). Y una prueba de extremo a extremo contra el despliegue real, que genera, valida, persiste y firma un plan en producción. Los resultados se enseñan en el capítulo 7.

El prototipo de Figma sirvió de referencia de la interfaz, y el código se fue adaptando a él según se programaba: algunas cosas se quitaron y otras se añadieron sobre la marcha, y a mitad del desarrollo revisé las diferencias acumuladas para poner al día ambos lados. Los cambios relevantes ya quedan justificados al explicar cada decisión; el resto responde a la versión mínima acordada con los tutores: la aplicación solo enseña lo que tiene una tabla detrás.

Quedan limitaciones asumidas: pantallas con recortes frente al diseño, datos de siembra con fechas relativas que caducan, y la deuda de diseño anterior a la reformulación que la auditoría hizo reducir. Se retoman como trabajo futuro en el capítulo 8.

Capítulo 5

El motor de generación

Este capítulo describe el motor de generación de planes: el componente que, a partir del perfil de un cliente y de un conjunto de restricciones, construye un borrador de plan nutricional que las cumple o explica por qué no existe ninguno que pueda cumplirlas. Funciona sin depender de ningún modelo de lenguaje: las restricciones le llegan ya estructuradas, tanto si las escribió el nutricionista en un formulario como si las propuso el traductor del capítulo 6. Esa independencia se acordó con los tutores y ordenó todo el desarrollo: primero un motor que funciona por sí solo, después la capa de IA por encima.

La sección 5.1 presenta brevemente el paradigma en el que se apoya el capítulo, y la sección 5.2 plantea el problema y su formulación. Las secciones 5.3 y 5.4 recorren las dos familias de restricciones: las que el modelo aplica siempre y las que configura el profesional. La sección 5.5 recoge la función objetivo y la estrategia de resolución, y las dos últimas cubren el diagnóstico de infactibilidad y el validador que audita cada plan. El cuerpo del capítulo se queda con las decisiones y las ecuaciones que las sustentan; la formalización completa, con las 15 reglas y los 16 tipos está en el Anexo B. Las pruebas del motor y las medidas de rendimiento se enseñan en el capítulo 7.

5.1. La programación con restricciones

La programación con restricciones es un paradigma declarativo: en lugar de programar los pasos que construyen una solución, se describe qué condiciones debe cumplir. Un problema se plantea como un modelo con tres piezas: unas variables de decisión (las incógnitas), un dominio para cada una (los valores que puede tomar) y un conjunto de restricciones que las relacionan. Un solver genérico explora los dominios buscando una asignación de valores que satisfaga todas las restricciones; el programa no dice cómo buscarla.

Las restricciones pueden ser de dos clases. Las duras definen qué es una solución válida: el solver descarta cualquier asignación que incumpla alguna. Las blandas expresan preferencias, y se agregan en una función objetivo que mide la calidad de cada solución, para que el solver busque, entre las válidas, la mejor.

Para este problema el paradigma tiene una ventaja más: el modelo se construye antes de cada resolución, así que se puede decidir en ese momento qué restricciones entran. El esqueleto es el mismo para todos los clientes, y sobre él cada generación añade las del cliente concreto: sus alergias, su objetivo calórico, sus preferencias. De esa modularidad sale el catálogo de restricciones de la sección 5.4, que el profesional combina sin que haya que tocar el motor.

5.2. Formulación del problema

El problema se puede enunciar en una frase: dados un cliente, una duración en días, un número de comidas por día y un conjunto de restricciones, decidir qué alimentos aparecen en cada comida y en qué cantidad, cumpliendo todas las restricciones obligatorias y acercándose lo máximo posible a las preferencias.

5.2.1. El problema y su contrato

El resultado es un borrador editable, no un plan cerrado: el nutricionista lo revisa, lo ajusta y decide si lo firma. De ese posicionamiento salen tres propiedades que atraviesan todo el modelo: el plan especifica qué restricciones aplicó, el profesional puede reducir las suyas si lo considera necesario, y cuando el problema no tiene solución el motor explica el conflicto en vez de fallar sin más.

En la implementación, el motor es una función pura: recibe todos sus datos como argumentos y no consulta la base de datos ni escribe en ella. Esto permite probarlo de forma aislada, con casos contruidos a mano, antes de integrarlo en la aplicación. La lectura de datos y la persistencia del plan corresponden al capítulo 4.

La salida distingue dos casos. Si existe un plan que cumple todas las restricciones obligatorias, se devuelve el plan (días, comidas y alimentos con sus gramos) junto con las métricas de la resolución y los avisos no bloqueantes que hayan surgido. Si no existe, se devuelve un subconjunto de restricciones en conflicto acompañado de una sugerencia legible y la lista de restricciones que se pueden reducir (sección 5.6).

Cuatro cosas quedan fuera a propósito. El motor no consulta ningún modelo de lenguaje, y quién produjo las restricciones le resulta indiferente. No valida clínicamente el resultado: esa auditoría es un paso posterior e independiente (sección 5.7). No decide la estructura del plan: el número de días y de comidas son parámetros de entrada. Y no compone recetas ni modela sustituciones entre alimentos: trabaja con alimentos sueltos, y el plan se expresa en alimentos con sus gramos. El estado real de esos dos requisitos se trata en los capítulos 3 y 8.

5.2.2. Elección de tecnología

El enfoque general se acordó con uno de mis directores del TFG, como comenté en el plan de trabajo: un solver de restricciones que construya el plan y un diagnóstico de infactibilidad cuando no exista. Para implementarlo, me indicó que probablemente la mejor opción fuese CP-SAT, el solver de programación con restricciones sobre dominios enteros de la biblioteca OR-Tools de Google (Perron y Furnon, 2024).

CP-SAT tenía dos ventajas. La primera es de integración: se usa como una biblioteca más de Python, así que el modelo convive con la carga de datos y con el resto del backend en el mismo lenguaje y el mismo proceso, sin un lenguaje de modelado intermedio. La otra es que CP-SAT permite declarar las restricciones como asunciones y, si el problema no tiene solución, señala qué subconjunto de ellas lo hace imposible. De esta forma, se construye el diagnóstico de la sección 5.6, para convertir el fallo del solver en una explicación útil para el profesional.

La notación del resto del capítulo es independiente de la herramienta; la implementación es CP-SAT.

5.2.3. Conjuntos y parámetros

Se definen cuatro conjuntos base:

- $F = \{\text{alimento}_1, \dots, \text{alimento}_n\}$: los alimentos del catálogo visible para la generación (los globales más los propios del nutricionista). Cada alimento lleva su composición nutricional por cada 100 g, un conjunto de etiquetas (familias, alérgenos, roles y franjas) y un perfil de ración con sus gramos mínimos y máximos por aparición.
- $D = \{1, \dots, \text{duration_days}\}$: días del plan.
- $M = \{1, \dots, \text{meals_per_day}\}$: comidas de un día. Cada comida m lleva asociado un código de franja $c(m)$ (desayuno, comida, cena, merienda).
- $N = \{\text{energía}, \text{proteína}, \text{hidratos}, \dots\}$: los códigos de nutriente presentes en el catálogo (grasa, sodio, calcio y el resto de nutrientes cargados).

Internamente, cada elemento de estos conjuntos se corresponde con un entero: el modelo indexa alimentos, días, comidas y nutrientes por posición, y los nombres solo existen fuera del solver. El tamaño del problema queda fijado por $|F| \cdot |D| \cdot |M|$. La duración y el número de comidas son la dimensión del problema, no algo que el solver decida: entran como argumentos de la función.

Antes de construir el modelo se precaculan en Python los parámetros de entrada. Los que hacen falta para seguir el capítulo son:

- $\text{kcal100}(f)$ y $\text{nut100}(f, n)$: energía y valor del nutriente n por 100 g del alimento f , escalados a entero (la escala se explica más abajo). Si el alimento no tiene dato para un nutriente, vale 0.
- min_f y max_f : gramos mínimos y máximos por aparición del alimento, derivados de su perfil de ración (sección 5.3.2).
- $\text{floor}(c)$: suelo calórico diario de seguridad del cliente, y las cotas diarias de proteína derivadas de su peso (sección 5.3.1).
- $\text{miembros}(t)$: para una etiqueta t , el subconjunto de alimentos que la llevan. Es la pieza con la que las restricciones razonan sobre familias enteras: prohibir la etiqueta `lactose` prohíbe todos sus miembros.

- Los umbrales estructurales del modelo (máximo de alimentos por comida, mínimos de variedad, y los demás que irán apareciendo), todos constantes con nombre en la configuración del solver. El listado completo con sus valores está en el Anexo B.

5.2.4. Variables de decisión

Al plantear las variables consideré dos formulaciones. La primera era una única variable entera de gramos por cada alimento, día y comida, donde el cero significaba que el alimento no está. La segunda, conseguía separar la presencia de la cantidad: una variable booleana que dice si el alimento aparece en esa comida y una entera con sus gramos. Así que elegí la segunda porque gran parte del modelo razona sobre presencia (prohibiciones, variedad, repeticiones) y expresar esas condiciones sobre una booleana es más directo; los gramos solo importan cuando el alimento está presente.

$$\text{ha_consumido}[f, d, m] \in \{0, 1\}, \quad \text{gramos_alimento}[f, d, m] \in \{0, 1, \dots, \max_f\}, \\ \forall f \in F, d \in D, m \in M$$

Las dos familias se enlazan para que los gramos sean cero si el alimento no está, y una ración dentro de su perfil si lo está:

$$\begin{aligned} \text{ha_consumido}[f, d, m] = 1 &\Rightarrow \text{gramos_alimento}[f, d, m] \geq \min_f, \\ \text{ha_consumido}[f, d, m] = 0 &\Rightarrow \text{gramos_alimento}[f, d, m] = 0 \end{aligned}$$

El mínimo del enlace importa más de lo que parece. En la primera versión bastaba con un gramo para contar como presente, y el solver lo aprovechaba metiendo alimentos a 1g solo para cumplir la presencia mínima por comida o para sumar variedad barata. Por ello existe un suelo absoluto de presencia de 10g, y desde la capa de plausibilidad el mínimo y el máximo reales de cada alimento son los de su perfil de ración (sección 5.3.2).

Los alimentos que se sirven por piezas (un huevo, un yogur) llevan además una variable entera de unidades que cuantiza sus gramos a piezas o medias piezas. Se presenta con el resto de la capa de plausibilidad, que es donde tiene su papel.

5.2.5. Trabajo en enteros

CP-SAT opera sobre dominios enteros, y mantener todo el modelo en enteros es la forma natural de trabajar con él. Los valores nutricionales, que en las tablas de composición vienen con decimales, se escalan a entero multiplicándolos por una constante $s = 10$ y redondeando. Así, la suma de un nutriente n en un día d ,

$$\text{nut}_{d,n} = \sum_{m \in M} \sum_{f \in F} \text{nut100}(f, n) \cdot \text{gramos_alimento}[f, d, m],$$

representa la cantidad real del nutriente multiplicada por $100 \cdot s$: los coeficientes son por 100 g y no se divide dentro del modelo. Para comparar esa suma con un valor real v (un objetivo calórico, un mínimo de proteína), se lleva v a la misma escala y se comparan dos enteros.

El mismo patrón, multiplicar en cruz en lugar de dividir, resuelve cualquier restricción expresada como porcentaje o cociente; esto aparecerá varias veces en las secciones siguientes. La resolución final, de un gramo y una décima de unidad nutricional, es más que suficiente en cuanto a precisión en la práctica.

5.3. Restricciones del modelo

Las restricciones de esta sección no las configura el profesional: se aplican siempre, a cualquier plan, con independencia de lo que llegue en la entrada. Son dos familias. Las estructurales (R1 a R8) dan al plan su lógica de conjunto: suelos fisiológicos, variedad y reparto. Las de plausibilidad (R9 a R15) llegaron después, al auditar los planes generados, y se ocupan de la composición de cada comida. La subsección 5.3.3 pone la frontera entre estas reglas y las del profesional.

5.3.1. Restricciones estructurales

Todas son duras: acotan el espacio de soluciones, y un plan que las incumpla no existe para el solver. Son ocho:

- **R1. Presencia mínima.** Cada comida contiene al menos un alimento.
- **R2. Tope de alimentos por comida.** Como mucho 4 alimentos distintos por comida.
- **R3. Suelo calórico diario.** La energía de cada día no baja del metabolismo basal del cliente (se desarrolla abajo).
- **R4. Rango humano de proteína.** La proteína diaria se mantiene entre 0,8 y 2,2 g por kilo de peso, pidan lo que pidan las restricciones.
- **R5. Grasa mínima.** La grasa aporta al menos el 15 % de la energía diaria, necesario para absorber las vitaminas liposolubles.
- **R6. No repetir alimento en el día.** Un mismo alimento aparece como mucho 2 veces entre las comidas de un día.
- **R7. Variedad diaria.** Cada día usa al menos 4 alimentos distintos.
- **R8. Variedad semanal.** Cada ventana de 7 días usa al menos 10 alimentos distintos.

El tope de R2 evita comidas con una lista interminable de ingredientes minúsculos. Lo fijé en cuatro buscando el equilibrio entre una comida completa y la complejidad del modelo, porque cada alimento admitido por comida multiplica las combinaciones que el solver explora. El valor concreto es revisable: una ensalada normal lo

supera si se cuentan el tomate, la cebolla o la lechuga, y distinguir esos acompañantes para que no consuman el tope queda como trabajo futuro (capítulo 8). Además, sobre esta cota se apoyan los dominios de la función objetivo (sección 5.5).

El suelo calórico de R3 es el metabolismo basal, no el gasto energético total. El basal se estima con la ecuación de Mifflin-St Jeor (Mifflin et al., 1990) a partir del peso w_c (kg), la altura h_c (cm) y la edad a_c del cliente:

$$\text{BMR}(c) = 10 w_c + 6,25 h_c - 5 a_c + \begin{cases} +5 & \text{en hombres} \\ -161 & \text{en mujeres} \end{cases}$$

y el suelo diario es el mayor entre ese basal y un suelo absoluto por sexo (1200 kcal en mujeres, 1500 en hombres). La distinción tiene consecuencia clínica. Al principio usé como suelo el gasto total (el basal por un factor de actividad), y con un cliente activo el suelo quedaba por encima de su objetivo de pérdida de peso, haciéndolo inalcanzable. El gasto total orienta el objetivo calórico que fije el profesional; el suelo que ninguna dieta puede cruzar es el basal, para permitir los planes de déficit. Si faltan los datos antropométricos, se usa el suelo absoluto por sexo y se emite un aviso.

R6 y R7 no estaban en el diseño inicial. Las añadí después de que los tutores probasen el motor ya desplegado y viesan que los planes salían matemáticamente válidos pero irreales: concentraban todo el pollo en un día y rellenaban el resto casi al azar, solo para cumplir los números. La tercera regla de sentido común, repartir un mismo alimento a lo largo del plan, es blanda a propósito y se cuenta con la función objetivo (sección 5.5): si el catálogo aprieta, el solver puede concentrar algo antes que declararse infactible, y se prefiere un plan imperfecto a un fallo. R6 y R7 sí son duras porque ningún plan real las viola. La tabla 5.1 resume su efecto sobre un mismo caso de una semana, generado con y sin las reglas.

	Sin reglas	Con reglas
Máximo de veces de un alimento en un día	3	2
Mínimo de alimentos distintos por día	3	6
Máximo de apariciones de un alimento en el plan	10	5

Tabla 5.1: Efecto de las reglas de sentido común sobre un mismo caso (7 días, 5 comidas).

Las ocho reglas comparten una asunción que conviene dejar explícita: se asumen válidas para la población general, y permitir al profesional anularlas caso a caso es trabajo futuro (capítulo 8). Uno de los directores del TFG puso el ejemplo que enseña el límite: una dieta renal de 45 g de proteína diaria para un cliente de 70 kg queda en 0,64 g/kg, por debajo del suelo de R4, y el motor la declararía infactible aunque esté prescrita a conciencia. La filosofía es la misma que la del suelo calórico: estas reglas son seguridad de último recurso, no una recomendación, y por eso se eligieron lo más inofensivas posible. La modularidad clínica existe en otro plano: las restricciones duras del profesional se incluyen o no en cada resolución según lo que él configure, sin tocar el motor (sección 5.4).

5.3.2. La capa de plausibilidad del catálogo

Las reglas anteriores dan lógica al plan en su conjunto, pero no dicen nada de qué es una ración razonable ni de cómo se compone una comida. Ese hueco lo destapó una auditoría de todos los planes generados hasta entonces (el método y las cifras están en el capítulo 7): en torno a dos de cada tres cantidades servidas caían exactamente en uno de los dos límites globales de gramos del modelo, raciones de relleno de 10 g para cumplir la variedad y raciones de 300 g de un solo alimento como volumen calórico barato. Los límites universales eran una decisión honesta de partida, pero actuaban de referencia para el motor: el modelo no sabía que una ración razonable depende del alimento, y 10 g de fruta o 300 g de aceite eran soluciones legales.

La respuesta fue sustituir los límites globales por un perfil de ración por alimento, que es un dato del catálogo y no una constante del modelo: cada alimento guarda sus gramos mínimos y máximos por aparición y, si se sirve por piezas, sus gramos por unidad. De ahí salen los límites de la formulación:

$$\begin{aligned}\min_f &= \max(\text{round}(\min_serving_g(f)), 10) \\ \max_f &= \text{round}(\max_serving_g(f))\end{aligned}$$

Un alimento sin perfil (típicamente uno dado de alta a mano por un profesional) usa unos valores por defecto de 20 y 250 g, más estrechos que los límites históricos, para que un alimento sin datos se sirva en raciones prudentes.

Con los perfiles llegan las siete reglas de la capa, que continúan la numeración estructural. Las dos primeras ya se han visto: R9 es el propio enlace presencia-gramos con los límites del perfil, y R10 es la cuantización por unidades. Los alimentos con gramos por unidad $gpu(f)$ solo admiten piezas y medias piezas,

$$2 \cdot \text{gramos_alimento}[f, d, m] = k[f, d, m] \cdot gpu(f),$$

con k entero contando medias piezas ($k = 3$ son 1,5 piezas), y un pequeño grupo que no se parte (huevo, yogur) solo admite piezas enteras. Los gramos por unidad se eligieron pares para que las medias piezas caigan en gramos enteros. En base de datos el plan sigue guardando gramos; las unidades son a la vez una restricción del modelo y la forma natural de enseñar esos alimentos en pantalla.

Las otras cinco reglas se apoyan en dos vocabularios nuevos del catálogo, montados sobre las etiquetas existentes: los roles (condimento, dulce) y los momentos del día que admite cada alimento.

- **R11. Lista blanca de franjas.** Un alimento etiquetado con momentos del día solo aparece en esas franjas (la merluza no es un desayuno); un alimento sin etiquetas de momento no queda restringido.
- **R12. Composición mínima de las principales.** Desayuno, comida y cena piden al menos 2 alimentos: una manzana a media mañana es un tentempié normal, una comida de un solo alimento no es una comida.
- **R13. El condimento nunca va solo.** Un condimento exige en su comida al menos un alimento que no lo sea.

- **R14. Tope diario de condimentos.** Entre todos los condimentos, como mucho 2 apariciones al día.
- **R15. Dulce y fruta como complemento.** Como mucho una aparición de dulce o fruta por comida; junto con R12, esto impide que la fruta sea el plato único de una comida principal.

Como muestra del patrón, R13 se expresa acotando cada condimento por la suma de sus posibles acompañantes:

$$\text{ha_consumido}[f, d, m] \leq \sum_{f' \in F \setminus \text{COND}} \text{ha_consumido}[f', d, m],$$

$$\forall f \in \text{COND}, d \in D, m \in M$$

donde COND son los alimentos con rol de condimento. Las siete formulaciones completas están en el Anexo B.

Las franjas de un plan las decide el llamante según su número de comidas, con las principales primero: 3 comidas son desayuno, comida y cena; 4 añaden la merienda; 5 la media mañana; 6 la recena. El número de comidas por defecto de la aplicación se bajó de 5 a 4 para reducir las comidas de muy baja energía. Las medidas de este ajuste están en el capítulo 7.

Las siete reglas de la capa son duras, porque una comida implausible no se compensa con nada, no son algo que el profesional pueda reducir (sección 5.6). La propiedad de la capa es que las reglas son fijas pero su alcance es dato: qué alimentos son condimento, qué franjas admite cada uno y qué ración es razonable vive en el catálogo. Dar de alta un alimento nuevo con su perfil y sus etiquetas lo somete a todas las reglas sin tocar el código.

5.3.3. Restricciones clínicas y restricciones de catálogo

Para organizar las restricciones del modelo, se responde a una pregunta de diseño: quién define cada cosa.

Las restricciones clínicas son las del profesional. Viven como filas en la base de datos, en un vocabulario cerrado de 16 tipos (sección 5.4), con un ámbito (un cliente concreto, o el estilo clínico del nutricionista aplicado a todos los suyos) y la prioridad que él decida. Expresan juicio clínico: una alergia, un objetivo calórico, una preferencia de dieta. Se pueden introducir mediante el formulario manual o mediante el traductor del capítulo 6, y el motor las consumirá igualmente.

Las restricciones de catálogo son las del modelo: las estructurales y las de plausibilidad. No las escribe nadie por cliente; se aplican siempre. Expresan lo que cualquiera da por hecho antes de hablar de nutrición: que una comida no es un chorro de aceite ni la merluza un desayuno. Este conocimiento no es vocabulario del profesional, y ponérselo delante sería ruido: ningún nutricionista prescribe que el aceite vaya acompañado, igual que no prescribe que los platos se sirvan en un plato.

La frontera delimita también la responsabilidad ante un plan malo: si viola una clínica, el fallo es del motor y el validador posterior lo cazaría (sección 5.7); si es

implausible, lo que falta es una regla o un dato de catálogo. Dónde vive cada mitad en la base de datos y cómo evoluciona se especifica en el capítulo 4.

5.4. El catálogo de restricciones clínicas

Las restricciones del profesional llegan al motor en un vocabulario cerrado de 16 tipos. De hecho, se cerró ese vocabulario antes de tocar ningún modelo de lenguaje para ordenar las dos mitades del sistema: cualquier productor tiene que expresarse en estos tipos, y el motor solo entiende de ellos. Cada fila trae el tipo, su objetivo (un alimento, una etiqueta, un nutriente, un valor), la prioridad que la hace dura o blanda y un peso de 1 a 10 que gradúa su importancia si es blanda.

Dos de los 16 tipos son estructurales (número de comidas y duración del plan): fijan la dimensión del problema y no generan restricción. Los otros 14 se agrupan en objetivos de energía y macros, límites de nutrientes, prohibiciones y preferencias, reparto y frecuencia, variedad y combinaciones. La tabla 5.2 los resume; las formulaciones completas están en el Anexo B.

Tipo	Familia	Por defecto	Patrón
meals_per_day	estructural	estructural	dimensiona las comidas
plan_duration_days	estructural	estructural	dimensiona los días
kcal_target	energía	blanda	fuera de bandas
macro_target	macros	blanda	fuera de bandas
nutrient_min	nutrientes	configurable	suma $\geq$ / defecto blando
nutrient_max	nutrientes	configurable	suma $\leq$ / exceso blando
nutrient_ratio	nutrientes	configurable	multiplicación cruzada
forbid_food	prohibición	dura	fija a cero
forbid_tag	prohibición	dura	fija a cero por familia
prefer_food	preferencia	blanda	bonifica en el objetivo
prefer_tag	preferencia	blanda	bonifica por familia
meal_kcal_ratio	reparto	blanda	desviación del reparto
max_servings_per_period	frecuencia	configurable	suma en ventana
no_repeat_food	variedad	blanda	ventana por alimento
no_repeat_tag	variedad	blanda	ventana por familia
forbid_combination	combinación	dura	suma por comida ≤ 1

Tabla 5.2: Catálogo de restricciones clínicas: los 16 tipos.

El patrón de cada tipo se ve con dos ejemplos. Una prohibición de etiqueta (el mecanismo de las alergias e intolerancias) es dura por defecto y fija a cero todas las apariciones de los alimentos afectados:

$$\text{ha_consumido}[f, d, m] = 0, \quad \forall f \in \text{miembros}(t), d \in D, m \in M$$

Un objetivo calórico es blando por defecto: no acota el espacio de soluciones, penaliza en la función objetivo la desviación de la energía diaria respecto al valor pedido fuera de dos bandas de tolerancia, con la formulación de la sección 5.5. Si el profesional lo marca como duro, pasa a ser una igualdad exacta por día.

Qué es duro y qué es blando se decide en tres niveles. Por la naturaleza del tipo: las prohibiciones son duras por defecto (una alergia es absoluta) y los objetivos y preferencias son blandos (son dianas a las que acercarse). Por la fila: el profesional tiene la última palabra y puede marcar cada restricción como dura o blanda. Y por decisión del modelo: las reglas de catálogo de la sección anterior son duras, salvo el reparto, que es blando para no volver infactible un caso con el catálogo justo.

Hay un límite conocido: la variante blanda del cociente entre nutrientes no se modela, porque penalizar la desviación de un cociente exige linealizar el propio cociente y no una simple multiplicación cruzada. Cuando llega, el motor no la aplica pero emite un aviso, para no silenciar una restricción que el profesional puso.

5.5. Función objetivo y resolución

Las restricciones blandas se agregan en una función objetivo que el solver minimiza: una suma de penalizaciones menos bonificaciones (las preferencias premian, y entran con signo negativo porque se minimiza), más dos términos estructurales de variedad y reparto:

$$\min \left(\sum_i \text{penalización}_i - \sum_j \text{bonificación}_j + w_{\text{variety}} \cdot P_{\text{variety}} + w_{\text{spread}} \cdot P_{\text{spread}} \right)$$

Acercarse a un objetivo es minimizar $|\text{suma} - \text{objetivo}|$, que no es una expresión lineal. Se linealiza con el patrón estándar de dos variables no negativas de exceso y defecto:

$$\text{suma} - \text{objetivo} = \text{over} - \text{under}, \quad |\text{suma} - \text{objetivo}| = \text{over} + \text{under}$$

Un detalle de este patrón resultó importante en la práctica: las cotas de las variables de exceso y defecto se derivan del catálogo real (el exceso llega como mucho a la cota superior del nutriente menos el objetivo, y el defecto como mucho al objetivo) en vez de usar una holgura fija. Con una cota fija, al crecer el catálogo el rango real de las sumas la superaba y el solver rechazaba el modelo entero; acotar ajustado además mejora la propagación. La cota superior de cada nutriente se apoya en la propia R2: como una comida lleva a lo sumo cuatro alimentos, basta sumar las cuatro mayores aportaciones individuales que el catálogo puede dar (la ración máxima de cada alimento por su densidad), en vez de suponer el catálogo entero coincidiendo en una comida. El techo resultante es entre uno y dos órdenes de magnitud menor sin excluir ninguna solución; su efecto sobre los tiempos se mide en el capítulo 7.

Sobre esa linealización, los objetivos blandos de energía y macronutrientes penalizan solo lo que sobresale de dos bandas de tolerancia: una por día y otra sobre la media del plan. Con $T_{\text{día}}$ y T_{media} las dos tolerancias llevadas a la escala entera y K el factor de la segunda:

$$\begin{aligned}
\text{exceso_diario}_d &\geq \text{over}_d + \text{under}_d - T_{\text{día}}, & \text{exceso_diario}_d &\geq 0, & \forall d \in D \\
\sum_{d \in D} (\text{over}_d - \text{under}_d) &= \text{over}_S - \text{under}_S, & \text{exceso_media} &\geq 0 \\
\text{exceso_media} &\geq \text{over}_S + \text{under}_S - |D| \cdot T_{\text{media}} \\
\text{penalización} &= \text{peso efectivo} \cdot \left(\sum_{d \in D} \text{exceso_diario}_d + K \cdot \text{exceso_media} \right)
\end{aligned}$$

La primera banda deja libre cada día dentro de $\pm T_{\text{día}}$ del objetivo. La segunda actúa sobre la suma de las desviaciones con signo, es decir, sobre la media del plan: unos días compensan a otros y la media queda a menos de T_{media} del objetivo. Los valores son 50 y 25 kcal para el objetivo calórico, un 5 y un 2,5 % del objetivo para los macronutrientes, y $K = 10$ para el ancla de la media, que la hace casi obligatoria sin ser dura. Con las dos tolerancias a cero la formulación se reduce a la desviación absoluta total.

Decidí las bandas porque clavar la energía al gramo no tiene relevancia clínica, y penalizarla desde la primera kcal hacía que el solver gastase el tiempo de búsqueda en eso en vez de en la estructura del plan. El ancla de la media la añadí para que los días no derivasen todos hacia el mismo borde de la banda. Las medidas que llevaron a esta decisión están en el capítulo 7. Las filas duras siguen exigiendo la igualdad exacta por día, y los mínimos y máximos de nutriente no llevan banda porque son umbrales de un solo lado.

P_{variety} penaliza suavemente cada alimento del catálogo que no se use en todo el plan, para favorecer usar más alimentos distintos cuando nada más lo decide. P_{spread} es la tercera regla de sentido común de la sección 5.3.1: penaliza que un alimento supere un tope de apariciones en el plan, para que la ingesta se reparta en vez de concentrarse. El tope es proporcional a la duración, 0,6 apariciones por día de plan: en una semana un alimento puede aparecer unas cuatro veces antes de penalizar, frente a las diez que llegaron a verse sin la regla (tabla 5.1).

Cada término lleva un peso efectivo que es el producto de dos niveles: un peso fijo por familia (10 el objetivo calórico, 8 la proteína, 4 000 las preferencias; el resto está en el Anexo B) y el peso de 1 a 10 de la fila concreta. Los pesos de las familias no nutricionales llevan incorporado un factor de 1 000 que compensa la escala entera de las desviaciones: una desviación nutricional se mide en unidades escaladas (una kcal son 1 000 unidades) y una preferencia o un alimento sin usar se cuentan de uno en uno, así que sin ese factor mover una familia estructural entera no compensaba ni una kcal de desviación. El análisis de magnitudes que destapó ese desequilibrio se enseña en el capítulo 7. Con la fila del objetivo calórico a peso 7, cada kcal fuera de la banda diaria cuesta 70 000 y cada kcal que la suma de las desviaciones del plan se salga de la banda de la media, 700 000; dentro de las bandas la energía no cuesta nada, así que ahí deciden la estructura y las preferencias. Esto da una regla de prioridad clara sin exponer al profesional un panel de ajuste global.

El reparto calórico por comidas es la excepción documentada: su linealización por multiplicación cruzada arrastra un factor de 1 000 propio, así que cuando el profesional lo activa su término domina la función objetivo. Igualar su escala exigiría

renormalizar el término entero, y queda como trabajo futuro (capítulo 8).

El modelo se resuelve con un límite de tiempo de 90 segundos, una brecha relativa del 2 % y un número de hilos de búsqueda configurable por entorno. Los tres parámetros cuentan lo mismo: un borrador editable no necesita el óptimo demostrado. El solver para al llegar al 2 % de la cota inferior y, si el tiempo expira, devuelve la mejor solución factible encontrada, distinguiendo en las métricas el óptimo demostrado del factible. El límite de tiempo está calibrado contra la CPU del alojamiento gratuito donde corre el backend, no contra la máquina de desarrollo. Y los hilos son 8 en desarrollo y 2 en ese alojamiento: cada hilo mantiene su propia copia del estado de búsqueda, y con 8 hilos el proceso superó el límite de memoria de la instancia. Adecuar el paralelismo a los recursos es una decisión de despliegue, no del modelo; el coste asumido es que más casos terminan en factible en vez de en óptimo, con la misma calidad de plan útil. Las medidas están en el capítulo 7.

5.6. Diagnóstico de infactibilidad

Cuando las restricciones duras no admiten solución, el motor no falla sin más: devuelve un subconjunto de restricciones duras que basta para explicar el conflicto. Se apoya en las asunciones de CP-SAT. Cada restricción clínica dura se envuelve en un literal booleano de activación, se aplica condicionada a ese literal y se declara el literal como asunción. Si el modelo resulta infactible, el solver devuelve un conjunto de literales responsables, que se traducen de vuelta a sus restricciones; CP-SAT garantiza que ese conjunto basta para explicar la infactibilidad, no que sea el más pequeño posible.

Solo las clínicas duras llevan literal. Las reglas de catálogo se aplican sin condición, así que nunca aparecen en el diagnóstico: el conflicto se explica siempre en términos de cosas que el profesional puso y puede cambiar.

El resultado lleva tres piezas: el núcleo, una sugerencia redactada a partir de los tipos del núcleo y la lista de restricciones que se pueden reducir. Por ejemplo, un objetivo duro de 900 kcal para un adulto de 88 kg devuelve el núcleo con esa única restricción: su metabolismo basal ronda las 1 800 kcal y el suelo calórico de R3 no deja bajar de ahí, así que ningún plan alcanza las 900 sin romper una regla que el profesional no puede quitar. Sobre este diagnóstico se construye después la explicación en lenguaje llano del capítulo 6.

5.7. El validador determinista

El plan factible no se da por bueno solo porque lo diga el solver. Antes de llegar al profesional pasa por un validador independiente: otra función pura que recalcula la nutrición del plan desde los datos del catálogo, sin fiarse de las métricas del solver, y lo audita en cuatro familias de reglas: seguridad fisiológica (suelo calórico, rangos de macros, topes de toxicidad de micronutrientes), consistencia con las restricciones clínicas duras (las re-verifica una a una sobre el plan ya construido), estructura (tamaño de las comidas, mínimos de gramos) y plausibilidad (las mismas garantías

de la sección 5.3.2). Son dos motores deterministas que deben coincidir: si el solver tuviera un error, el validador lo cazaría antes de la firma.

Cada hallazgo del validador es un rechazo o un aviso, y el criterio para asignarlos no es clínico, es de coherencia: el validador solo rechaza lo que el solver garantiza (las duras, los suelos, la estructura, la plausibilidad) y avisa de lo que el solver no modela. El caso que fijó el criterio fueron los micronutrientes: el solver no les pone techo, así que el calcio o el hierro de un plan válido oscilan de una generación a otra y a veces cruzan un tope de toxicidad. Con ese tope como rechazo, el veredicto dejaba de ser determinista y ahora pasaba o fallaba según la corrida. Como aviso, la distinción queda donde debe: un plan inseguro se rechaza, un plan mejorable se enseña al profesional con sus avisos, y es él quien decide.

Con el validador se cierra el recorrido del motor: de un perfil y unas restricciones a un borrador auditado, o a una explicación de por qué no puede existir. La evaluación de todo ello (los bancos de casos, los tiempos y el efecto medido de cada capa de reglas) está en el capítulo 7, y cómo se expone el motor como servicio dentro de la aplicación, con su persistencia y la firma del profesional, en el capítulo 4.

Capítulo 6

El copiloto de IA

El copiloto de IA es la capa que permite al nutricionista pedir un plan escribiendo lo que quiere en su propio lenguaje. Sobre el motor determinista del capítulo 5 se añade un modelo de lenguaje que convierte ese texto en restricciones del vocabulario cerrado, y una interfaz que enseña cada paso intermedio. Realmente funciona como un asistente de redacción: la IA propone restricciones, y el profesional decide cuáles se aplican, cuáles se guardan y qué plan llega al cliente. Este capítulo explica qué escribe, qué le devuelve el sistema y qué decisiones toma el copiloto.

La sección 6.1 presenta el flujo completo y los puntos donde decide el profesional. Las tres siguientes desarrollan sus piezas: cómo se convierte el texto en restricciones (sección 6.2), por qué el modelo de lenguaje se ejecuta en local (sección 6.3) y cómo se revisa y arbitra lo que la IA propone (sección 6.4). La sección 6.5 cierra el recorrido desde la petición hasta el plan firmado. La generación asíncrona y la validación se cuentan aquí solo en lo que el profesional percibe, porque sus mecanismos ya están descritos en las secciones 4.4 y 5.7; y las pruebas de los modelos de lenguaje, con sus bancos y mediciones, se enseñan en el capítulo 7.

6.1. El flujo del copiloto

El punto de partida del diseño es que el responsable del plan es el profesional colegiado que lo firma. Por eso el copiloto no genera dietas por su cuenta: redacta restricciones a partir de lo que el nutricionista escribe, se las enseña antes de aplicarlas y no guarda nada sin su decisión. Lo que se le pide es traducir la petición a parámetros, para agilizar el proceso.

El flujo se entiende mejor siguiendo una petición real. El nutricionista elige al cliente y escribe el encargo como lo apuntaría en consulta: "sin gluten, unas 2000 kcal al día y al menos 120 g de proteína". El sistema revisa primero el mensaje y, si detecta que falta un dato, que algo choca con lo ya guardado o que la petición se sale de su ámbito, se lo devuelve con el motivo concreto para que lo corrija. Si el mensaje es viable, el traductor lo convierte en restricciones del vocabulario del capítulo 5 y se las presenta en frases llanas, sin jerga técnica. La figura 6.1 recoge el recorrido completo.

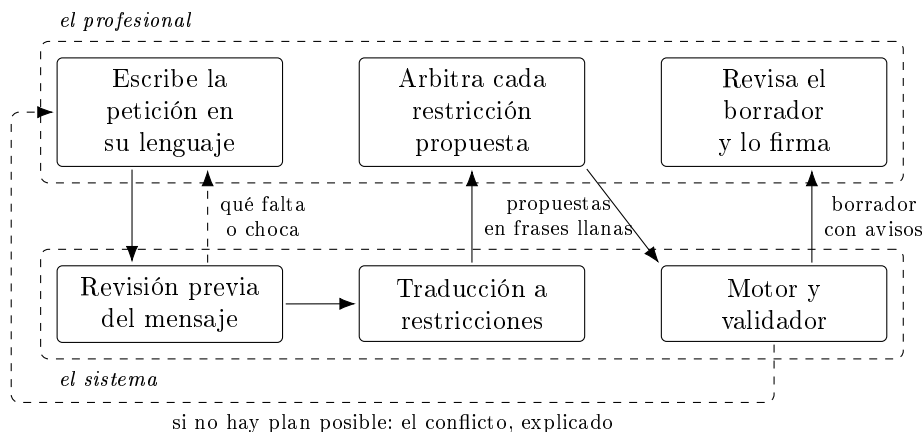


Figura 6.1: El flujo del copiloto. La fila superior son las decisiones del profesional y la inferior el trabajo del sistema: ningún resultado avanza hacia el cliente sin pasar antes por una decisión suya.

Sobre cada restricción propuesta el profesional decide una de tres cosas: guardarla en el perfil del cliente, usarla solo para esta generación o descartarla. Lo que el traductor no entendió también se le enseña con su motivo. Y en la misma pantalla puede añadir restricciones a mano, así que las propuestas de la IA y las suyas conviven en una única lista antes de generar.

Con las restricciones decididas, el motor construye el borrador y el validador lo audita. Si no existe ningún plan que cumpla todo, el conflicto se le explica en una frase con los nombres de su propio catálogo (sección 6.5). El plan generado queda como borrador, y solo pasa a ser visible para el cliente cuando el profesional lo firma, con la política de visibilidad de la sección 4.3.

El copiloto es además opcional en cada paso; la misma pantalla permite saltarlo y fijar las restricciones a mano, y los formularios de la ficha del cliente y del estilo clínico escriben sobre la misma tabla que el traductor (figura 4.2): el motor no distingue el origen. La vía manual queda así como respaldo permanente: si el modelo de lenguaje no está disponible, la aplicación entera funciona igual y el profesional solo pierde la comodidad de escribir en su lenguaje.

6.2. Del texto a las restricciones

El mensaje no va directo al modelo de lenguaje. Antes pasa por una revisión previa: una comprobación determinista que descarta los idiomas que el sistema no soporta, y tres comprobaciones separadas, cada una con una llamada aislada al modelo, que valoran si la petición es del ámbito dietético, si está completa o si se contradice a sí misma. Cuando alguna falla, el profesional recibe el mensaje original con sus detalles precargado para corregirlo sin reescribirlo.

Cada comprobación usa el modelo más pequeño que hace bien ese trabajo. Al medirlas por separado con un banco de mensajes propio, el tamaño necesario resultó distinto en cada una: el modelo que basta para juzgar el ámbito falla la completitud,

y ningún modelo único daba buen resultado en las tres. La matriz de esa medición se enseña en el capítulo 7.

Las contradicciones se reparten entre tres capas, cada problema en la capa más fiable que puede resolverlo. El modelo de lenguaje detecta las categoriales dentro del mensaje, como quitar la carne y a la vez pedir más ternera. También, ya en código y sin pasar por el modelo, se comparan las restricciones propuestas entre sí y con las ya guardadas del cliente, avisando con los nombres reales de lo que choca. Por otro lado, las aritméticas, como un objetivo calórico incompatible con un mínimo de proteína, no las juzga nadie por adelantado: las detecta el motor como infactibilidad y se explican como cuenta la sección 6.5.

Tras la revisión, la traducción se reparte entre el modelo y una capa determinista. El modelo de lenguaje solo extrae la intención: el tipo de restricción, elegido de la lista cerrada de tipos del catálogo que va en el propio prompt, el objetivo (un nutriente o una familia por su código, un alimento por su nombre) y el valor. La capa determinista resuelve después cada código y nombre contra el catálogo real de la base de datos y valida la restricción con las mismas reglas que el formulario manual.

El criterio del reparto es que el modelo elige de un menú, no inventa identificadores, y lo que no se puede resolver se rechaza con su motivo en vez de adivinarse. Si el nutricionista escribe un alimento que no existe en su catálogo, esa petición aparece entre las no aplicadas con su explicación, y el resto del mensaje se traduce igual. Una restricción que supera este camino es idéntica a una creada a mano: mismas columnas, misma validación, mismo consumidor.

Este reparto hace además al traductor poco sensible a cómo esté redactado el mensaje: la lista cerrada y la validación posterior absorben la variación de formato. El estudio que lo midió se enseña en el capítulo 7.

6.3. La elección del modelo de lenguaje

Decidí ejecutar el modelo de lenguaje en local. Es la decisión de privacidad que la sección 4.5 dejaba remitida a este capítulo: los mensajes del profesional hablan de datos clínicos de personas concretas, y uno de los requisitos no funcionales del capítulo 3 fija que ese dato no se envía a ningún proveedor de IA. Con un modelo abierto ejecutado en la propia máquina, el texto no sale del entorno del profesional; la consecuencia sobre el despliegue, que el worker corre donde corre el modelo, ya se contó en la sección 4.5.

El modelo es qwen2.5 (Qwen Team, 2024), una familia abierta con variantes de varios tamaños, y lo ejecuto con el entorno de Ollama, que sirve modelos en local y admite salida con esquema. El traductor usa la variante 7B ajustada a instrucciones; las comprobaciones de la revisión previa usan variantes más pequeñas, elegidas como se contó en la sección 6.2.

A la privacidad se suma el coste: cada consulta a un modelo local cuesta cero, y eso permitió medir, probar y repetir durante todo el desarrollo sin presupuesto. En cambio, con las API gratuitas en la nube, en sus planes sin coste, el proveedor puede usar los mensajes para entrenar, justo lo que el requisito de privacidad prohíbe.

El traductor no depende de esta elección ya que habla con el modelo a través de una interfaz mínima, así que cambiar a un motor remoto con garantías contractuales de no retención sería escribir otra implementación de esa interfaz, sin tocar el resto, por lo que esa puerta queda abierta por diseño, y la comparación entre el motor local y uno remoto queda como trabajo futuro en el capítulo 8. Cada llamada deja además una fila de traza en la base de datos, con la entrada, la salida, el modelo, los tokens y la latencia: información que luego se usará para las mediciones del capítulo 7.

6.4. Revisión y arbitraje

Todas las restricciones que van a alimentar una generación aparecen en la misma pantalla de revisión. Las que propuso el traductor aparecen marcadas como tales, y el profesional puede añadir otras a mano en la misma lista, con el mismo formulario de la ficha del cliente. Sobre cada restricción toma una de las tres decisiones que adelantó la sección 6.1: guardarla en el perfil del cliente (queda en su ficha), aplicarla solo a esta generación (viaja con la tarea y no se guarda) o descartarla.

Los fallos también aparecen aquí. Las peticiones que el traductor no supo aplicar se enseñan con su motivo; los veredictos de la revisión previa, con su aviso, su detalle y el mensaje precargado para corregirlo; y las generaciones fallidas quedan listadas con su explicación.

Los tutores pidieron además que todo esto hablara el idioma del profesional, es decir, que los mensajes mostrados no fuesen confusos o relativos a la base de datos. Por ello, una restricción se enseña siempre como una frase (nunca incluir gluten, unas 2000 kcal al día) y su prioridad como obligatoria o preferible, sin los nombres internos de la base de datos. Las frases salen de una única función compartida por la ficha del cliente, los ajustes y esta pantalla, para que la misma fila se lea igual en toda la aplicación. También el formulario manual se renombró por intención (fijar un objetivo, evitar un alimento o una familia, dar forma a las comidas) en vez de por el nombre técnico de cada tipo.

Además de las restricciones del cliente, el profesional mantiene en sus ajustes sus reglas de estilo: las que aplica a todos sus planes, como no repetir carne roja en días seguidos. Usan el mismo vocabulario y el mismo formulario, y el motor las une con las del cliente en cada generación; es el tercer productor de la figura 4.2. La versión ambiciosa de esta idea, aprender el estilo del profesional a partir de sus correcciones, queda como trabajo futuro en el capítulo 8.

6.5. De la petición al plan firmado

Generar con IA es lento para una petición web: entre el modelo de lenguaje y el solver, una generación tarda desde varios segundos hasta más de un minuto. La primera versión lo hacía todo en la misma petición y bloqueaba la pantalla mientras tanto. Uno de mis directores me recomendó encolar el trabajo, responder al instante y consultar el estado periódicamente. La mecánica, con la cola en la base de datos y el worker que la consume, se describió en la sección 4.4.

De esta forma, para el profesional, la aplicación no se queda colgada: confirma la generación, puede irse a otra pantalla y el apartado de planes enseña cada tarea con su estado hasta que el plan aparece. Si el worker no está encendido, la interfaz avisa de que la tarea sigue esperándolo (sección 4.5).

Cuando el motor no encuentra ningún plan, el diagnóstico de la sección 5.6 se convierte en una frase con los nombres reales del catálogo, algo como: ningún plan puede cumplir el objetivo de 900 kcal con los alimentos disponibles, y conviene relajarlo. La generación fallida queda en la lista con esa explicación y el detalle del conflicto, y el profesional decide si cambia esa restricción o el mensaje.

El borrador que sí existe llega auditado por el validador de la sección 5.7, y sus avisos se enseñan junto al plan: un aviso no bloquea, y es el profesional quien decide si el plan vale así. La firma es el último paso y el único que convierte el borrador en el plan que ve el cliente; antes de escribirla, el backend vuelve a validar el plan y la propiedad del cliente (sección 4.4).

Con la firma se cierra el recorrido del capítulo: de una petición escrita en el lenguaje del profesional a un plan auditado que él decidió restricción a restricción. La evaluación de las piezas de IA, con los bancos de las comprobaciones, el estudio de formatos y el comportamiento medido del modelo local, está en el capítulo 7.

Evaluación

Este capítulo recoge lo que se midió durante el desarrollo. Las secciones 7.1 a 7.4 evalúan el sistema: los bancos de verificación que acompañan a cada bloque, la calidad de los planes que produce el motor, su eficiencia y las piezas de IA del copiloto. La sección 7.5 recoge la prueba con nutricionistas, separada a propósito de las medidas de rendimiento y configuración.

Todas las evaluaciones comparten un método que fijé en la primera de ellas, el estudio de formatos del mensaje (sección 7.4.1), y que los experimentos del motor heredaron después: el criterio de decisión se escribe antes de medir, cada medida se repite, los resultados crudos se versionan junto al documento del experimento y las tablas y figuras se regeneran por script. Los veredictos se apoyan en garantías (óptimo probado, factible, infactible) y no en soluciones concretas, porque con CP-SAT en paralelo y al límite de tiempo la solución exacta no es reproducible. Dos incidentes de instrumentación obligaron además a cuidar el reloj: la suspensión del portátil durante una pasada larga infla el tiempo medido (CP-SAT llegó a reportar 20273 s con un límite de 90), y dos campañas concurrentes se contaminan por la CPU compartida. Los harness piden ahora al sistema no suspender, descartan relojes imposibles, y las campañas afectadas se repitieron enteras.

7.1. Bancos de verificación y línea base

La sección 4.6 describe las tres familias de verificación que cierran cada bloque de desarrollo: los bancos deterministas del motor y del validador, los runners de seguridad multi-actor y las pruebas de extremo a extremo contra el despliegue. La tabla 7.1 resume su estado con el motor final, el 3 de septiembre de 2026.

Los bancos deterministas crecieron con los bloques: el del solver pasó de 19 comprobaciones en su primera versión a 32, y el del validador de 25 a 38, porque cada capa nueva del modelo trajo sus propias garantías. Los runners de seguridad acumulan ocho tandas de evidencia; las dos últimas entran con tres actores (dos nutricionistas y un cliente) y son las que separan lo que cada rol puede leer y escribir. La prueba de extremo a extremo contra el backend en producción se repitió tras cada cambio del motor, y el 29 de 29 de la tabla corresponde al motor con bandas de tolerancia servido desde el alojamiento.

Banco	Qué comprueba	Resultado
Solver	ocho casos con veredicto esperado: factibles, infactibles con núcleo, garantías de las reglas base y de plausibilidad, bandas de tolerancia	32 de 32
Validador	planes reales que deben pasar y planes artificiales que violan una familia de reglas cada uno	38 de 38
Cola asíncrona	ciclo de vida de las tareas con el worker y el motor real	14 de 14
Validación previa	los cinco estados de las pasadas con sus tres modelos, conflictos deterministas y explicación de infactibilidad	44 de 44
Traductor	casos representativos con cliente simulado y con el modelo real	15 de 15
Productor-consumidor	las restricciones de ámbito nutricionista se aplican en la generación	7 de 7
Seguridad (lectura)	tres actores leen tabla a tabla bajo las políticas de fila	60 de 60
Seguridad (escritura)	los mismos actores escriben, incluidos los intentos de suplantación	106 de 106
Extremo a extremo, backend	los doce casos de diseño, la firma y el rechazo sin token, contra producción	29 de 29
Extremo a extremo, panel del cliente	flujo completo del cliente contra el frontend desplegado	39 de 39

Tabla 7.1: Bancos de verificación permanentes y su resultado con el motor final.

Antes de los experimentos del motor se capturó además una línea base de tiempos del banco del solver, en la máquina de desarrollo con 8 hilos y tres repeticiones (tabla 7.2). Dos lecturas de esa tabla condicionan el resto del capítulo. Los casos pesados viven cerca del límite de 90 segundos y el caso 5 cambia de estado entre pasadas, así que con casos al límite la reproducibilidad se define sobre veredictos y garantías. Y la dispersión entre repeticiones de un mismo caso llega al doble (caso 2), así que ningún veredicto de rendimiento de este capítulo se apoya en una pasada única.

7.2. La calidad del plan

Esta sección sigue dos hilos. El primero pregunta si el plan es comida real: la auditoría que destapó los sinsentidos del modelo inicial y la comparativa entre versiones que mide lo que compró cada capa de reglas. El segundo pregunta qué optimiza el solver cuando elige entre planes válidos: las magnitudes de la función objetivo y la jerarquía que quedó tras corregirlas.

Caso	Rep. 1	Rep. 2	Rep. 3	Estado
1. María, 3 días, sin configurables	0,4 s	0,3 s	0,4 s	óptimo
2. María, 7 días, objetivo calórico	9,3 s	4,0 s	5,5 s	óptimo
3. María, objetivo calórico y preferencia	24,8 s	22,1 s	28,3 s	óptimo
4. John, alergia y mínimo de proteína	80,7 s	61,1 s	86,3 s	óptimo
5. Emma, intolerancia y máximo de sodio	90,0 s	39,5 s	36,6 s	factible / óptimo / óptimo
8. Diseño, cuatro tipos configurables	90,0 s	90,0 s	90,0 s	factible

Tabla 7.2: Línea base de tiempos del banco del solver antes de los experimentos del motor (1 de septiembre de 2026). Los casos 6 y 7 son de infactibilidad y no aparecen.

7.2.1. La auditoría de sinsentidos

Tras las reglas base de la sección 5.3.1, los planes cumplían los números y seguían teniendo cosas que ningún profesional firmaría. Antes de diseñar más reglas decidí medir. Escribí un instrumento de auditoría que recorre todos los planes generados hasta la fecha y cuenta patrones: raciones pegadas a los límites, comidas de un solo alimento, comidas de muy baja energía, alimentos en franjas donde no se comen, varias frutas en una misma comida. Los umbrales que usa para señalar candidatos son heurísticos de revisión a mano, no constantes del modelo: el script mide y el juicio va después. En julio de 2026 se pasó sobre los 12 planes vivos, todos producidos por el pipeline real, con 1 382 raciones en 420 comidas.

El hallazgo central fue que los dos límites globales de gramos del modelo inicial (10 y 300 g para cualquier alimento) actuaban de imanes. El 35,3 % de las raciones estaba exactamente en el suelo y el 31,0 % en el techo: dos de cada tres cantidades del plan no las decidía ninguna lógica nutricional. La mecánica se ve en los datos. El suelo de 10 g es el relleno barato con el que el solver cumple la variedad mínima diaria (10 g de jamón cocido, 10 g de pera, testigos que no aportan nada a la comida), y el techo de 300 g es donde aparca volumen de baja densidad calórica para ajustar la energía sin pasarse (verduras crudas a 300 g en todas las franjas). Los totales diarios sí estaban garantizados por el solver y el validador; el sinsentido vivía un nivel por debajo, dentro de cada comida, justo donde el modelo no tenía reglas.

De la auditoría salió un catálogo de siete sinsentidos con ejemplos reales: raciones fuera de rango (un desayuno con 158 g de aceite), comidas testimoniales (un desayuno de 8 kcal), comidas principales de un solo alimento, condimentos como plato, alimentos fuera de franja (bacalao en el desayuno), fruta apilada y planes monotemáticos (48 apariciones de verdura en una semana). Cada regla de la capa de plausibilidad de la sección 5.3.2 nace de uno de esos sinsentidos documentados, y no al revés.

La misma auditoría, sobre los planes regenerados con la capa puesta, es el antes y después de la tabla 7.3. Las raciones fuera del perfil de su alimento y los alimentos

fuera de franja no aparecen en la tabla porque pasaron a ser garantías duras del modelo, comprobadas en el banco.

Contador	Antes (12 planes)	Después (9 planes)
Raciones en el suelo global de 10 g	488 (35,3 %)	23 (2,4 %)
Raciones en el techo global de 300 g	429 (31,0 %)	34 (3,5 %)
Comidas de un solo alimento	60 (14,3 %)	5 (2,0 %)
Comidas solo de grasas, frutos secos o condimentos	18	0
Comidas con más de una fruta	47	0
Comidas principales de solo fruta	15	0
Comidas por debajo de 150 kcal	106 (25,2 %)	13 (5,2 %)

Tabla 7.3: La auditoría de sinsentidos antes y después de la capa de plausibilidad. Antes: 12 planes con 1 382 raciones en 420 comidas, 5 comidas al día. Después: los 9 planes de demostración regenerados, 961 raciones en 252 comidas, 4 comidas al día. Las raciones que quedan en 10 o 300 g son mínimos y máximos legítimos del perfil de su alimento.

El residuo de comidas de baja energía merece su propia línea, porque de él salió la decisión del número de comidas por defecto. Con la capa puesta y 5 comidas al día, las comidas por debajo de 150 kcal bajaban del 25,2 al 15,6 %, casi todas meriendas y medias mañanas. En vez de añadir un suelo de energía por comida, decidí bajar el defecto de 5 a 4 comidas y medirlo con los mismos nueve objetivos: las comidas de baja energía cayeron al 6,3 % y los tentempiés de solo frutos secos desaparecieron sin regla nueva. Al implementarlo apareció un fallo que ninguna prueba había visto: el mapeo de franjas tomaba las primeras N por orden del día, así que un plan de 3 comidas se quedaba con desayuno, media mañana y comida, sin cena. El mapeo pasó a ser explícito por número de comidas. El cambio tiene un coste medido en los objetivos calóricos altos: el plan de Tomás, con 3 000 kcal, pasó de una desviación del 11 al 41 % al quitarle una comida, con óptimo demostrado, porque cuatro comidas de cuatro raciones acotadas por su perfil no llegan a esa energía. El número de comidas sigue siendo editable por el profesional, y para objetivos altos se suben.

La capa trajo también una lección sobre el propio banco. Dos comprobaciones heredadas comparaban el plan con reglas contra una pasada con las constantes relajadas, para demostrar que las reglas cambiaban algo. Cuando los límites de ración pasaron a ser datos del catálogo, la pasada relajada dejó de existir: el mundo sin reglas ya no se puede reproducir relajando constantes, y las dos comprobaciones fallaban de forma sistemática. Se reformularon como garantías (el tope se cumple, la variedad mínima se cumple) y el banco quedó en 30 comprobaciones.

7.2.2. Comparativa de versiones del modelo

Uno de los directores del TFG pidió comparar la primera versión del modelo con las posteriores. La comparativa reconstruye tres versiones del motor a partir de sus commits: la inicial (v0: estructura, suelos fisiológicos y catálogo de restricciones), la que añadió las reglas base de sentido común (v1) y la actual (v2: v1 más la capa de

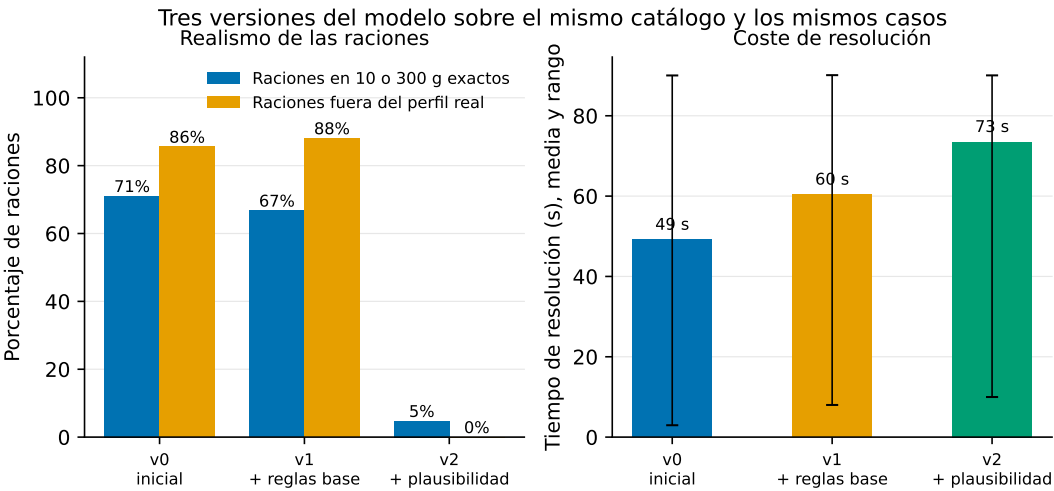


Figura 7.1: Realismo de las raciones frente a coste de resolución por versión del modelo. A la derecha, media y rango del tiempo de resolución.

plausibilidad y los dos ajustes del objetivo de las secciones siguientes). Para aislar el efecto de las reglas, las tres se ejecutan sobre el mismo catálogo actual de 100 alimentos, los mismos 11 casos (los cinco del banco con términos blandos y los seis clientes de demostración) y la misma configuración (90 segundos, 8 hilos), con tres repeticiones por caso. Las métricas las calcula un analizador común fuera de las versiones. El catálogo también cambió entre versiones (de 28 a 100 alimentos), pero eso es una decisión de datos y aquí no se mide.

La expectativa registrada antes de ejecutar era que la v0 sería la más rápida y la menos plausible, y la v2 la más lenta y la más plausible: la tabla debía enseñar el precio de cada capa de realismo.

	v0 inicial	v1 + reglas base	v2 + plausi-bilidad
Óptimos probados (de 33 re-soluciones)	21	20	15
Tiempo medio de resolución	49,1 s	60,4 s	73,5 s
Desviación calórica media	8,28 %	8,09 %	8,24 %
Raciones en 10 o 300 g exactos	71,0 %	66,7 %	4,8 %
Raciones fuera del perfil real de su alimento	85,6 %	88,1 %	0,0 %
Alimentos distintos por día	13,5	13,5	14,3
Máximo del mismo alimento en un día	5	2	2

Tabla 7.4: Tres versiones del modelo sobre el mismo catálogo y los mismos 11 casos, con 3 repeticiones por caso.

La expectativa se cumple con márgenes claros (tabla 7.4 y figura 7.1). La v0 es la más rápida y la que produce planes que ningún profesional firmaría: el 71 % de

las raciones cae exactamente en 10 o 300 gramos, el 85,6 % queda fuera del perfil de ración real de su alimento y un mismo alimento llega a aparecer 5 veces en un día. Las reglas base de la $v1$ eliminan la concentración (máximo 2 apariciones al día) por un 23 % más de tiempo, y no tocan las raciones. La capa de plausibilidad de la $v2$ es la que convierte los planes en comida real: raciones fuera de perfil al 0 % y raciones en los límites globales al 4,8 % (el residuo son alimentos cuyo perfil incluye legítimamente 10 o 300 gramos), a cambio de un 50 % más de tiempo que la $v0$ y de probar el óptimo en menos casos dentro del mismo presupuesto. La desviación calórica es indistinguible entre versiones: lo que compran las capas es realismo alimentario, no macronutrientes.

La única resolución sin respuesta del experimento es una repetición del caso de diseño en la $v0$, que agotó el límite sin encontrar solución ni probar infactibilidad; las otras dos repeticiones del mismo caso sí resolvieron, otra muestra de que el portfolio multihilo no es determinista al límite de tiempo.

7.2.3. Las magnitudes de la función objetivo

La función objetivo de la sección 5.5 suma términos de naturaleza distinta. Las desviaciones nutricionales (objetivo calórico, macronutrientes, mínimos y máximos de nutriente) se miden en las unidades escaladas del modelo, donde una kcal son 1 000 unidades; multiplicadas por el peso de familia y el peso de fila, llegan a aportar 100 000 unidades de objetivo por cada kcal de desviación en un día. Los términos estructurales y de preferencia se cuentan de uno en uno, por alimento o por aparición: el término de variedad completo, con un catálogo de 100 alimentos y peso 5, tiene un rango total de 500.

Al leer el capítulo 5, mis directores me indicaron que esos sumandos no eran conmensurables y que todo lo que no fuese nutricional podía haber quedado reducido a ruido numérico. Uno de ellos pidió medirlo antes de decidir nada, y ofreció la salida de documentarlo como limitación si la corrección exigía renormalizar la formulación a fondo. Así que antes de tocar el motor dejé escrito qué contaría como desequilibrio y qué haría en cada caso.

El criterio registrado fue este: hay desequilibrio si, en la mediana de los casos con algún objetivo nutricional, el rango completo alcanzable de una familia no nutricional (preferencias, reparto del plan o variedad) es menor que lo que cuesta una unidad natural de desviación (una kcal, un gramo o un miligramo) del término nutricional dominante del caso. Llamo Q al cociente entre ambas cantidades: si $Q < 1$, mover esa familia entera de su mejor a su peor valor no compensa ni una unidad de desviación, y la familia es numéricamente irrelevante. El remedio también quedó registrado: multiplicar por 1 000 los pesos de las familias no nutricionales, que equivale a medir las desviaciones en unidades naturales enteras en vez de en milésimas, sin tocar ninguna restricción ni la escala del modelo. El ajuste solo se mantendría si los bancos seguían en verde y la desviación calórica del banco seguía por debajo del 5 %; si degradaba la nutrición, se retiraba y el desequilibrio quedaba documentado con la tabla de esta sección como evidencia.

Para medirlo, cada caso se resuelve con el motor real y cada término del objetivo se recompone desde la solución devuelta: su valor crudo, su peso efectivo y su

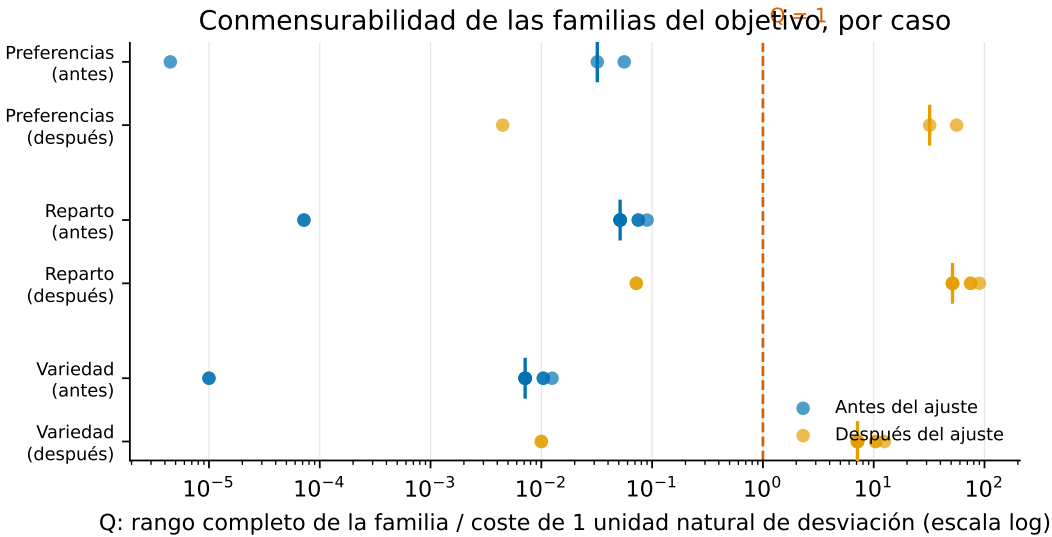


Figura 7.2: Q de cada familia en cada caso, antes y después del ajuste (escala logarítmica). La línea discontinua marca $Q = 1$; la barra vertical de cada serie es su mediana.

contribución, más una cota analítica de su rango. La recomposición se comprueba contra el valor de objetivo que reporta el solver, para que la tabla refleje la función objetivo real y no una reconstrucción. Los casos son los cinco del banco del solver que llevan términos blandos y los seis clientes de demostración con sus restricciones reales, a 7 días y 4 comidas. La recomposición coincidió con el valor reportado en 10 de los 11 casos; en el restante (Tomás) el valor reportado quedó 1 568 unidades por encima, sobre un objetivo de 63 208 millones. La diferencia es holgura de la descomposición del valor absoluto: las variables de exceso y defecto solo quedan apretadas si la minimización las empuja, y en un caso parado dentro de la brecha del 2 % puede sobrar holgura. Como la recomposición trabaja sobre la solución, da el valor verdadero en los 11 casos.

Familia	Q antes (mediana)	Q después (mediana)
Preferencias	0,032	32
Reparto del plan	0,051	51
Variedad	0,007	7

Tabla 7.5: Commensurabilidad de las familias no nutricionales del objetivo, antes y después del ajuste de pesos. Mediana de los 11 casos; el detalle por caso está en el Anexo D.

La sospecha quedó confirmada con margen (tabla 7.5 y figura 7.2). En la mediana, mover la familia de preferencias entera de su mejor a su peor valor compensaba 0,032 kcal de desviación, y la de variedad, 0,007 kcal. El caso más extremo es el de Tomás, que lleva un reparto calórico por comidas: ese término aporta 62 508 millones de unidades y la variedad 135, once órdenes de magnitud entre sumandos que conviven en la misma suma. En los dos casos con reparto por comidas activo las tres familias

caen a la diezmilésima.

Una observación matiza el diagnóstico sin cambiarlo. Cuando el solver consigue desviación nutricional exactamente cero (los casos 2 y 3 del banco), lo que queda del objetivo son solo los términos pequeños, y entonces sí deciden entre soluciones nutricionalmente perfectas. El desequilibrio actúa en los casos con conflicto real, donde ceder una fracción de kcal resulta preferible a cualquier mejora estructural. En la práctica el objetivo se comportaba como una jerarquía lexicográfica de hecho, en la que la nutrición manda y la estructura solo desempata, sin que esa jerarquía estuviese decidida ni escrita en ningún sitio.

Con Q muy por debajo de 1 en las tres familias, apliqué el remedio registrado: las preferencias pasan de peso 4 a 4 000, la no repetición blanda y la variedad de 5 a 5 000 y el reparto del plan de 4 a 4 000. Las proporciones dentro de cada grupo no cambian. Tras el ajuste una kcal de desviación sigue costando el doble que dejar un alimento entero sin usar (10 000 frente a 5 000 con pesos de fila iguales): la nutrición sigue por delante, y la estructura deja de ser invisible. La re-verificación sobre los mismos 11 casos cumplió las dos condiciones registradas. Los bancos quedaron en verde (solver 30 de 30, validador 38 de 38) con desviación calórica del 0,0 % en los dos casos del banco con objetivo calórico, y la desviación por caso fue idéntica antes y después (el 41 % de Tomás es un conflicto del propio caso, con el mismo valor en ambas pasadas). La variedad realizada mejoró o empató en 8 de los 11 casos: el caso 4 del banco pasó de 7 alimentos sin usar a 1, y de factible a óptimo probado; el 3, de 20 a 17; David, de 43 a 40; Lucía, de 46 a 43. Los dos retrocesos (Tomás de 27 a 33 y Carlos de 44 a 49) son casos que paran por límite de tiempo, donde la solución concreta oscila entre pasadas. Las medianas de Q quedaron en 32, 51 y 7.

Queda una limitación que el ajuste no resuelve. El término de reparto por comidas arrastra un factor de 1 000 propio de su formulación, porque conserva un decimal del porcentaje mediante una multiplicación cruzada, así que en los casos donde está activo sigue empequeñeciendo al resto de familias (Q de 0,07 y 0,01 en los dos casos afectados). Igualarlo exigiría renormalizar la formulación del término y no solo sus pesos, que es la renormalización profunda que quedó excluida; se documenta como asunción en la sección 5.5 y como trabajo futuro en el capítulo 8. La jerarquía que quedó tras el ajuste, con la energía por delante desde la primera kcal, la cuestioné después con un criterio clínico; es el segundo paso del mismo hilo y se cuenta en la sección siguiente.

7.2.4. La jerarquía del objetivo: bandas de tolerancia

Tras el reescalado, el coste de una kcal de desviación en un día es el peso de familia (10) por el peso de fila por las 1 000 unidades escaladas. Ninguna fila de objetivo calórico de la aplicación lleva peso 1: los clientes de demostración llevan 7, y el traductor asigna 5 por defecto. Con el peso 7, que es el habitual, la jerarquía real quedaba como enseña la tabla 7.6: una kcal de desviación en un solo día equivalía a 14 alimentos sin usar, y 20 kcal a 280, casi tres catálogos enteros. Los crudos del experimento anterior enseñaban la consecuencia. En los dos clientes de demostración con objetivo calórico que agotan el límite de tiempo, el solver terminaba con una desviación del 0,01 % y 40 y 49 alimentos sin usar de 100: gastaba el presupuesto

de búsqueda en clavar la energía al gramo, que no le importa a nadie, en vez de en la variedad, que sí se ve.

Concepto	Coste en unidades del objetivo
1 kcal de desviación en un día, fila de peso 7	70 000
1 alimento del catálogo sin usar en todo el plan	5 000
1 aparición por encima del tope de reparto	4 000
1 aparición de un alimento preferido, filas de peso 4 a 6	16 000 a 24 000
1 repetición blanda prohibida, fila de peso 5	25 000

Tabla 7.6: Jerarquía real de la función objetivo tras el reescalado de la sección 7.2.3.

La jerarquía en la que la nutrición manda siempre fue una decisión implícita de la formulación original, que el primer experimento hizo visible pero no cuestionó. Al mirar esa tabla la cuestioné con un criterio clínico: decenas de kcal arriba o abajo en un día no tienen relevancia para el profesional, y un plan repetitivo sí la tiene. Deliberé las opciones con números delante. Bajar el peso calórico a secas penaliza la primera kcal igual que la número cien, que es justo lo que no tiene sentido; una banda muerta, que solo penaliza lo que sobresale de una tolerancia, codifica la afirmación clínica tal cual. Dentro de la banda no cabe un “penaliza muy poco”, porque con pesos enteros el mínimo es 1, que con fila 7 son 7 000 por kcal, más que un alimento sin usar: dentro de la banda el coste es cero. Una banda solo diaria deja al solver pegar cada día al borde que le convenga, y un plan de 1 300 kcal con banda de 100 podría quedarse en 1 400 todos los días, que ya no es una desviación pequeña. Así que decidí dos bandas: una diaria de ± 50 kcal ($\pm 5\%$ en macronutrientes) sin penalización, y una segunda de ± 25 kcal ($\pm 2,5\%$) sobre la media del plan, con un factor 10, casi obligatoria pero blanda para no volver infactibles los casos de conflicto. Unos días compensan a otros y la media queda pegada al objetivo. La formulación está en la sección 5.5.

También registré un segundo brazo que multiplicaba por 10 los pesos de la familia estructural, porque fuera de la banda la pendiente de 70 000 por kcal seguía aplastando a la estructura; la sesión valoró un factor 100 y lo descartó por excesivo.

El pre-registro, confirmado por escrito antes de lanzar ninguna medición, comparaba tres brazos: la base (el motor sin bandas), A (bandas y ancla con los pesos de entonces) y B (lo mismo con la familia estructural multiplicada por 10). Los casos son los once del experimento anterior más uno de diseño, necesario porque ningún caso existente permitía medir la banda de macronutrientes salvo el de conflicto: María a 7 días y 5 comidas con un objetivo calórico blando de 1 500 kcal y un objetivo blando de proteína de 90 g. Tres repeticiones por caso y brazo, 90 segundos y 8 hilos, ejecución secuencial. Los casos se dividen en tres grupos: los afectados limpios (banco 2, banco 3, David, Carlos y el de diseño: objetivo blando de energía o macronutrientes sin conflicto), el control de conflicto (Tomás, cuya desviación del 41 % es del propio caso y debe seguir ahí) y el control de regresión (banco 4, banco 5, banco 8, Lucía, Sofía y Nadia: sin objetivo blando, la banda no toca su objetivo).

Un brazo sería admisible si cumplía a la vez cinco criterios: (1) estructura: en al

menos 3 de los 5 casos afectados limpios la mediana de alimentos sin usar baja o el caso pasa de límite a óptimo probado, y en ninguno de los 5 sube; (2) nutrición: en los óptimos el exceso fuera de bandas es exactamente cero, y en los que paran por tiempo queda acotado; (3) factibilidad: ningún caso pasa a infactible o sin respuesta; (4) tiempo: ningún caso que la base prueba óptimo en las 3 repeticiones pierde la prueba en más de 1; (5) bancos en verde. Si A y B eran admisibles se quedaba B, la jerarquía decidida en la deliberación; si solo A, se quedaba A y la reponderación se documentaba como no sostenida por los datos; si ninguno, se revertía el motor y el experimento se documentaba como hallazgo negativo, igual que la simetría de la sección 7.3.3 y el reescritor de la sección 7.4.1. Dejé también dos hipótesis escritas: que la media con signo se quedaría dentro de su banda por construcción y se acercaría al borde superior, y que los casos que agotaban el límite afinando la energía resolverían más rápido o probarían el óptimo.

El criterio del banco del solver cambió con el pre-registro. Los casos 2 y 3 comprobaban que la desviación calórica media fuese menor del 5 %; con la banda esa desviación sube hasta la tolerancia, y 50 sobre 1 500 es un 3,3 %, dentro del umbral antiguo, pero por casualidad de los valores. El criterio nuevo es el que dice algo sobre el modelo: cada día dentro de ± 50 kcal y la media del plan dentro de ± 25 . El banco pasa de 30 a 32 comprobaciones.

Caso	Grupo	Base	A	B
banco 2	afectado	3/3; 8,1 s; 0	3/3; 0,9 s; 0	3/3; 1,0 s; 0
banco 3	afectado	3/3; 37,7 s; 20	3/3; 8,7 s; 22	3/3; 8,7 s; 22
David	afectado	0/3; 90 s; 33	3/3; 41,1 s; 13	3/3; 35,8 s; 13
Carlos	afectado	0/3; 90 s; 44	0/3; 90 s; 26	0/3; 90 s; 15
diseño macro	afectado	0/3; 90 s; 3	3/3; 9,7 s; 0	3/3; 12,2 s; 0
Tomás	conflicto	3/3; 14,8 s; 29	3/3; 24,5 s; 31	2/3; 19,6 s; 27
banco 4	control	3/3; 48,1 s; 1	3/3; 48,2 s; 1	3/3; 54,1 s; 1
banco 5	control	3/3; 30,7 s; 12	3/3; 30,8 s; 12	3/3; 64,0 s; 12
banco 8	control	0/3; 90 s; 37	0/3; 90 s; 37	0/3; 90 s; 40
Lucía	control	0/3; 90 s; 38	0/3; 90 s; 38	0/3; 90 s; 39
Sofía	control	3/3; 74,6 s; 9	3/3; 73,7 s; 9	3/3; 75,0 s; 9
Nadia	control	3/3; 76,8 s; 12	3/3; 76,6 s; 12	3/3; 76,6 s; 12

Tabla 7.7: Bandas de tolerancia: óptimos probados de 3 repeticiones, mediana del tiempo y mediana de alimentos del catálogo sin usar, por caso y brazo. Las desviaciones con signo están en el Anexo D.

La campaña fueron 108 resoluciones (tabla 7.7, figuras 7.3 y 7.4), sin ninguna repetición descartada por reloj. En los casos afectados el efecto es el previsto y grande. David pasa de agotar los 90 segundos con 33 alimentos sin usar a probar el óptimo en 41 segundos con 13; el caso de diseño, de 90 segundos sin óptimo a 10 segundos con los 100 alimentos usados; el banco 2 de 8 segundos a 1 y el banco 3 de 38 a 9; Carlos, que sigue en el límite, baja de 44 a 26 alimentos sin usar. La hipótesis registrada sobre los tiempos se cumple: al dejar de exigir la energía al gramo, los casos que gastaban el presupuesto en afinarla resuelven en una fracción del tiempo o prueban el óptimo. Los seis casos de control quedan idénticos en el brazo A, que solo cambia el objetivo donde hay un objetivo blando; en B cambian

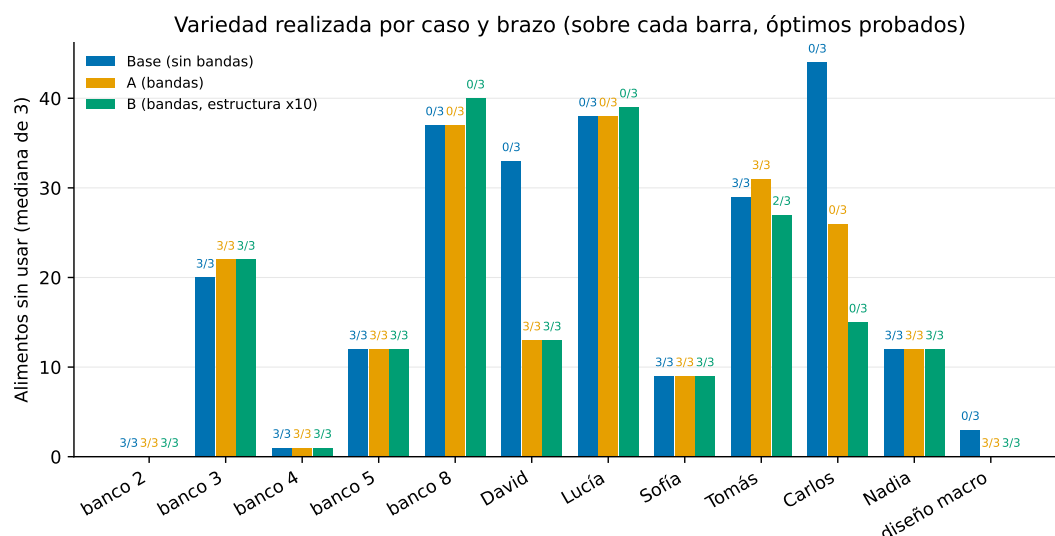


Figura 7.3: Variedad realizada por caso y brazo, con los óptimos probados sobre cada barra.

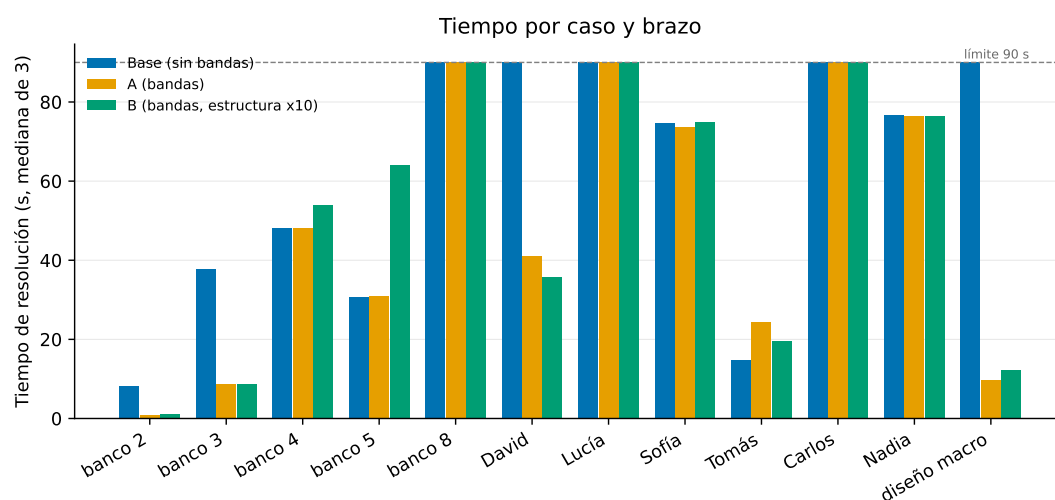


Figura 7.4: Tiempo de resolución por caso y brazo (mediana de 3 repeticiones).

algo por el reescalado (el banco 5 tarda el doble sin perder el óptimo, Lucía y el banco 8 devuelven 1 y 3 alimentos menos en el límite, y Tomás pierde un óptimo de tres). Como en los casos limpios ningún día sale de las bandas, A y B tienen en ellos el mismo óptimo salvo escala; la diferencia entre ambos en Carlos (26 frente a 15) es un efecto del reescalado sobre la búsqueda, no de la jerarquía.

El solver usa las bandas: varios días se pegan al borde de ± 50 y la media se acerca al suyo de ± 25 . La hipótesis de que la deriva iría hacia el borde superior se cae: en el banco 2 y en David va hacia abajo en todas las repeticiones, y en el banco 3 y en el caso de diseño cambia de signo entre repeticiones. Lo que importa para el modelo es que el ancla funciona por construcción: en ninguna de las 45

resoluciones de los casos limpios con bandas hay exceso fuera de la banda diaria ni de la banda de la media. Tomás sigue devolviendo plan con su energía 1 240 kcal por debajo del objetivo en los tres brazos, que era la razón de dejar el ancla blanda. La recomposición externa del objetivo coincidió con el valor reportado en 104 de las 108 resoluciones; las cuatro diferencias son de Tomás, con la holgura ya explicada en la sección anterior. El pre-registro sirve también para esto: las hipótesis equivocadas quedan escritas.

Criterio	A	B
1. Estructura: 3 de 5 casos limpios mejoran	pasa (David, Carlos, diseño)	pasa (los mismos)
1. Estructura: ninguno empeora	falla: banco 3 de 20 a 22	falla: banco 3 de 20 a 22
2. Fuera de banda cero en óptimos, acotado en el límite	pasa (cero en todos)	pasa (cero en todos)
3. Factibilidad	pasa	pasa
4. Óptimos probados conservados	pasa	pasa (Tomás pierde 1 de 3)
5. Bancos: solver con el criterio redefinido	32 de 32	32 de 32
5. Bancos: validador	38 de 38	38 de 38

Tabla 7.8: Evaluación de los criterios pre-registrados, brazo a brazo.

El veredicto tiene dos tiempos y conviene dejarlos en este orden. En la lectura estricta del pre-registro (tabla 7.8) ningún brazo es admisible: los dos cumplen la mitad exigente del criterio 1 y los dos fallan la cláusula de que ningún caso empeore, porque el banco 3 sube de 20 a 22 alimentos sin usar. Según la regla registrada eso llevaba a revertir el motor. Lo dejo escrito antes de nada porque el valor del método está en que el criterio no se reescriba después de ver los datos.

Los datos dicen algo más sobre ese fallo. El banco 3 termina óptimo en las tres repeticiones de los tres brazos, pero óptimo en este motor significa dentro de la brecha relativa del 2 % con que para el solver. Con un objetivo de unos 2,12 millones de unidades, ese 2 % son 42 000 unidades, algo más de 8 alimentos sin usar, y la propia base oscila entre 17 y 21 alimentos sin usar según la repetición. La subida de 20 a 22 está dentro del ruido de parada del solver: el criterio se redactó sin tener en cuenta esa holgura. La lección metodológica es que un criterio sobre conteos estructurales debe llevar una tolerancia al menos del tamaño de la brecha de parada, o exigir óptimos a brecha cero; queda anotada para futuros pre-registros.

Con los datos delante, y como decisión tomada a posteriori y registrada como tal, mantuve el cambio y elegí el brazo A: bandas con ancla de la media y los pesos de siempre. La desviación respecto al pre-registro es una sola y acotada, aceptar el banco 3 como neutro por estar dentro de la holgura de parada. Es el tercer caso del patrón “criterio antes, datos después” de este capítulo: el reescritor y la simetría fueron negativos limpios, y aquí el criterio falló por su propia redacción.

Elegí A frente a B, en contra de la regla registrada que prefería B si ambos valían, por una razón que salió de mirar los resultados. Más alimentos distintos no hacen

mejor una dieta: una semana con 87 alimentos diferentes es difícil de comprar y de seguir, y el término de variedad es un proxy que solo debe decidir entre planes que ya son equivalentes en lo que importa. El brazo B multiplica por diez el peso de ese proxy frente a la energía justo en el único sitio donde A y B se distinguen (cuando algún día tiene que salirse de la banda), y tiene costes medidos en los controles. A se queda con casi toda la ganancia sin tocar nada fuera de los objetivos blandos, y deja el control de la variedad en el estilo clínico del profesional, cuyas preferencias ya pesan más que la variedad: en el banco 3 la preferencia vegetariana llena los 140 huecos del plan y la variedad no se lo impide. Con el brazo A los bancos quedaron en 32 de 32, 38 de 38 y 14 de 14 en la cola asíncrona, y la prueba de extremo a extremo contra el despliegue dio 29 de 29 con el motor nuevo servido; la desviación reportada en el caso con objetivo calórico pasó del 0,0 al 2,57 %, señal de que producción ya servía las bandas.

Las bandas conectan este eje con el siguiente. Son también la mayor mejora de tiempos del proyecto en los casos con objetivo calórico, porque el solver deja de demostrar optimalidad sobre la igualdad al gramo; el resto de la eficiencia se mide en la sección 7.3.

7.3. La eficiencia del motor

Esta sección responde a dos preguntas de los directores del TFG sobre el rendimiento: las cifras del tamaño del problema que sostienen el argumento, y experimentos de escalado y la rotura de simetría entre días. La búsqueda de mejoras trajo además una que no estaba prevista, las cotas de dominio, y la sección cierra con lo que el alojamiento gratuito enseñó sobre el motor.

7.3.1. El tamaño del problema

La tabla 7.9 recoge el modelo que construye el caso real de la aplicación (un cliente de demostración con 4 restricciones clínicas, 4 comidas al día y el catálogo completo de 100 alimentos), medido instrumentando la construcción por etapas y capturando el resumen de presolve de CP-SAT.

Configuración	Booleanas	Enteras	Restricciones	Tras pre-solve
3 días, 4 comidas	1 702	1 627	4 257	2 037
7 días, 4 comidas (caso real)	3 802	3 663	9 634	4 505
14 días, 4 comidas	8 002	7 226	19 574	9 512

Tabla 7.9: Tamaño del modelo con el catálogo de 100 alimentos: variables de decisión, restricciones y variables que quedan tras el presolve de CP-SAT.

De las 9 634 restricciones del caso real, 6 356 son del modelo base (enlaces entre presencia y gramos, cuantización por unidades y sumas de nutrientes), 1 592 de las reglas estructurales, 987 de la plausibilidad, 399 de las 4 restricciones clínicas del

cliente y 300 del ensamblaje del objetivo. Tras el presolve, CP-SAT reduce las 7 465 variables a 4 505 y trabaja con 5 530 restricciones internas, dominadas por las lineales y las cadenas booleanas de los enlaces.

Dos lecturas. El tamaño crece lineal con los días (duplicar días duplica variables), así que la dificultad práctica no viene del tamaño bruto. Y el caso real es un problema moderado para CP-SAT (miles de variables, no millones), y aun así los casos con reparto por comidas agotan los 90 segundos: la dureza está en probar optimalidad sobre ese término, no en propagar el modelo.

7.3.2. Escalado

El escalado barre una dimensión cada vez alrededor del caso por defecto (7 días, 4 comidas, catálogo completo), con el mismo cliente y sus restricciones reales, 3 repeticiones por celda. El submuestreo del catálogo es sistemático por identificador (un alimento de cada k), reproducible y repartido entre grupos de alimentos.

Celda	Tiempo medio	Estado	Brecha media
3 días	90 s	factible	28,5 %
7 días (defecto)	90 s	factible	6,9 %
14 días	90 s	factible	9,1 %
3 comidas	90 s	factible	16,0 %
4 comidas (defecto)	90 s	factible	6,9 %
5 comidas	90 s	factible	6,1 %
catálogo de 30	8,7 s	óptimo	1,9 %
catálogo de 60	67,4 s	óptimo	1,9 %
catálogo de 100 (defecto)	90 s	factible	6,9 %

Tabla 7.10: Escalado por dimensión, el resto en el caso por defecto. Medias de 3 repeticiones.

La lectura principal de la tabla 7.10 y la figura 7.5 es que el tamaño del catálogo gobierna el coste: con 30 alimentos el caso se prueba óptimo en 8,7 segundos, con 60 en 67,4 y con 100 se agota el presupuesto. Es coherente con la tabla anterior: las variables crecen con el producto de alimentos, días y comidas, pero el catálogo multiplica además las alternativas por hueco, que es lo que dispara la búsqueda. Con el catálogo completo, todas las celdas de días y comidas saturan los 90 segundos y la comparación se traslada a la brecha de optimalidad, moderada en el entorno del caso por defecto (6 a 9 %) y disparada en las configuraciones cortas (28,5 % a 3 días, 16 % a 3 comidas).

Conviene decir por qué menos días o menos comidas no facilitan el problema en brecha relativa, porque sorprende. El cliente del experimento lleva un reparto por comidas, cuyo término domina el objetivo (sección 7.2.3); con menos días o comidas el valor absoluto del objetivo es menor y la misma holgura absoluta pesa más en términos relativos. La brecha relativa entre duraciones es indicativa, no comparable. El dato robusto es el del catálogo, donde el estado cambia de categoría, de óptimo probado a límite agotado.

Escalado por dimensión (resto en el caso por defecto). Con el tiempo en el límite, la brecha separa las configuraciones

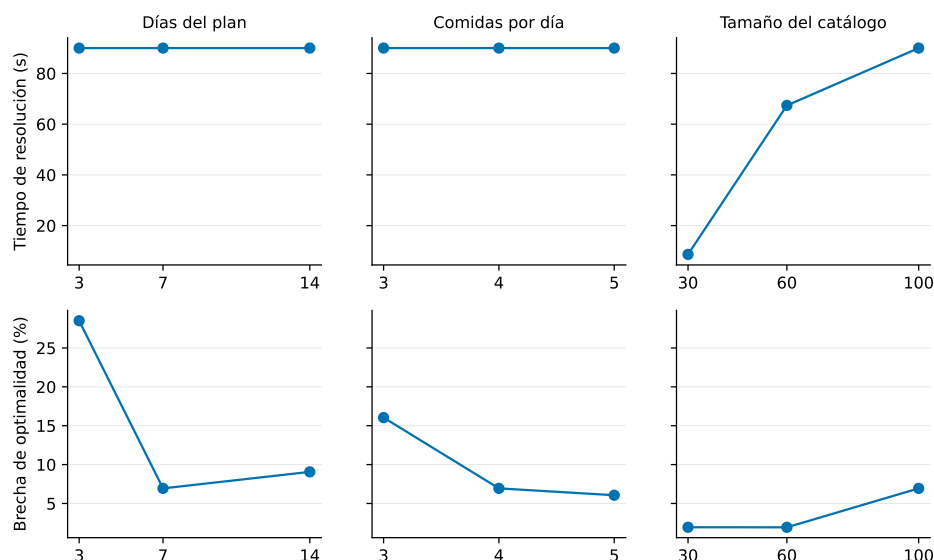


Figura 7.5: Escalado por dimensión: tiempo de resolución (arriba) y brecha de optimalidad (abajo). Con el tiempo en el límite, la brecha separa las configuraciones.

7.3.3. Rotura de simetría entre días

Dos días de un plan son intercambiables cuando ninguna restricción los distingue: permutar sus asignaciones produce otra solución válida con el mismo objetivo, y el solver puede explorar hasta $7! = 5040$ variantes equivalentes de un mismo plan semanal. Uno de los directores del trabajo de fin de grado propuso romper esa simetría con un score entero de pesos $2^{F(M-m)+(F-f)}$ que ordenase los días. Ese score define un orden lexicográfico, pero materializar el número con F cercano a 100 desborda cualquier entero de 64 bits, así que la adaptación implementa el mismo orden sin materializarlo: una cadena lexicográfica dura entre días consecutivos sobre el vector de presencias de cada día, con los booleanos de prefijo igual habituales en programación con restricciones. En el caso de 7 días, 4 comidas y 100 alimentos añade unos 4800 booleanos auxiliares y 10000 restricciones.

La condición de corrección, registrada antes de implementar, es que la cadena solo se añade cuando permutar los días es una simetría verdadera del problema: plan de 7 días o menos (con más, las ventanas deslizantes de variedad semanal distinguen los días), ninguna restricción clínica de no repetición con separación mayor que un día, y toda restricción de raciones máximas por periodo con ventana de un día o de la duración entera. Si no se cumple, imponer un orden podría excluir soluciones óptimas. La rotura ordena las presencias y no los gramos; es parcial y correcta. La medición registrada fueron los diez casos que activan la condición (el caso 8 del banco queda fuera por su no repetición clínica), 5 repeticiones por brazo, comparando tiempo, estado, objetivo y desviación calórica; si el cambio empeoraba, los datos se presentaban antes de decidir.

La cadena lexicográfica salió peor (tabla 7.11 y figura 7.6): seis casos empeoran

Caso	Sin orden	Cadena lexicográfica	Orden escalar
banco 2	óptimo, 4,9 s	óptimo, 5,2 s	óptimo, 3,2 s
banco 3	óptimo, 22,4 s	óptimo, 23,3 s	óptimo, 9,7 s
banco 4	óptimo, 69,7 s	límite en 5 de 5, objetivo hasta 2,8 veces peor	límite en 5 de 5, objetivo 20 veces peor
banco 5	óptimo, 66,7 s	límite en 5 de 5, objetivo 1,4 veces peor	límite en 5 de 5, objetivo 1,25 veces peor
David	límite, desviación 0,02 %	límite, desviación 0,56 %, objetivo 13 veces peor	límite, mejor incumbente
Lucía	límite	límite, algo peor	límite, algo mejor
Sofía	1 de 5 óptimos, 80,8 s	3 de 5 óptimos, 79,7 s, mejor objetivo	5 de 5 óptimos, 63,5 s, mejor objetivo
Tomás	óptimo, 9,8 s	óptimo, 11,2 s	óptimo, 15,3 s
Carlos	límite	límite, mejor incumbente	límite, mejor incumbente
Nadia	óptimo, 52,9 s	3 de 5 sin probar el óptimo, 85,6 s	óptimo, 69,8 s

Tabla 7.11: Rotura de simetría entre días en sus dos codificaciones, frente al motor sin orden (5 repeticiones por caso y brazo). Los valores de objetivo están en el Anexo D.

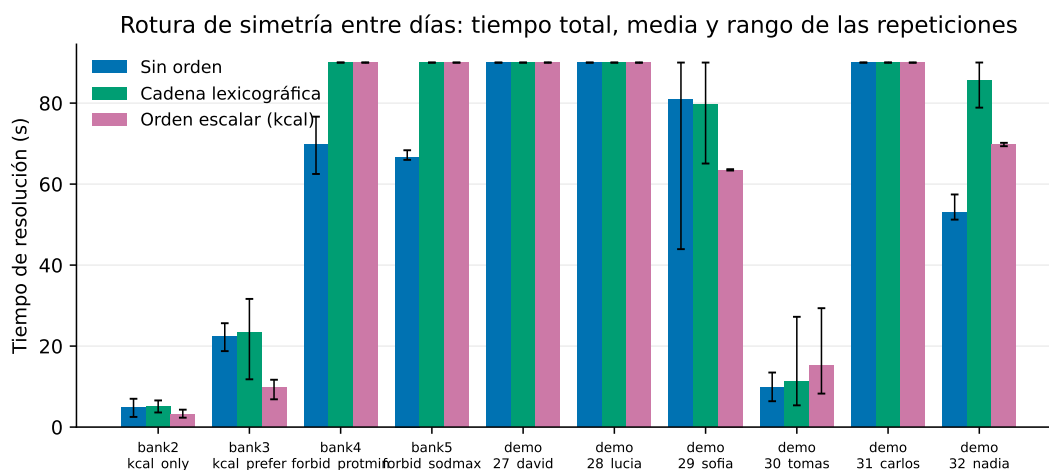


Figura 7.6: Tiempo de resolución por caso y brazo, media y rango de las 5 repeticiones. Donde las tres barras tocan el límite, la diferencia está en el estado y en la calidad del incumbente, que recoge la tabla 7.11.

(los bancos 4 y 5 y Nadia pierden la prueba de optimalidad que tenían, y David devuelve un incumbente 13 veces peor), tres mejoran (Sofía con claridad, Carlos en el incumbente) y dos quedan igual. Tras ese resultado registré y medí una variante mínima con la misma semántica de orden y coste casi nulo: energía diaria no creciente entre días consecutivos, seis restricciones lineales sobre sumas ya existentes y cero variables auxiliares. El balance es mejor (seis casos mejoran, el banco 3 a menos de la mitad de tiempo), pero los daños donde falla son severos (el banco 4 pasa de óptimo probado a un objetivo veinte veces peor en el límite) y no hay un predicado que separe a priori los casos que mejoran de los que empeoran: la hipótesis natural, que ayuda cuando hay un objetivo calórico que ancle la magnitud ordenada, muere

con los datos, porque Sofía y Lucía mejoran sin objetivo calórico y Tomás empeora teniéndolo.

Las dos codificaciones se retiran. La explicación es coherente con lo que hace el solver por dentro: CP-SAT ya detecta y explota simetrías en el presolve y reparte estrategias entre sus hilos de búsqueda, así que un orden manual compite con un mecanismo interno que funciona, y su efecto real es reordenar la búsqueda, a veces a favor y a veces en contra, sin patrón explotable. No se cablea nada en el motor; el código queda en el script del experimento, reproducible, y el hallazgo negativo se documenta con estas tablas. Es la misma lógica que la decisión del reescritor en el estudio de formatos: el criterio se fijó antes de medir, los datos no acompañan y la decisión se toma con ellos.

7.3.4. Cotas de dominio ajustadas

La búsqueda de eficiencia no terminó en la simetría. Al revisar el modelo con la mirada en su tamaño efectivo apareció una holgura heredada: la cota superior de la suma escalada de cada nutriente por comida se calculaba como si el catálogo entero pudiera coincidir en una comida (número de alimentos por la mayor ración por la mayor densidad del catálogo, tres pesimismos multiplicados). La regla R2 garantiza que una comida lleva a lo sumo 4 alimentos, así que la cota correcta es la suma de las 4 mayores aportaciones individuales alcanzables. Los techos de dominio bajan en un factor de 25 a 200 según el nutriente, y las variables de exceso y defecto del objetivo heredan la reducción (sección 5.5).

Lo que hace seguro este cambio, y lo distingue de la simetría, es que una cota válida no cambia el conjunto de soluciones ni el óptimo: solo cuánto tarda el solver en podar. El riesgo era únicamente de rendimiento, y se midió con el mismo protocolo (los mismos 10 casos, 5 repeticiones).

Caso	Cotas holgadas	Cotas ajustadas
banco 2	óptimo, 4,9 s	óptimo, 5,3 s
banco 3	óptimo, 22,4 s	óptimo, 21,6 s
banco 4	óptimo, 69,7 s	óptimo, 45,4 s
banco 5	óptimo, 66,7 s	óptimo, 26,4 s
David	límite, objetivo 430 100	límite, objetivo 260 400
Lucía	límite, objetivo -2 414 000	límite, objetivo -2 058 000
Sofía	1 de 5 óptimos, 80,8 s, objetivo 110 000	5 de 5 óptimos, 40,2 s, objetivo 45 000
Tomás	óptimo, 9,8 s	óptimo, 27,2 s
Carlos	límite, objetivo 495 300	límite, objetivo 449 500
Nadia	óptimo, 52,9 s	3 de 5 óptimos, 80,1 s, mismo objetivo

Tabla 7.12: Cotas de dominio apoyadas en R2 frente a las cotas heredadas (5 repeticiones por caso). El objetivo se minimiza: menor es mejor.

Leída sobre los casos que rozan o agotan el límite, que son el problema real de rendimiento del motor, la tabla 7.12 mejora 5 de 6: los dos casos pesados del

banco recortan un 35 y un 60 % manteniendo el óptimo probado, Sofía pasa de 1 óptimo de 5 a 5 de 5 en la mitad de tiempo, y David y Carlos devuelven mejor incumbente con el mismo presupuesto. El coste es que tres casos que iban sobrados se vuelven más lentos sin dejar de resolverse dentro del límite, y que Lucía devuelve un incumbente peor. Como beneficio lateral, dominios más pequeños reducen la memoria del proceso, que es el recurso crítico del alojamiento gratuito. El cambio se mantiene: mejora sistemática donde el motor sufría, coste acotado donde iba sobrado, mecanismo causal claro y bancos en verde.

7.3.5. El motor en el alojamiento gratuito

El backend corre en el plan gratuito de un alojamiento con media CPU virtual y 512 MB de memoria, y varias decisiones del capítulo 4 salieron de medir el motor ahí. Se reúnen aquí las cifras.

El límite de tiempo del solver se calibró contra esa CPU y no contra la máquina de desarrollo. Con el primer despliegue, el caso más pesado del banco de extremo a extremo (objetivo calórico exacto con reparto por comidas a 7 días), que tardaba unos 15 segundos hasta el óptimo en local, expiraba siempre en producción con el límite de 10 segundos y devolvía infactible; el límite subió a 30. Al ampliar el catálogo de 28 a 58 alimentos la prueba de extremo a extremo dio 27 de 29 con 30 segundos y 28 de 29 con 60, y solo 29 de 29 con 90: los fallos eran casos de objetivo calórico que en local resolvían al óptimo con desviación cero, y en producción no llegaban. Un detalle de esa medición vale como lección: el caso 2, que parecía trivial (un objetivo de 1500 kcal, 3 segundos en local), era en realidad más pesado porque la API le suma las restricciones que el cliente ya tiene en la base de datos, y hay que contarlas al medir.

El paralelismo también se decidió con una medida. El solver llevaba 8 hilos de búsqueda fijos, y en producción cada llamada síncrona superaba la memoria de la instancia y reiniciaba el proceso. Con 8 hilos y media CPU no hay más velocidad, solo más memoria (cada hilo mantiene su propia copia del estado de búsqueda) y cambios de contexto. El número de hilos pasó a configurarse por entorno, 8 en desarrollo y 2 en producción. El coste medido en local con 2 hilos es que más casos del banco agotan los 90 segundos y terminan factibles en vez de óptimos (los casos 1, 3 y 8, frente a solo el 8 con 8 hilos), con todas las comprobaciones de calidad intactas; en producción la prueba de extremo a extremo volvió a dar 29 de 29 sin reinicios.

Al pasar el catálogo de 58 a 100 alimentos volví a medir con línea base, y esa línea base ya no estaba en 29 de 29: el caso 2 salía al 8,53 % de desviación, y repetido tres veces daba 1,33, 5,03 y 6,09 %, siempre con los 90 segundos agotados y sin cota inferior alguna (en producción el solver devuelve el incumbente que alcanzó, sin acotar nada), mientras en local el mismo caso era óptimo con desviación cero en 2,1 segundos. El caso era inestable desde antes del cambio, y el 29 de 29 anterior había sido una pasada buena. Sin la línea base, esa degradación se le habría achacado al catálogo. Con 100 alimentos el caso 2 mejoró (1,36, 1,36 y 0,13 %: más combinaciones cuadran 1500 kcal y el buen incumbente aparece antes) y el caso con reparto por comidas empeoró (12 a 14 %); en local, 100 alimentos fue el mismo óptimo al doble

de tiempo. El coste real en producción es el tiempo de la petición completa, que llegó a 114 segundos: 90 de resolución más la carga del catálogo, la validación y la serialización de 350 raciones.

Ampliar el catálogo tuvo un efecto que no había previsto: encarece demostrar una infactibilidad más que resolver un caso factible. El caso de infactibilidad del banco pide un mínimo de calcio con las familias ricas en calcio prohibidas. Con 100 alimentos, a 3 000 mg el caso resultó factible; a 3 200 el solver agotaba el tiempo sin demostrar nada y devolvía un núcleo de conflicto vacío; a 3 500 la demostración tardaba 13 segundos en local, demasiado para producción; a 6 000 tardaba 2 segundos, y ahí quedó calibrado el caso. Una infactibilidad sin demostrar devuelve un núcleo vacío, que degrada la explicación de la sección 5.6: responder y diagnosticar son cosas distintas, y la segunda es más cara. Los umbrales de los casos de infactibilidad se recalibran con el catálogo, y ya van dos veces.

La última medida es la del estilo clínico. Al sembrar dos restricciones de ámbito nutricionista, los casos con prohibiciones de ese profesional y el propio botón de generar pasaron a devolver infactible con núcleo vacío en producción: dos términos blandos más en el objetivo empujaron la búsqueda del primer incumbente fuera de los 90 segundos con media CPU. La atribución se hizo con una sonda, quitando y poniendo las dos filas (sin ellas los mismos casos eran factibles, y uno óptimo en 60 segundos; con ellas, tres de tres sin incumbente), y en local ni se apreciaba. Es la demostración empírica de que una restricción de ámbito nutricionista es una restricción de verdad del modelo, con su coste. Con la capa de plausibilidad puesta, el mismo estilo clínico volvió a resolverse en producción, factible en 92 segundos, y los datos de demostración lo llevan en la cartera del segundo nutricionista.

7.4. El copiloto de IA

Las piezas de IA del capítulo 6 se evaluaron de dos formas: un estudio empírico sobre el traductor, que fue la primera evaluación pre-registrada del proyecto, y el hilo de hallazgos de primera mano sobre los modelos locales que salió de construir y medir cada pasada.

7.4.1. El estudio de formatos del mensaje

La pregunta era si la forma de redactar el mensaje (la misma información expresada de maneras distintas) afecta a la calidad de la traducción a restricciones, y si la diferencia justificaba un módulo previo que reescribiese el mensaje al formato óptimo antes de traducir. Mis directores habían sugerido ese estudio en la segunda reunión de julio. La decisión sobre el módulo quedó condicionada al resultado, y el criterio se fijó por escrito antes de la primera medición: el reescritor se construye si y solo si el mejor formato alternativo supera al natural (el texto libre de un nutricionista, la línea base) en al menos 10 puntos porcentuales de acierto por mensaje, y además iguala o supera al natural en al menos 9 de los 13 mensajes. La primera condición exige una mejora relevante y la segunda que sea consistente y no el efecto de un par de mensajes atípicos.

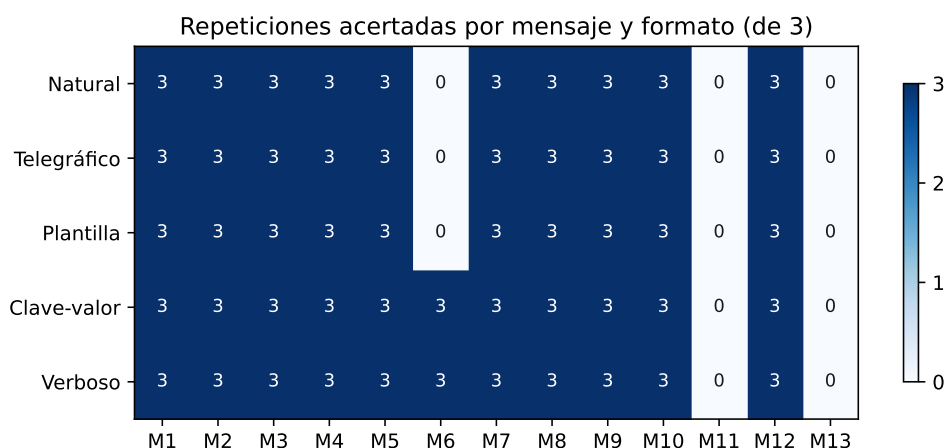


Figura 7.7: Repeticiones acertadas por mensaje y formato, de 3. Diez mensajes se traducen bien en los cinco formatos; los fallos se concentran en tres.

Se compararon cinco formatos, del más humano al más artificial, con los mismos requisitos en cada mensaje: natural, telegráfico (viñetas), plantilla (campos con etiqueta), clave-valor (pares casi de configuración) y verboso (conversacional con ruido de consulta). Los 13 mensajes llevan su lista de restricciones esperadas verificada contra el catálogo real, 24 en total, y cubren 10 de los 14 tipos traducibles; se excluyó a propósito el límite de raciones con alimento por nombre, una limitación ya conocida del modelo local que habría medido el modelo y no el formato. El acierto se mide por propiedades con tolerancia: una restricción esperada cuenta si el traductor produce una válida con su mismo tipo, su mismo objetivo resuelto contra el catálogo y, si hay número, un valor dentro del 10 %; las válidas que no corresponden a ninguna esperada cuentan como espurias y las que el sistema rechaza se contabilizan aparte; un mensaje acierta cuando están todas sus esperadas y no hay espurias. La prioridad no puntúa, porque es criterio clínico del modelo y no fidelidad al texto. Cinco formatos por trece mensajes por tres repeticiones son 195 llamadas reales al traductor con el mismo modelo y configuración que en producción, sin escribir trazas en la base de datos: los crudos van a fichero y de ahí salen todas las cifras. El harness se autoproboó antes con un cliente simulado que devuelve respuestas perfectas.

Formato	Acierto	Exhaustividad	Precisión	Espurias	Rechazadas
Natural	76,9 %	83,3 %	90,9 %	6	9
Telegráfico	76,9 %	87,5 %	95,5 %	3	6
Plantilla	76,9 %	83,3 %	95,2 %	3	9
Clave-valor	84,6 %	87,5 %	100 %	0	9
Verboso	84,6 %	87,5 %	100 %	0	9

Tabla 7.13: Estudio de formatos: acierto por mensaje, exhaustividad y precisión por restricción, y restricciones espurias y rechazadas, por formato (13 mensajes, 3 repeticiones).

Dos observaciones previas al análisis de la tabla 7.13. La desviación entre repe-

ticiones fue cero en todos los formatos: con temperatura 0 y salida estructurada el modelo respondió de forma idéntica en las tres pasadas de cada celda, así que el acierto es todo o nada por mensaje. Esto matiza lo que había observado al construir el traductor, donde el modelo variaba entre llamadas: con el prompt y el esquema definitivos, es determinista. Y el resultado es notablemente plano: el mejor formato alternativo (clave-valor, empatado con verboso) supera a la línea base en 7,7 puntos, y la exhaustividad se mueve en una banda de 4 puntos.

La figura 7.7 es la clave del estudio: 10 de los 13 mensajes se traducen perfectamente en los cinco formatos y los fallos se concentran en tres. Sondeé cada uno a mano. En el mensaje 6 (reparto calórico: desayuno 30 %, comida 40 %) el modelo en formato natural captura ambas comidas pero las emite como dos restricciones de reparto separadas en vez de una sola con el reparto conjunto; para el motor ambas representaciones son equivalentes, pero la métrica registrada exige la fila única y lo puntúa como fallo. En telegráfico y plantilla el fallo es real: pierde una de las dos comidas. En el mensaje 11 (evitar pescado, preferir legumbres) el modelo confunde categoría con alimento en los cinco formatos, y emite prohibición y preferencia de alimento por nombre en vez de las etiquetas del catálogo; la capa determinista no encuentra alimentos con esos nombres y rechaza ambas, que es el comportamiento defensivo diseñado. En el mensaje 13 (no repetir carne roja en días consecutivos) el modelo elige el tipo correcto y se inventa el código de la etiqueta, invirtiendo el real, en los cinco formatos, y la resolución contra el catálogo cerrado lo rechaza. Los mensajes 11 y 13 fallan por igual en los cinco formatos, así que son limitaciones del modelo frente a tipos concretos, no efectos del formato. Y el único mensaje donde los formatos difieren, el 6, difiere en parte por un artefacto de representación: si se acepta la representación equivalente de dos filas, el natural alcanza el mismo 84,6 % y la diferencia entre el mejor formato y la línea base baja de 7,7 a 0 puntos. Esta relectura es posterior al pre-registro y por eso no interviene en la decisión, pero apunta en la misma dirección.

Las medidas secundarias no cambian nada. Los tokens de entrada apenas varían entre formatos (unos 1 550 por llamada) porque el prompt de sistema, con el catálogo cerrado y los ejemplos, domina el tamaño; el verboso solo añade unos 40. Sobre una API remota de pago por token el coste por traducción sería el mismo con cualquier formato, así que el formato tampoco es una palanca económica. Las latencias, de 16 a 20 segundos por llamada en el equipo local, difieren menos que su dispersión. Las tablas y figuras completas están en el Anexo D.

Como medida secundaria, cada uno de los 65 pares formato-mensaje se pasó una vez por las pasadas de validación previa de la sección 6.2 (figura 7.8). Las pasadas, pensadas para texto humano, penalizan los formatos artificiales: el telegráfico pierde 5 de 13 mensajes antes de llegar al traductor, y en clave-valor el detector de idioma llega a clasificar un mensaje como lengua desconocida. Si el estudio hubiese justificado el reescritor, su sitio era detrás de las pasadas y nunca delante; y descarta también pedir al nutricionista que escriba en un formato artificial. La segunda lectura es un hallazgo colateral sobre el propio filtro: la comprobación de ámbito bloqueaba dos mensajes naturales perfectamente legítimos, frases cortas e imperativas sin cifras; su arreglo se cuenta en la sección siguiente.

La decisión fue no construir el reescritor. El criterio exigía 10 puntos y el me-

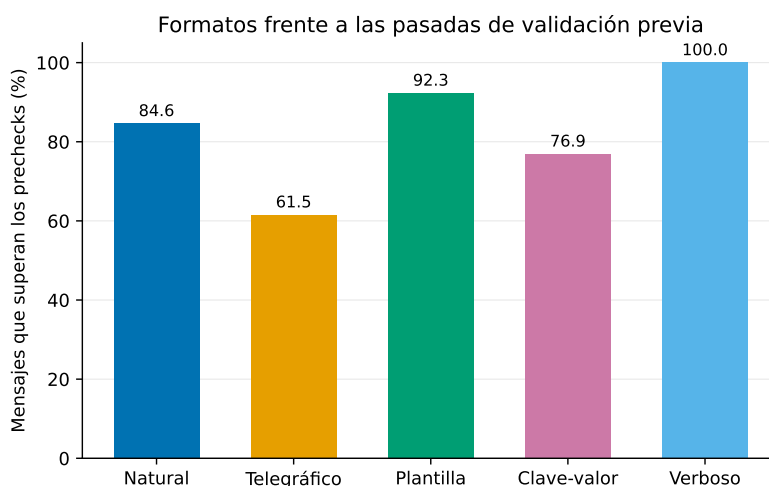


Figura 7.8: Mensajes que superan las pasadas de validación previa, por formato (una pasada por cada uno de los 65 pares formato-mensaje).

jor formato mejora 7,7; la relectura del mensaje 6 reduce la diferencia real a cero. El traductor es robusto al formato del mensaje gracias a su arquitectura: con un catálogo cerrado en el prompt, salida estructurada y una capa determinista de resolución y validación detrás, da igual que el nutricionista escriba prosa, viñetas o pares clave-valor. Los fallos residuales son del modelo frente a dos situaciones concretas, y la vía de mejora es el catálogo del prompt o un modelo mayor. Un reescritor habría añadido una llamada más por petición, más latencia y otra fuente de error, para perseguir una mejora que los datos no respaldan. Los límites del estudio quedan dichos: mide un único modelo, los 13 mensajes fueron redactados para el estudio y no recogidos de nutricionistas en producción, y el harness guarda los conteos de cada celda y no las restricciones producidas, por lo que la caracterización de los fallos necesitó sondas manuales.

7.4.2. Comportamiento de los modelos locales

Construir el traductor y las pasadas de validación previa sobre modelos locales pequeños dejó una serie de hallazgos de primera mano que ninguna documentación adelantaba. Se recogen aquí porque condicionaron decisiones de la sección 6.2 y porque son el tipo de evidencia que un tribunal puede querer contrastar.

El modelo omite los campos que el esquema marca como opcionales, y el que más omite es el valor numérico. Con el valor opcional en el esquema de salida, el traductor devolvía mínimos de nutriente sin número, que son inútiles. Forzar el valor a requerido y numérico solo en el esquema que se le pasa al modelo (el de parseo sigue tolerante) subió el banco de 10 a 13 aciertos de 14. Lo que no se arregló con ningún prompt fue el límite de raciones con alimento por nombre: cuando el alimento compite con un número y una ventana de días, el modelo no rellena el nombre. Prohibir o preferir un alimento por nombre sí funciona, porque el alimento es el sujeto claro de la frase. El sistema hace lo correcto, rechaza la restricción

incompleta en vez de inventar el alimento, y la limitación quedó documentada y fuera del estudio de formatos.

Los modelos pequeños hacen eco de los ejemplos del prompt. Con ejemplos concretos en el prompt de sistema, los modelos de 1,5 y 3 mil millones de parámetros devolvían el ejemplo como veredicto del mensaje real, y una de las pasadas llegó a detectar un conflicto de 2 200 kcal que solo existía en el ejemplo. Los prompts definitivos de las pasadas son abstractos, sin cifras de ejemplo (Anexo C).

El orden de los campos del esquema es un razonamiento barato. Bajo salida estructurada el modelo genera los campos en el orden en que se declaran, así que un campo de análisis antes del veredicto le obliga a razonar antes de responder. Eso es lo que hizo fiables la comprobación de contradicción con el modelo de 3 mil millones y la de completitud con el de 7 mil millones. El encuadre importa más que el modelo en la completitud: preguntar qué falta sesga a encontrar siempre algo (todos los modelos alrededor de 2 a 4 aciertos de 6), y encuadrarlo como una petición que apunta a un dato que el mensaje no contiene dio 6 de 6.

Ningún modelo pequeño identifica el idioma de forma fiable. Los tres fallaban en las dos direcciones (un texto dietético en alemán como dentro de ámbito, un español sin tildes como otro idioma) y meter la pregunta en el prompt degradaba la comprobación de ámbito, así que el idioma pasó a una puerta determinista con un detector estadístico, que bloquea solo con probabilidad alta y nunca lenguas vecinas del español.

De ahí salió la decisión de un modelo por pasada. Un benchmark superficial daba al modelo más pequeño 9 aciertos de 10 en conjunto; al medir cada comprobación en profundidad con su propio banco, ese modelo solo era fiable en el ámbito, y en completitud y contradicción daba falsos positivos que habrían bloqueado mensajes válidos. La matriz final asignó el modelo más pequeño al ámbito, el de 7 mil millones a la completitud y el de 3 mil millones a la contradicción, con 24 aciertos de 25 frente a unos 13 de 25 usando el pequeño para todo. El coste son unos 25 segundos por mensaje, asumibles en un flujo asíncrono.

Esa matriz tuvo que corregirse con un dato posterior, que es el que da la conclusión de esta sección. El estudio de formatos y la primera prueba de uso destaparon que la comprobación de ámbito bloqueaba mensajes legítimos cortos e imperativos sobre el plan (“que no le pongas carne roja dos días seguidos”), mientras dejaba pasar las frases que describían al cliente. Probé tres arreglos de prompt y medí la matriz modelo por prompt con 18 casos y 3 repeticiones (tabla 7.14). El arreglo era el modelo, no el prompt: el de 3 mil millones acierta 54 de 54 incluso con el prompt original, y el pequeño no llega ni con el prompt reescrito. Añadirle al pequeño el campo de análisis previo, el mismo recurso que había hecho fiables a los otros dos, lo hundió: pasó a contestar fuera de ámbito en todo. Y el mismo mensaje con y sin tildes recibía veredictos distintos, la señal más clara de que se estaba persiguiendo ruido. El modelo de 3 mil millones pasó a juzgar también el ámbito, y el pequeño dejó de usarse en el proyecto. El prompt reescrito se mantuvo por ser mejor especificación, sin atribuirle el mérito.

La conclusión matizada es que “el modelo mínimo que hace bien cada trabajo” exige bancos que cubran las clases de mensaje reales, porque el mínimo puede ser mayor que el medido: el banco de 6 casos con que se eligió el pequeño no tenía

Modelo de la comprobación de ámbito	Prompt original	Prompt reescrito
qwen2.5 de 1,5 mil millones	39 de 54 (15 falsos positivos)	43 de 54 (10 falsos positivos, 1 fuga)
qwen2.5 de 3 mil millones	54 de 54	54 de 54

Tabla 7.14: Matriz modelo por prompt de la comprobación de ámbito (18 casos, 3 repeticiones).

imperativos cortos. Las cifras de coste cierran la sección: la traducción tarda de 16 a 20 segundos por llamada en el equipo local con unos 1 550 tokens de entrada y 65 de salida, y las tres pasadas de validación previa unos 25 segundos por mensaje; todo ello sostenido por la generación asíncrona de la sección 6.5.

7.5. Prueba con nutricionistas

La última evaluación fue una prueba guiada con dos nutricionistas en ejercicio sobre la aplicación desplegada, entre el 25 de agosto y el 4 de septiembre de 2026. Les envié una guía de cuatro páginas con ocho tareas planteadas por objetivo y sin decir cómo llegar a él, para ver si la aplicación los llevaba sola: seis como nutricionista (hacerse una idea del estado de la consulta y de cómo le va a una clienta concreta, dar de alta un cliente, registrar sus restricciones, pedir al asistente una dieta semanal con sus palabras y revisarla con ojo profesional, comprobar y ampliar el catálogo de alimentos, y atender mensajes y peticiones de cita) y dos como cliente (ver qué toca comer hoy, marcar las comidas hechas y anotar el peso; escribir al nutricionista y pedirle cita). Cada tarea se valoraba en tres niveles (fácil, no obvio, no conseguido) con comentarios libres, y la guía terminaba con una opinión general. También avisaba de que la cuenta era compartida, de que la interfaz está en inglés, de que el asistente dependía del worker de mi ordenador y de que generar una dieta tarda uno o dos minutos.

Los dos completaron las ocho tareas y las dieciséis valoraciones fueron “fácil”; ninguno dejó comentarios por tarea. En la opinión general coincidieron: uno destacó “lo simplificado e intuitivo que es todo” y el otro la describió como “sumamente intuitiva”. El único pero fue el tiempo de espera al confeccionar la dieta. Uno de ellos valoró expresamente que el propio paciente marque las comidas realizadas, porque facilita seguir la adherencia. Ninguno propuso cambios, y los dos dijeron que la usarían: uno “sin duda”, porque “me quita tiempo de trabajo a la hora de confeccionar los planes”; el otro, “realmente la usaría si se pudiera”.

Con dos participantes y sin comentarios por tarea, la prueba no mide la usabilidad con detalle. Lo que sí dice es que los flujos principales de los dos paneles se recorren sin instrucciones, y confirma dos cosas del proyecto: que el valor percibido está en el tiempo que ahorra al preparar el plan, la motivación del pivote del capítulo 3, y que la latencia de la generación es lo que más se nota, en línea con las medidas de la sección 7.3 y con la decisión de generar en segundo plano. De ahí salen dos pautas para el capítulo 8: que el worker de generación esté siempre disponible

en vez de depender de mi ordenador, y repetir la prueba con más profesionales y sobre planes reales.

Conclusiones y Trabajo Futuro

Este capítulo cierra la memoria. Primero repasa los objetivos de la introducción uno a uno, con la evidencia que sostiene cada uno y lo que quedó a medias; después recoge las limitaciones del sistema tal como se entrega; y por último agrupa el trabajo futuro por capas, con la razón por la que cada línea no entró en el TFG.

8.1. Conclusiones

El objetivo general era diseñar, construir y evaluar una plataforma web para nutricionistas cuyo núcleo fuese un copiloto de generación de planes. La plataforma existe y está desplegada (sección 4.5): el profesional escribe lo que necesita en castellano, el traductor lo convierte en restricciones formales, el motor construye un borrador que las cumple, el validador lo audita y el plan solo llega al cliente cuando el profesional lo firma (capítulos 4 a 6). El proceso completo de producto también se recorrió: la idea se refinó con nutricionistas antes de escribir código, el alcance se reformuló con lo que pedían, y el resultado se midió con criterios fijados antes de medir (capítulo 7). La pieza que falta de ese recorrido es la prueba con nutricionistas sobre la aplicación desplegada, que está en curso en el momento de redactar esta versión (sección 7.5).

Respecto a los objetivos específicos:

1. *Analizar el flujo de trabajo real de un nutricionista para extraer los requisitos.* Los requisitos salieron de las conversaciones con el nutricionista que motivó el proyecto y con los profesionales que contacté después (sección 3.1), y el catálogo inicial de 61 requisitos se redujo a 27 cuando esas mismas conversaciones enseñaron dónde estaba el cuello de botella. Ese recorte es la parte del objetivo que más valor tuvo: el copiloto no estaba en la idea original y salió de escuchar la demanda real. Dos requisitos conservados en el capítulo 3 no se llegaron a construir (las notas privadas del cliente, RF-03, y el *dashboard* de métricas del copiloto, RF-39); se recogen en la sección 8.2.
2. *Realizar un estudio de mercado y análisis de la competencia.* El capítulo 2 analiza cuatro plataformas del sector con sus webs consultadas en septiembre de

2026 y las compara con lo que la aplicación entrega (tabla 2.1); la comparativa de la primera versión, más amplia y anterior a la reformulación, se conserva en el Anexo A. El posicionamiento que resulta es concreto: las plataformas que generan planes lo hacen a partir de números y ninguna explica por qué un plan no puede existir (sección 2.5). El análisis se quedó a propósito en el posicionamiento funcional; un estudio económico del producto queda fuera de este trabajo.

3. *Diseñar la arquitectura del sistema y su modelo de datos.* Los dos paneles comparten una misma base de datos y el aislamiento entre profesionales y entre cliente y profesional está en la propia capa de datos, mediante políticas de seguridad a nivel de fila que los runners multi-actor comprueban tabla a tabla (secciones 4.2, 4.3 y 7.1). La tabla de restricciones funciona como interfaz entre tres productores (el formulario del cliente, el traductor y el estilo clínico del profesional) y un único consumidor, el motor (figura 4.2); esa separación es la que permitió construir y verificar el motor antes de que existiera la capa de IA. Este objetivo se cumplió tal como estaba planteado.
4. *Construir un motor de generación basado en programación con restricciones, con un validador independiente y sin depender de la IA.* El motor del capítulo 5 genera, valida, persiste y firma planes sin ningún modelo de lenguaje, y así se verificó primero con los bancos deterministas y después de extremo a extremo contra el despliegue (sección 7.1). La calidad de los planes se midió y se corrigió por capas: la auditoría de sinsentidos bajó las razones en los límites del 66 % al 6 % (sección 7.2.1), y la función objetivo se reequilibró dos veces con datos (secciones 7.2.3 y 7.2.4). Quedaron límites conocidos: las reglas estructurales se asumen válidas para la población general y el profesional no puede anularlas (sección 5.3.1), el término de reparto por comidas sigue dominando la función objetivo cuando está activo (sección 5.5) y con el catálogo de cien alimentos el solver no siempre demuestra el óptimo dentro del límite de tiempo (sección 7.3.2).
5. *Integrar la capa de IA que hace usable el motor en lenguaje natural.* El traductor, la revisión previa del mensaje, la generación asíncrona y la pantalla de arbitraje están contruidos e integrados (capítulo 6), y el profesional decide sobre cada restricción propuesta y sobre cada plan antes de que llegue al cliente. El modelo se ejecuta en local para que el dato clínico no salga a ningún proveedor, y la evaluación enseñó lo que eso cuesta: un modelo de siete mil millones de parámetros comete fallos residuales que el diseño contiene pero no elimina (sección 7.4.2), y el estudio de formatos descartó con datos construir un módulo que reescribiera el mensaje (sección 7.4.1). La parte que quedó a medias es el estilo clínico: se declara como reglas, y aprenderlo de las correcciones del profesional no se abordó.

La prueba con nutricionistas (sección 7.5) cerró el ciclo con dos profesionales: las ocho tareas les resultaron fáciles, destacaron lo intuitivo de la aplicación y el tiempo que ahorra, y su único pero fue la espera al generar la dieta. Sus dos pau-

tas, un worker siempre disponible y una prueba con más profesionales, están en la sección 8.3.

Hay tres cosas que aprendí en el proceso y que condicionaron el diseño más que cualquier tecnología concreta. Cerrar el vocabulario de restricciones antes de tocar el modelo de lenguaje hizo que la IA fuese una capa de conveniencia sobre un motor que ya funcionaba, y no al revés. Verificar cada capa por separado, con bancos que se re-ejecutan al cerrar cada bloque, es lo que permitió cambiar el motor varias veces en verano sin romper lo anterior. Y fijar el criterio antes de medir dio el mismo valor a los resultados negativos que a los positivos: el reescritor y la rotura de simetría no se construyeron porque los datos no los respaldaban, y las bandas de tolerancia se mantuvieron con la desviación del criterio documentada (sección 7.2.4).

8.2. Limitaciones

Las limitaciones que siguen son las del sistema tal como se entrega. Casi todas están ya dichas en el capítulo donde se tomó la decisión; aquí se juntan para que el lector las tenga en un solo sitio.

En el motor, las reglas estructurales se asumen válidas para la población general y el profesional no puede anularlas caso a caso, así que una prescripción fuera de esos rangos, como la dieta renal que puso de ejemplo uno de mis directores, se declararía infactible (sección 5.3.1). El validador solo rechaza lo que el solver garantiza y avisa del resto, con lo que un plan con un micronutriente por encima del tope llega al profesional como aviso y no como rechazo (sección 5.7). El término de reparto por comidas domina la función objetivo cuando está activo, el cociente entre nutrientes solo existe como restricción dura (sección 5.5) y una restricción dura de energía o de macronutriente exige la igualdad exacta por día, sin la tolerancia que sí tienen las blandas. Con el catálogo de cien alimentos el solver no siempre demuestra el óptimo dentro del límite de tiempo, y ese límite es de 90 segundos con dos hilos porque el alojamiento gratuito no da para más (secciones 7.3.2 y 7.3.5).

El catálogo es propio: cien alimentos con sus nutrientes y perfiles de ración cargados a mano, sin la base de referencia BEDCA que preveía el capítulo 3. Los alimentos que añade el profesional no tienen perfil de ración y el motor les aplica un rango genérico (sección 5.3.2); y los umbrales de los casos de infactibilidad de los bancos se han tenido que recalibrar cada vez que el catálogo creció, porque con más fuentes de un nutriente cuesta más demostrar que algo no puede existir.

En el copiloto, el modelo de lenguaje corre en local y es pequeño; la evaluación enseñó fallos residuales que el diseño contiene pero no elimina, como confundir una categoría con un alimento concreto (sección 7.4.2). El worker que ejecuta ese modelo es, durante el desarrollo y la evaluación, mi propio ordenador: si está apagado, la aplicación funciona entera y las tareas de IA esperan en la cola (sección 4.5).

En la plataforma, la interfaz está en inglés aunque la petición al copiloto se escriba en castellano, porque los mockups se diseñaron así desde el principio. Dos requisitos que el capítulo 3 conserva no se construyeron: las notas privadas del profesional sobre cada cliente (RF-03) y el *dashboard* de métricas del copiloto (RF-39), que en su lugar enseña datos reales de clientes, planes y alimentos. Algunas panta-

llas tienen recortes frente al diseño, la mensajería se refresca por sondeo cada quince segundos, las altas que escriben en varias tablas desde el navegador no son transaccionales y se compensan con un borrado lógico si falla un paso (sección 4.2), los datos de demostración llevan fechas relativas que caducan (sección 4.6) y el repositorio no tiene protección de rama.

En la evaluación, la prueba con nutricionistas se hizo con dos profesionales y sin comentarios por tarea, así que valida los flujos pero no mide la usabilidad con detalle (sección 7.5); los bancos del motor usan casos contruïdos a mano, no planes reales de profesionales; y la comparativa con los competidores se apoya en lo que sus webs decían en septiembre de 2026, que cambiará.

8.3. Trabajo futuro

Las líneas se agrupan por la capa a la que pertenecen. Cada una lleva la razón por la que no entró en el TFG: tiempo, una decisión de alcance tomada a propósito, o una dependencia de otra pieza que tampoco existe.

8.3.1. Motor de generación

- **Variedad por familias de alimentos.** El término de alimentos sin usar es lineal: pasar de cinco a seis alimentos distintos vale lo mismo que pasar de cincuenta a cincuenta y uno, y un plan con ochenta alimentos a la semana no es mejor ni más fácil de seguir. La versión que tiene sentido clínico premia que cada familia esté presente con un peso propio, sin fijar un número de ingredientes. Requiere una taxonomía de familias en el catálogo que hoy solo existe en parte; mientras, la variedad la controla el estilo clínico del profesional (sección 7.2.4).
- **Empates entre soluciones.** Cuando varios planes empatan en la función objetivo, el solver devuelve el primero que encuentra y nada distingue a los demás. Uno de los directores del TFG señaló dos salidas: un orden de prioridad entre las restricciones, resolviendo por niveles para que el empate lo deshaga la más importante (un objetivo lexicográfico), o devolver una lista corta de planes alternativos con el mismo valor para que el profesional elija, que encaja con el marco del proyecto: el sistema propone y el profesional decide. Las dos se apoyan en el modelo actual; la primera cambia la estrategia de resolución y la segunda la interfaz. La variedad por familias del punto anterior sería un criterio natural para deshacer esos empates.
- **Empates entre soluciones.** Cuando varios planes empatan en la función objetivo, el solver devuelve el primero que encuentra y nada distingue a los demás. Uno de los directores del TFG señaló dos salidas: un orden de prioridad entre las restricciones, resolviendo por niveles para que el empate lo deshaga la más importante (un objetivo lexicográfico), o devolver una lista corta de planes alternativos con el mismo valor para que el profesional elija, que encaja con el marco del proyecto: el sistema propone y el profesional decide. Las dos

se apoyan en el modelo actual; la primera cambia la estrategia de resolución y la segunda la interfaz. La variedad por familias del punto anterior sería un criterio natural para deshacer esos empates.

- **Reparto calórico por comidas.** Renormalizar el término para quitarle el factor de mil que arrastra su linealización, con división entera y una banda porcentual como la de la energía. No se midió porque solo dos casos de los bancos lo activan y uno de ellos es un conflicto deliberado (sección 7.2.3).
- **Tolerancias configurables.** Que cada fila de objetivo calórico o de macro-nutriente lleve su propia banda en vez de la global, y que las filas duras tengan una tolerancia estrecha en lugar de la igualdad exacta por día: un “debe” clínico significa alrededor de un valor, no ese valor al gramo. Toca la validación, el formulario y el traductor a la vez; decidí no tocar el motor en el cierre del proyecto.
- **Reglas estructurales editables.** Que el profesional pueda ver las ocho reglas de sentido común, retirarlas o endurecerlas para un cliente concreto, y así cubrir casos como el renal sin declararlos infactibles. No hubo tiempo de hacerlas configurables; se documentaron como asunción (sección 5.3.1).
- **Acompañantes.** Un rol de acompañante (tomate, cebolla, lechuga) que no consume el tope de cuatro alimentos por comida, como señaló uno de los directores del TFG al revisar el tope; exige marcar ese rol en el catálogo.
- **Cociente entre nutrientes blando.** Hoy solo existe como restricción dura porque penalizar la desviación de un cociente obliga a linealizarlo (sección 5.4).
- **Recetas y sustituciones (RF-26 y RF-27).** El motor trabaja con alimentos sueltos por diseño. Las recetas exigirían que el motor eligiera entre preparaciones con su composición agregada y una interfaz para definirlas; las sustituciones, un modelo de equivalencia que ninguna pieza tiene hoy.
- **Validación por patología y medicación.** El validador aplica reglas de población general. Afinarlo por patología o por fármaco depende de ampliar la ficha del cliente con esas variables y el catálogo con los nutrientes implicados, ampliación que el capítulo 3 pospone a propósito.
- **Residuos de la capa de sentido común.** Un desayuno de solo frutos secos sigue siendo posible en teoría, y los tentempiés de una pieza y los alimentos sin perfil dependen de rangos genéricos. La prueba con nutricionistas no señaló ninguno de estos casos; la calibración fina queda pendiente de una prueba con más profesionales.

8.3.2. Copiloto de IA

- **Aprender el estilo del profesional.** El estilo clínico se declara como reglas (sección 6.4). La versión ambiciosa lo aprende de las correcciones que el profesional hace sobre cada borrador y lo arranca con un cuestionario. Se construyó

la versión mínima acordada con los tutores; el aprendizaje necesita además la edición del plan, que tampoco existe.

- **Edición del plan generado.** Hoy el borrador se acepta o se regenera con otras restricciones; falta poder cambiar un alimento o una cantidad y que el validador vuelva a auditar el resultado. Se dejó fuera porque el flujo de generación sustituyó a la edición manual de planes en el alcance.
- **Métricas del copiloto (RF-39).** La tasa de aprobación al primer intento y la distancia entre el borrador y la versión entregada, tal como las define el capítulo 3, solo se pueden medir cuando exista esa edición; los tokens y la latencia de cada llamada ya quedan en la traza (sección 6.3).
- **Motor de lenguaje remoto.** El traductor habla con el modelo a través de una interfaz mínima, así que un motor remoto con garantías contractuales de no retención sería otra implementación de esa interfaz. Permitiría comparar coste, latencia y calidad frente al modelo local y repetir el estudio de formatos con un modelo mayor; no entró por la decisión de privacidad de la sección 6.3 y porque el modelo local bastó para el alcance.
- **Reescritor del mensaje.** No es trabajo futuro: el estudio de formatos descartó construirlo con datos (sección 7.4.1), y solo se reabría si un modelo distinto cambiara ese resultado.

8.3.3. Plataforma

- **Catálogo de referencia.** Cargar BEDCA o USDA con sus nutrientes, manteniendo los perfiles de ración por alimento que la capa de sentido común necesita. El catálogo propio de cien alimentos bastó para construir y evaluar el motor.
- **Funciones de gestión recortadas por la versión mínima.** Editar y borrar alimentos propios, las notas privadas del cliente (RF-03), el calendario en rejilla semanal, las notificaciones y la invitación de clientes desde el panel del profesional. Son pantallas con su tabla detrás ya existente o trivial, dejadas fuera por tiempo.
- **Seguimiento.** Volcar el peso que registra el cliente a la ficha con validación del profesional, y caducar las solicitudes de cita pendientes con un proceso periódico. Quedaron fuera del alcance del panel del cliente.
- **Mensajería en tiempo real.** El sondeo cada quince segundos fue una decisión deliberada para el volumen de un profesional; las suscripciones en tiempo real de la propia base de datos quitarían la latencia a cambio de gestionar conexiones persistentes.
- **Interfaz en castellano.** Traducir la interfaz, en inglés desde los mockups, es la mejora que más notarían los profesionales a los que va dirigida.

- **Despliegue.** Mover el worker a un servidor de la clínica o a la nube, y pasar a un alojamiento de pago que permita bajar el límite de tiempo del solver y devolverle los hilos (sección 7.3.5).
- **Robustez de datos.** Pasar las altas de varias tablas desde el navegador a funciones de la base de datos, para que sean transaccionales; y datos de demostración con fechas que se vuelvan a anclar solas.

8.3.4. Evaluación

La prueba con nutricionistas (sección 7.5) deja dos pautas: que el worker de generación esté siempre disponible, porque la espera fue su único pero, y repetirla con más profesionales. Después, la evaluación que tiene más valor es la de uso real: las métricas del copiloto que define RF-39, medidas sobre planes de profesionales en vez de sobre casos contruidos a mano, y la comparativa con un motor de lenguaje remoto. Y una lección de método que conviene arrastrar a cualquier experimento futuro sobre el motor: los criterios sobre conteos deben llevar una tolerancia del tamaño de la brecha de parada del solver, o exigir óptimos a brecha cero (sección 7.2.4).

Sobre los requisitos de la primera versión. Los cinco bloques que el capítulo 3 elimina (formularios, alertas, lista de la compra, marketplace y automatizaciones) y la mayoría de los requisitos individuales de la tabla 3.12 no aparecen en las listas anteriores a propósito. Se descartaron porque los nutricionistas no los pedían y porque no contribuyen al copiloto, y el Anexo A los conserva como registro de la idea inicial. Solo recojo como trabajo futuro los que tienen un enganche en lo construido: las recetas y sustituciones, las notas privadas y el *dashboard* de métricas.

8.4. Cierre

Empecé este trabajo con la condición de que sirviera para algo real, y lo que más me ha enseñado es lo lejos que estaba mi primera idea de lo que los nutricionistas necesitaban. Sin las conversaciones con ellos habría construido una plataforma de gestión más, y el copiloto, la parte que da sentido al proyecto, no habría existido. La otra lección es la indicación de mis directores del trabajo de fin de grado: la versión mínima que aplique las funcionalidades, verificada, vale más que una ambiciosa a medias. Aplicarla me costó al principio, porque suponía dejar fuera cosas que ya tenía diseñadas, pero es lo que permitió llegar a una aplicación desplegada y con un motor medido en vez de a un prototipo bonito. Lo que digan los profesionales cuando la usen es lo que queda por escuchar.

Introduction

This chapter presents the starting point of the project: the motivation behind it, the objectives it pursues and the work plan that has guided its development.

Motivation

When I planned this Bachelor's Thesis there was one clear premise: the result had to have a real application. Solving an invented problem to meet an academic requirement did not motivate me, so I looked for a way to use the knowledge acquired during the degree and aim it at a concrete need of someone close to me. The idea was that the work would serve as a technical exercise and, at the same time, as a solution to a problem that someone experienced in their daily professional life.

That someone turned out to be a friend who works as a nutritionist. In a conversation about his daily routine I found a problem that he himself did not perceive as such, because he had normalised it: his workflow was fragmented across five or more tools that did not talk to each other. He used Google Forms to collect information from his clients, Google Calendar to manage appointments, WhatsApp as a direct communication channel, a physical notebook for his private notes on each case, and his clinic's management platform for everything else. To get a complete picture of a single client he had to consult all these sources by hand and rebuild the information on his own. On top of that, a large part of his time went into answering repetitive questions: whether an ingredient could be replaced, when the next appointment should be, and countless small doubts about how to fix the plan after skipping or swapping meals; almost trivial questions that could be resolved without his direct involvement.

From that conversation came the central idea of this project: to design a platform that would centralise all those scattered tools in a single place. If all client information (forms, appointments, messages, notes, nutritional plans, daily records) lived in the same database, the split between sources would disappear and new possibilities would open up, such as detecting patterns automatically, raising alerts when a client shows signs of dropping out, or building a complete profile without cross-referencing data by hand. In short, the nutritionist could spend his time on what really matters: designing diets, accompanying his clients and making the treatments work, instead of investing it in administrative tasks that a well-designed application can solve.

That initial vision, a platform that would centralise the whole workflow, guided the first version of the project: everything such a tool could offer was catalogued and a complete prototype was designed with it, from customisable forms to a marketplace of educational resources. When the proposal was presented to more practising nutritionists, their concerns helped to rethink the approach. What they really demanded was not to have everything in one place, which obviously would also be a great help, but they also ran into a bottleneck with the task that takes them the most time as nutritionists: building each diet to measure. The client's record (their data, measurements and restrictions) mattered to them as support for that task; communication with the client they saw as a complement, not as the core; and centralising the rest of their processes depends, in practice, on the clinic they work in, not on the tool they choose.

That contrast with the real demand led me to reformulate the project, as detailed in Chapter 3. The centre stopped being the all-in-one platform and became a copilot for generating tailored nutritional plans: the nutritionist describes what they need in their own language, the system builds a draft that meets verifiable constraints, and the professional reviews it, adjusts it and signs it before it reaches the client. The platform still exists around it (clients, appointments, messages, follow-up) but as support for that core, not as an end in itself.

Objectives

This Bachelor's Thesis is developed within the Double Degree in Computer Science Engineering and Business Administration, which allows it to be approached from two complementary perspectives: the technical one, ranging from the design of the database to the integration of an optimisation engine and language models; and the business one, applying the market analysis and product viability tools of the Business Administration training. The aim is not simply to meet an academic requirement, but to take advantage of this double background to build a fully functional project with a real application.

The general objective of this work is to design, build and evaluate a web platform for nutritionists whose core is a copilot for generating nutritional plans: the professional expresses a client's needs in their own language, artificial intelligence translates them into formal constraints, an optimisation engine builds a draft that satisfies them and an independent validator audits it, with the final decision always remaining in the hands of the professional. The intention is to go through the complete process of a real product, refining the idea with professionals and aiming at public deployment and empirical evaluation, so that the result is technically sound and also defensible as a real tool.

To achieve this general objective, the following specific objectives are set:

1. To analyse the real workflow of a professional nutritionist in order to extract the functional requirements of the system. This means understanding how they work day to day, which tools they currently use, which problems they face and what they really need. In this way the requirements are not invented

but extracted from direct observation of a real case, so that the design responds to concrete needs rather than assumptions.

2. To carry out a market study and competitor analysis in the field of nutrition platforms, applying knowledge from the Business Administration training, to identify the main competitors, their strengths and shortcomings, and to position NutriApp with a differentiated proposal.
3. To design the system architecture and its data model: two panels (nutritionist and client) over a single database, with the isolation between professionals guaranteed in the data layer itself, and with a constraints table that acts as the interface between whoever produces them (the professional or the AI) and the engine that consumes them.
4. To build a plan generation engine based on constraint programming, together with a deterministic and independent clinical validator, capable of working on its own without depending on any artificial intelligence component.
5. To integrate the AI layer that makes that engine usable in natural language: translation of the professional's requests into formal constraints, prior validation of the messages and asynchronous generation, keeping the professional as the arbiter of every decision.

Work plan

The development of this project has followed an iterative process in which each phase builds on the results of the previous one. The phases are described below, from the initial research with professionals to the evaluation of the final system.

Phase 1. Research with real nutritionists (May 2025 to February 2026)

The starting point was to understand in depth the problem to be solved. I held a series of conversations with the nutritionist whose case motivated the project to learn his working method thoroughly: which tools he used, where he lost the most time and which information slipped away because it was spread across disconnected platforms.

After consulting the supervisors, they recommended that I contact more nutritionists so as not to base the product on the experience of a single person. So I contacted other professionals in the sector through LinkedIn and asked them the same questions. This round of consultations allowed me to contrast ideas and build a broader and more solid vision of the product.

Phase 2. Competitor and market analysis (February to March 2026)

Once the needs had been defined from the user's side, it was time to research the market. Obviously, very similar solutions already existed, so it was worth knowing who the competitors were, what they offered and where they left gaps. I analysed the main platforms in the sector (Nutrium, Indya, Dietopro, Nutrimeal and Practice Better), studying their features, business models, strengths and shortcomings, to detect which aspects had stayed off the radar in the previous phase and whether the needs I had collected were already covered or there was a real opportunity. By the end of this phase the product idea was much better defined, with a clearer value proposition. That first analysis, made for the initial scope, is kept in Appendix A; the definitive comparison, already focused on the copilot, is the one in Chapter 2.

Phase 3. Visual design of the prototype (March 2026)

With the requirements extracted, I turned them into a concrete interface: the application mockups. The purpose of the screens was twofold: that the nutritionist could manage all their work from a single point of access (clients, appointments, forms, plans, messages and notes) and that the client had tools to consult their plan, record their habits and follow their progress. Each screen covered use cases identified in the previous phases.

The set of screens followed the same design system to keep visual consistency. What had already been designed was reviewed to incorporate improvements detected during the requirements analysis, such as the daily, weekly and monthly views of the calendar or the client's settings screen. The result was a first navigable prototype with the two panels interconnected and all the initial functional requirements present.

Phase 4. Second iteration: competitors in depth and new features (April 2026)

After completing the first prototype, I analysed the competition a second time, this time contrasting NutriApp with seven reference platforms. Four additions to the design came out of that analysis: a configurable automations engine, a marketplace of educational resources, a food database with intelligent recognition, and video calls integrated in the chat; plus several refinements, such as the detailed client record. The result was a complete navigable prototype of twenty-four screens, with the sixty-one functional requirements represented and the navigation wired in Figma, so that an external evaluator could walk through it as if it were a real application.

Phase 5. Validation with real users (April 2026)

With the prototype complete, I prepared two usability test guides (nutritionist and client), each with twelve navigation flows phrased as small challenges ("you want to check the average adherence of your clients, which section of the side menu would

you go to?”). The evaluator had to simulate the navigation flow in each case and mark whether it was obvious, confusing, or whether they simply would not know how to get there.

Phase 6. Reformulation of the scope (April 2026)

Unexpectedly, the complete “final” prototype (for the plan that existed at that time) turned out to be the best tool for testing the implementation idea. When it was presented to more nutritionists and discussed further with them to receive feedback, it became clear that the value was not in covering every possible feature but in automating the preparation of tailored diets, the task that takes them the most time.

In addition, when the supervisors tried the prototype, they made me see the complexity and the number of features I wanted to program on my own. This made me see that I had to focus on covering the primary need of nutritionists and concentrate my efforts in that direction. So I decided to cut the scope drastically down to what supports that task and to concentrate the technical effort on the plan generation copilot, which is where the real value lay. Chapter 3 documents the complete triage and its criteria.

Phase 7. Design and implementation of the database (May 2026)

Following the order that my thesis supervisors recommended to me (first the database, then its connection and finally the screens), I designed and implemented the complete data model on managed PostgreSQL: the catalogue, people, constraints and plan tables; the row-level security policies that guarantee isolation between professionals; and the seed data needed to work with realistic cases from day one.

Phase 8. Construction of the web application (May and June 2026)

On that base I built the application in two steps. First, the connection of the two pieces that talk to the database: the backend in Python and the frontend in Next.js, each verified with real reads and writes before drawing a single screen. Then the nutritionist’s panel section by section (clients, plans, foods, calendar, messaging and settings), always on real data and without simulating functionality that did not exist: if a screen showed something, there was a table behind it to support it.

The application was deployed publicly as soon as it had its first sections, following the supervisors’ recommendation, which made it possible to show it and test it in real conditions as early as possible. At the end of this phase I compared the Figma prototype with what had been implemented and reviewed the differences, so as not to forget elements and to check again whether some of them were unnecessary, justifying the divergence.

Phase 9. Plan generation engine (June 2026)

The technical heart of the project: the formalisation of the diet as a constraint satisfaction problem. After consulting one of my supervisors about which approach would be best, I was advised that its implementation should consist of a constraint programming solver and an independent deterministic validator that audits every plan before accepting it, and that I could also add an infeasibility diagnosis for when the requests are incompatible with each other. The engine works without any artificial intelligence component, which made it possible to verify it first with a bank of cases and then from the web itself (by creating a section to modify the parameters used to generate the plan with the engine).

Phase 10. Artificial intelligence layer (July 2026)

With the engine verified, the layer that makes it usable in natural language was added: the translator that converts the nutritionist's requests into formal constraints, the prior checks that examine each message before translating it, asynchronous generation, and the clinical style that each professional defines once and that applies to all their plans. All of this was implemented through queries to an open language model, qwen2.5, run locally with Ollama.

Phase 11. Client panel (July 2026)

I then developed the second role of the application: the client panel. The client accesses their plan once it has been signed, marks the meals they have completed, records their weight and requests appointments; all that information flows back to the nutritionist's panel as follow-up metrics. With this phase the complete cycle between professional and client is closed, and all the information that had to be shared between both roles was already implemented.

Phase 12. Evaluation (July to September 2026)

The evaluation accompanied the whole development. It includes a pre-registered study on the effect of message format on translation (with a negative result that avoided building an unnecessary module), performance measurements of the engine in the production environment as the food catalogue grew, and a systematic audit of the nutritional quality of the generated plans, whose before and after justified a whole new layer of rules in the engine. And it closed with a guided test with two nutritionists on the deployed application, which completed the cycle that began in Phase 1: from the professional to the prototype, from the prototype to the product, and from the product back to the professional.

Conclusions and Future Work

This chapter closes the thesis. It first goes back over the objectives of the introduction one by one, with the evidence that supports each of them and what was left half done; it then collects the limitations of the system as it is delivered; and finally it groups the future work by layers, with the reason why each line did not make it into the thesis.

Conclusions

The general objective was to design, build and evaluate a web platform for nutritionists whose core would be a plan generation copilot. The platform exists and is deployed (Section 4.5): the professional writes what they need in Spanish, the translator converts it into formal constraints, the engine builds a draft that satisfies them, the validator audits it and the plan only reaches the client when the professional signs it (Chapters 4 to 6). The complete product process was also covered: the idea was refined with nutritionists before writing any code, the scope was reformulated according to what they asked for, and the result was measured with criteria fixed before measuring (Chapter 7). The missing piece of that journey is the test with nutritionists on the deployed application, which is under way at the time of writing this version (Section 7.5).

Regarding the specific objectives:

1. *To analyse the real workflow of a nutritionist in order to extract the requirements.* The requirements came out of the conversations with the nutritionist who motivated the project and with the professionals I contacted afterwards (Section 3.1), and the initial catalogue of 61 requirements was reduced to 27 when those same conversations showed where the bottleneck was. That cut is the part of the objective that had the most value: the copilot was not in the original idea and came from listening to the real demand. Two requirements kept in Chapter 3 were never built (the private notes on the client, RF-03, and the copilot metrics *dashboard*, RF-39); they are collected in Section 8.2.
2. *To carry out a market study and competitor analysis.* Chapter 2 analyses four platforms in the sector, with their websites consulted in September 2026, and compares them with what the application delivers (Table 2.1); the comparison

of the first version, broader and prior to the reformulation, is kept in Appendix A. The resulting positioning is concrete: the platforms that generate plans do so from numbers and none of them explains why a plan cannot exist (Section 2.5). The analysis deliberately stopped at functional positioning; an economic study of the product falls outside this work.

3. *To design the system architecture and its data model.* The two panels share a single database, and the isolation between professionals and between client and professional lives in the data layer itself, through row-level security policies that the multi-actor runners check table by table (Sections 4.2, 4.3 and 7.1). The constraints table works as the interface between three producers (the client's form, the translator and the professional's clinical style) and a single consumer, the engine (Figure 4.2); that separation is what made it possible to build and verify the engine before the AI layer existed. This objective was met as planned.
4. *To build a generation engine based on constraint programming, with an independent validator and without depending on AI.* The engine of Chapter 5 generates, validates, persists and signs plans without any language model, and it was verified that way, first with the deterministic banks and then end to end against the deployment (Section 7.1). The quality of the plans was measured and corrected in layers: the audit of nonsensical plans brought the portions at the limits down from 66 % to 6 % (Section 7.2.1), and the objective function was rebalanced twice with data (Sections 7.2.3 and 7.2.4). Known limits remain: the structural rules are assumed valid for the general population and the professional cannot override them (Section 5.3.1), the meal distribution term still dominates the objective function when it is active (Section 5.5), and with the catalogue of one hundred foods the solver does not always prove the optimum within the time limit (Section 7.3.2).
5. *To integrate the AI layer that makes the engine usable in natural language.* The translator, the prior review of the message, asynchronous generation and the arbitration screen are built and integrated (Chapter 6), and the professional decides on every proposed constraint and on every plan before it reaches the client. The model runs locally so that clinical data never leaves for any provider, and the evaluation showed what that costs: a model of seven billion parameters makes residual mistakes that the design contains but does not eliminate (Section 7.4.2), and the format study ruled out with data building a module to rewrite the message (Section 7.4.1). The part left half done is the clinical style: it is declared as rules, and learning it from the professional's corrections was not addressed.

The test with nutritionists (Section 7.5) closed the cycle with two professionals: they found all eight tasks easy, highlighted how intuitive the application is and the time it saves, and their only complaint was the wait while generating the diet. Their two guidelines, an always-available worker and a test with more professionals, are in Section 8.3.

There are three things I learnt in the process that shaped the design more than any particular technology. Closing the constraint vocabulary before touching the language model made the AI a convenience layer over an engine that already worked, and not the other way round. Verifying each layer separately, with banks that are re-run when each block is closed, is what made it possible to change the engine several times over the summer without breaking what came before. And fixing the criterion before measuring gave negative results the same value as positive ones: the rewriter and the symmetry breaking were not built because the data did not support them, and the tolerance bands were kept with the deviation from the criterion documented (Section 7.2.4).

Limitations

The limitations that follow are those of the system as it is delivered. Almost all of them are already stated in the chapter where the decision was taken; here they are gathered so that the reader has them in one place.

In the engine, the structural rules are assumed valid for the general population and the professional cannot override them case by case, so a prescription outside those ranges, such as the renal diet that one of my supervisors gave as an example, would be declared infeasible (Section 5.3.1). The validator only rejects what the solver guarantees and warns about the rest, so a plan with a micronutrient above its cap reaches the professional as a warning and not as a rejection (Section 5.7). The meal distribution term dominates the objective function when it is active, the ratio between nutrients only exists as a hard constraint (Section 5.5), and a hard energy or macronutrient constraint demands exact equality per day, without the tolerance that soft ones do have. With the catalogue of one hundred foods the solver does not always prove the optimum within the time limit, and that limit is 90 seconds with two threads because the free hosting does not allow for more (Sections 7.3.2 and 7.3.5).

The catalogue is our own: one hundred foods with their nutrients and portion profiles loaded by hand, without the BEDCA reference database that Chapter 3 envisaged. The foods that the professional adds have no portion profile and the engine applies a generic range to them (Section 5.3.2); and the thresholds of the infeasibility cases in the banks have had to be recalibrated every time the catalogue grew, because with more sources of a nutrient it is harder to prove that something cannot exist.

In the copilot, the language model runs locally and is small; the evaluation showed residual mistakes that the design contains but does not eliminate, such as confusing a category with a specific food (Section 7.4.2). The worker that runs that model is, during development and evaluation, my own computer: if it is switched off, the whole application keeps working and the AI tasks wait in the queue (Section 4.5).

In the platform, the interface is in English even though the request to the copilot is written in Spanish, because the mockups were designed that way from the start. Two requirements that Chapter 3 keeps were not built: the professional's private notes on each client (RF-03) and the copilot metrics *dashboard* (RF-39),

which instead shows real data on clients, plans and foods. Some screens have cuts with respect to the design, messaging refreshes by polling every fifteen seconds, the creations that write to several tables from the browser are not transactional and are compensated with a soft delete if a step fails (Section 4.2), the demo data carry relative dates that expire (Section 4.6) and the repository has no branch protection.

In the evaluation, the test with nutritionists was carried out with two professionals and without per-task comments, so it validates the flows but does not measure usability in detail (Section 7.5); the engine banks use hand-built cases, not real plans from professionals; and the comparison with competitors relies on what their websites said in September 2026, which will change.

Future work

The lines are grouped by the layer they belong to. Each one carries the reason why it did not make it into the thesis: time, a scope decision taken on purpose, or a dependency on another piece that does not exist either.

Generation engine

- **Variety by food families.** The unused foods term is linear: going from five to six different foods is worth the same as going from fifty to fifty-one, and a plan with eighty foods a week is neither better nor easier to follow. The version that makes clinical sense rewards each family being present, with its own weight, without fixing a number of ingredients. It requires a taxonomy of families in the catalogue that today only partly exists; meanwhile, variety is controlled by the professional's clinical style (Section 7.2.4).
- **Ties between solutions.** When several plans tie on the objective function, the solver returns the first one it finds and nothing distinguishes the rest. One of the supervisors of this thesis pointed out two ways forward: a priority order among the constraints, solving level by level so that the most important one breaks the tie (a lexicographic objective), or returning a short list of alternative plans with the same value for the professional to choose from, which fits the framework of the project: the system proposes and the professional decides. Both rest on the current model; the first changes the solving strategy and the second the interface. Variety by families from the previous point would be a natural criterion for breaking those ties.
- **Calorie distribution across meals.** Renormalise the term to remove the factor of one thousand it carries from its linearisation, with integer division and a percentage band like the one for energy. It was not measured because only two cases in the banks activate it and one of them is a deliberate conflict (Section 7.2.3).
- **Configurable tolerances.** That each calorie or macronutrient target row carries its own band instead of the global one, and that hard rows have a

narrow tolerance instead of exact equality per day: a clinical “must” means around a value, not that value to the gram. It touches validation, the form and the translator at the same time; I decided not to touch the engine at the close of the project.

- **Editable structural rules.** That the professional can see the eight common-sense rules, withdraw them or tighten them for a specific client, and so cover cases like the renal one without declaring them infeasible. There was no time to make them configurable; they were documented as an assumption (Section 5.3.1).
- **Side foods.** A side role (tomato, onion, lettuce) that does not consume the cap of four foods per meal, as one of the supervisors of this thesis pointed out when reviewing the cap; it requires marking that role in the catalogue.
- **Soft ratio between nutrients.** Today it only exists as a hard constraint because penalising the deviation of a ratio forces its linearisation (Section 5.4).
- **Recipes and substitutions (RF-26 and RF-27).** The engine works with individual foods by design. Recipes would require the engine to choose among preparations with their aggregated composition and an interface to define them; substitutions, an equivalence model that no piece has today.
- **Validation by pathology and medication.** The validator applies general population rules. Refining it by pathology or by drug depends on extending the client’s record with those variables and the catalogue with the nutrients involved, an extension that Chapter 3 postpones on purpose.
- **Leftovers of the common-sense layer.** A breakfast made only of nuts is still possible in theory, and single-piece snacks and foods without a profile depend on generic ranges. The test with nutritionists did not flag any of these cases; fine calibration awaits a test with more professionals.

AI copilot

- **Learning the professional’s style.** The clinical style is declared as rules (Section 6.4). The ambitious version learns it from the corrections the professional makes on each draft and bootstraps it with a questionnaire. The minimal version agreed with the supervisors was built; learning also needs plan editing, which does not exist either.
- **Editing the generated plan.** Today the draft is accepted or regenerated with other constraints; what is missing is being able to change a food or a quantity and have the validator audit the result again. It was left out because the generation flow replaced manual plan editing in the scope.
- **Copilot metrics (RF-39).** The first-attempt approval rate and the distance between the draft and the delivered version, as Chapter 3 defines them, can only be measured once that editing exists; the tokens and latency of each call are already recorded in the trace (Section 6.3).

- **Remote language engine.** The translator talks to the model through a minimal interface, so a remote engine with contractual guarantees of non-retention would be another implementation of that interface. It would allow comparing cost, latency and quality against the local model and repeating the format study with a larger model; it did not make it in because of the privacy decision of Section 6.3 and because the local model was enough for the scope.
- **Message rewriter.** This is not future work: the format study ruled out building it with data (Section 7.4.1), and it would only be reopened if a different model changed that result.

Platform

- **Reference catalogue.** Loading BEDCA or USDA with their nutrients, keeping the per-food portion profiles that the common-sense layer needs. The own catalogue of one hundred foods was enough to build and evaluate the engine.
- **Management functions cut by the minimal version.** Editing and deleting own foods, the private notes on the client (RF-03), the weekly grid calendar, notifications and inviting clients from the professional's panel. They are screens whose table already exists or is trivial, left out for lack of time.
- **Follow-up.** Carrying the weight recorded by the client over to the record with the professional's validation, and expiring pending appointment requests with a periodic process. They were left out of the scope of the client panel.
- **Real-time messaging.** Polling every fifteen seconds was a deliberate decision for the volume of one professional; the database's own real-time subscriptions would remove the latency in exchange for managing persistent connections.
- **Interface in Spanish.** Translating the interface, in English since the mock-ups, is the improvement that the professionals it is aimed at would notice the most.
- **Deployment.** Moving the worker to a clinic server or to the cloud, and switching to paid hosting that allows lowering the solver's time limit and giving it back its threads (Section 7.3.5).
- **Data robustness.** Moving the multi-table creations from the browser into database functions, so that they are transactional; and demo data with dates that re-anchor themselves.

Evaluation

The test with nutritionists (Section 7.5) leaves two guidelines: that the generation worker be always available, since the wait was their only complaint, and repeating it with more professionals. After that, the evaluation with the most value is that of real use: the copilot metrics defined by RF-39, measured on plans from

professionals instead of hand-built cases, and the comparison with a remote language engine. And a lesson of method worth carrying into any future experiment on the engine: criteria over counts must carry a tolerance the size of the solver's stopping gap, or demand optima at zero gap (Section 7.2.4).

On the requirements of the first version. The five blocks that Chapter 3 removes (forms, alerts, shopping list, marketplace and automations) and most of the individual requirements in Table 3.12 do not appear in the lists above on purpose. They were discarded because nutritionists did not ask for them and because they do not contribute to the copilot, and Appendix A keeps them as a record of the initial idea. I only collect as future work those with a hook in what was built: recipes and substitutions, private notes and the metrics *dashboard*.

Closing remarks

I started this work on the condition that it would be useful for something real, and what has taught me the most is how far my first idea was from what nutritionists needed. Without the conversations with them I would have built yet another management platform, and the copilot, the part that gives the project its meaning, would not have existed. The other lesson is my thesis supervisors' guidance: the minimal version that applies the features, verified, is worth more than an ambitious one left half done. Applying it was hard for me at first, because it meant leaving out things I had already designed, but it is what made it possible to arrive at a deployed application with a measured engine instead of a pretty prototype. What the professionals say when they use it is what remains to be heard.

Bibliografía

*Y así, del mucho leer y del poco dormir, se
le secó el cerebro de manera que vino a
perder el juicio.*
(modificar en Cascaras\bibliografia.tex)

Miguel de Cervantes Saavedra

AGENCIA ESPAÑOLA DE SEGURIDAD ALIMENTARIA Y NUTRICIÓN. BEDCA: Base de datos española de composición de alimentos. <https://www.bedca.net>, 2026. Consultado el 5 de septiembre de 2026.

ANTHROPIC. Claude Code documentation. <https://code.claude.com/docs>, 2026a. Consultado el 5 de septiembre de 2026.

ANTHROPIC. Model deprecations. <https://platform.claude.com/docs/en/about-claude/model-deprecations>, 2026b. Documentación de la Claude Developer Platform, con los identificadores y el estado de cada modelo. Consultado el 5 de septiembre de 2026.

BALINTFY, J. L. Menu planning by computer. *Communications of the ACM*, vol. 7(4), páginas 255–259, 1964.

CRONOMETER SOFTWARE INC. Cronometer pro: Nutrition tracking for health professionals. <https://cronometer.com/pro/>, 2025. Consultado en 2025.

DANTZIG, G. B. The diet problem. *Interfaces*, vol. 20(4), páginas 43–47, 1990.

DIETOPRO. DietoPro: Software nutricional para dietistas-nutricionistas. <https://dietopro.com>, 2026. Consultado el 5 de septiembre de 2026.

EAT THIS MUCH. Eat this much: Automatic meal planner. <https://www.eatthismuch.com>, 2026. Consultado el 3 de septiembre de 2026.

FIGMA, INC. Figma. <https://www.figma.com>, 2026a. Consultado el 5 de septiembre de 2026.

- FIGMA, INC. Guide to the Figma MCP server. <https://help.figma.com/hc/en-us/articles/32132100833559-Guide-to-the-Figma-MCP-server>, 2026b. Consultado el 5 de septiembre de 2026.
- FOODZILLA. Foodzilla: Meal planning software for dietitians and nutritionists. <https://foodzilla.com>, 2026. Consultado el 3 de septiembre de 2026.
- GAZAN, R., BROUZES, C. M. C., VIEUX, F., MAILLOT, M., LLUCH, A. y DARMON, N. Mathematical optimization to explore tomorrow's sustainable diets: A narrative review. *Advances in Nutrition*, vol. 9(5), páginas 602–616, 2018.
- GOOGLE. Gemini API release notes. <https://ai.google.dev/gemini-api/docs/changelog>, 2026. Consultado el 5 de septiembre de 2026.
- HEALTHIE. Healthie: Practice management and patient engagement platform. <https://gethealthie.com>, 2025. Consultado en 2025.
- INDYA. INDYA: Software de nutrición para nutricionistas. <https://getindya.com>, 2026. Consultado el 5 de septiembre de 2026.
- KAHRAMAN, A. y SEVEN, H. A. Healthy daily meal planner. En *Proceedings of the 7th Annual Workshop on Genetic and Evolutionary Computation (GECCO 2005 Workshops)*. ACM, 2005.
- KAMBHAMPATI, S., VALMEEKAM, K., GUAN, L., VERMA, M., STECHLY, K., BHAMBRI, S., SALDYT, L. y MURTHY, A. Position: LLMs can't plan, but can help planning in LLM-Modulo frameworks. En *Proceedings of the 41st International Conference on Machine Learning (ICML 2024)*, PMLR 235, páginas 22895–22907. 2024.
- MICHAILIDIS, K., TSOUROS, D. y GUNS, T. Constraint modelling with LLMs using in-context learning. En *30th International Conference on Principles and Practice of Constraint Programming (CP 2024)*, LIPIcs 307, páginas 20:1–20:27. Schloss Dagstuhl, 2024.
- MIFFLIN, M. D., ST JEOR, S. T., HILL, L. A., SCOTT, B. J., DAUGHERTY, S. A. y KOH, Y. O. A new predictive equation for resting energy expenditure in healthy individuals. *The American Journal of Clinical Nutrition*, vol. 51(2), páginas 241–247, 1990.
- NETLIFY, INC. Netlify. <https://www.netlify.com>, 2026. Consultado el 5 de septiembre de 2026.
- NUTRIMEAL. Nutrimeal: Herramienta de gestión y personalización de dietas para profesionales. <https://nutrimeal.es>, 2026. Consultado el 5 de septiembre de 2026.
- NUTRIUM. Nutrium: Software de nutrición para profesionales. <https://nutrium.com>, 2026. Consultado el 3 de septiembre de 2026.

- OLLAMA. Ollama: Get up and running with large language models locally. <https://ollama.com>, 2026. Repositorio en <https://github.com/ollama/ollama>. Consultado el 5 de septiembre de 2026.
- OPEN FOOD FACTS. Open food facts: Base de datos colaborativa y abierta de productos alimentarios. <https://world.openfoodfacts.org>, 2025. API REST pública. Consultado en 2025.
- PAPASTRATIS, I., KONSTANTINIDIS, D., DARAS, P. y DIMITROPOULOS, K. AI nutrition recommendation using a deep generative model and ChatGPT. *Scientific Reports*, vol. 14, página 14620, 2024.
- PERRON, L. y FURNON, V. Or-tools. <https://developers.google.com/optimization>, 2024. Google.
- PETOT, G. J., MARLING, C. y STERLING, L. An artificial intelligence system for computer-assisted menu planning. *Journal of the American Dietetic Association*, vol. 98(9), páginas 1009–1014, 1998.
- PRACTICE BETTER. Practice better: All-in-one practice management software. <https://practicebetter.io>, 2026. Consultado el 3 de septiembre de 2026.
- QWEN TEAM. Qwen2.5 technical report. <https://arxiv.org/abs/2412.15115>, 2024. Alibaba Group.
- RAMÍREZ, S. FastAPI. <https://fastapi.tiangolo.com>, 2026. Consultado el 5 de septiembre de 2026.
- RENDER. Render: Cloud application platform. <https://render.com>, 2026. Consultado el 5 de septiembre de 2026.
- STIGLER, G. J. The cost of subsistence. *Journal of Farm Economics*, vol. 27(2), páginas 303–314, 1945.
- SUPABASE, INC. Supabase: The Postgres development platform. <https://supabase.com>, 2026. Consultado el 5 de septiembre de 2026.
- TAILWIND LABS. Tailwind CSS. <https://tailwindcss.com>, 2026. Consultado el 5 de septiembre de 2026.
- THAT CLEAN LIFE. That clean life: Nutrition planning software for health professionals. <https://thatcleanlife.com>, 2026. Consultado el 3 de septiembre de 2026.
- THE POSTGRESQL GLOBAL DEVELOPMENT GROUP. PostgreSQL: The world's most advanced open source relational database. <https://www.postgresql.org>, 2026. Consultado el 5 de septiembre de 2026.
- VALMEEKAM, K., MARQUEZ, M., SREEDHARAN, S. y KAMBHAMPATI, S. On the planning abilities of large language models: A critical investigation. En *Advances in Neural Information Processing Systems 36 (NeurIPS 2023)*. 2023.

VERCEL, INC. Next.js: The React framework. <https://nextjs.org>, 2026. Consultado el 5 de septiembre de 2026.

WORKOS. Radix Primitives: Unstyled, accessible components for React. <https://www.radix-ui.com>, 2026. Consultado el 5 de septiembre de 2026.

Catálogo original de requisitos (versión V1)

Este anexo recoge el catálogo completo de requisitos *tal y como se formuló inicialmente*, antes del pivote de alcance consensuado con los directores del TFG en abril de 2026. Su función no es normativa (el alcance que realmente se implementa es el descrito en el capítulo 3), sino *histórica*: conservar la visión inicial como registro del proceso de reformulación del proyecto y facilitar la trazabilidad de cada decisión de recorte.

La versión V1 contenía 61 requisitos funcionales, 7 requisitos no funcionales y 8 innovaciones clave. Tras dos iteraciones de triaje, la versión implementada (V2) reduce estas cifras a 27 requisitos funcionales, 14 requisitos no funcionales (siete de ellos específicos del copiloto IA) y una única innovación clave centrada en la generación semi-automática de planes nutricionales por nutricionista. Las razones y el criterio de triaje que llevaron a esta reformulación se desarrollan en la sección 3.1 del cuerpo principal.

Todo el contenido que sigue (tablas de requisitos, dependencias funcionales y funcionalidades resultantes) se presenta sin modificaciones respecto a su formulación original, incluyendo terminología, etiquetas de *innovación clave* y referencias a bloques que posteriormente han sido descartados o redimensionados.

A.1. Requisitos no funcionales (V1)

Los requisitos no funcionales definen las restricciones y condiciones de calidad que debe cumplir el sistema, independientemente de las funcionalidades concretas que ofrezca. Cada uno de ellos se deriva de las necesidades observadas durante la fase de investigación con nutricionistas y del contexto de uso de la aplicación.

RNF-01 — Interfaz diseñada para escritorio. Durante las entrevistas con nutricionistas se constató que tanto los profesionales como sus clientes trabajan desde un portátil durante las consultas. Ninguno de los nutricionistas consultados realizaba gestiones desde el móvil en su entorno laboral. Por este motivo, toda la

interfaz se ha diseñado para una resolución de escritorio (1280 px de ancho mínimo), siguiendo un enfoque *desktop-first*. Esto no excluye el acceso desde otros dispositivos, pero la experiencia principal está optimizada para portátil.

RNF-02 — Aplicación web accesible desde navegador. Al tratarse de una aplicación web, no requiere instalación ni configuración local. Esto es especialmente relevante porque muchos nutricionistas trabajan en varias clínicas distintas y no siempre utilizan el mismo equipo. Acceder desde cualquier navegador garantiza disponibilidad sin dependencia del hardware.

RNF-03 — Interfaz moderna, limpia y profesional. La aplicación va a ser utilizada por profesionales de la salud en su entorno de trabajo, por lo que debe transmitir confianza y profesionalidad. Se tomó como referencia visual Nutrium, el líder del mercado identificado durante el análisis de competencia, por su interfaz limpia y orientada a productividad. El objetivo es que el nutricionista perciba la herramienta como un instrumento profesional, no como un producto genérico.

RNF-04 — Navegación intuitiva con sidebar permanente. Al tratarse de una herramienta de uso diario, el nutricionista necesita acceder rápidamente a cualquier sección sin perder el contexto de lo que está haciendo. Un sidebar fijo y visible en todo momento (similar al de aplicaciones como Gmail o Notion) permite esta navegación inmediata. Se descartó el menú tipo hamburguesa porque en una interfaz de escritorio hay espacio suficiente para mantener la navegación siempre visible.

RNF-05 — Separación clara entre panel de nutricionista y panel de cliente. Ambos perfiles de usuario tienen necesidades completamente distintas: el nutricionista gestiona múltiples clientes, citas, formularios y planes; el cliente interactúa con sus propios datos, su plan y su progreso. Mezclar ambas experiencias en una sola interfaz generaría confusión. Por ello, cada perfil dispone de su propio panel con navegación, funcionalidades y diseño adaptados, aunque ambos paneles comparten la misma base de datos subyacente.

RNF-06 — Escalabilidad. La arquitectura del sistema debe funcionar tanto para un nutricionista autónomo como para una clínica con varios profesionales. Esto afecta directamente al diseño de la base de datos (la relación entre nutricionistas y clínicas se modela como N:M) y a la estructura de permisos, que debe contemplar distintos niveles de acceso según el contexto.

RNF-07 — Seguridad y privacidad de datos. La plataforma maneja datos de salud de los clientes, que son especialmente sensibles. El sistema debe contemplar autenticación segura, roles diferenciados y acceso restringido a la información según el perfil del usuario. Por ejemplo, las notas privadas que el nutricionista escribe sobre un cliente no deben ser visibles para dicho cliente en ningún caso.

A.2. Requisitos funcionales (V1)

Los requisitos funcionales describen las acciones concretas que el sistema debe permitir realizar. Se organizan por bloques temáticos, indicando para cada requisito el actor principal que lo ejecuta: el nutricionista (N), el cliente (C) o el propio sistema de forma automática (S). Aquellos bloques que constituyen innovaciones clave del proyecto (funcionalidades no presentes en ningún competidor analizado) se señalan expresamente.

A.2.1. Gestión de clientes

ID	Requisito	Actor
RF-01	Crear, editar y consultar perfiles de clientes con historial clínico, datos personales, objetivos y mediciones.	N
RF-02	Clasificar y filtrar clientes por estado (activo, inactivo, nuevo) y buscar por nombre u otros criterios.	N
RF-03	Escribir notas privadas asociadas a cada cliente, visibles únicamente para el nutricionista, sustituyendo la agenda física.	N
RF-04	Editar datos personales, registrar mediciones corporales y subir fotos de progreso.	C
RF-05	Consultar las mediciones, fotos de progreso y evolución de cada uno de sus clientes.	N
RF-60	Acceder a una ficha detallada por cliente que consolide foto, datos básicos, notas privadas, plan actual, próxima sesión, alertas recientes e histórico de check-ins en una sola pantalla.	N

Tabla A.1: Requisitos funcionales V1 — Gestión de clientes.

A.2.2. Calendario y citas

ID	Requisito	Actor
RF-06	Gestionar una agenda de citas con vista diaria, semanal y mensual.	N
RF-07	Diferenciar citas por tipo (clínica A, clínica B, privada, online) mediante un código de colores.	N
RF-08	Gestionar múltiples clínicas y asignar citas a cada una de ellas.	N
RF-09	Configurar la disponibilidad horaria para que los clientes puedan solicitar citas.	N
RF-10	Ver citas programadas, solicitar nuevas según disponibilidad, y cancelar o reprogramar citas existentes.	C
RF-11	Enviar recordatorios automáticos de citas próximas.	S

Tabla A.2: Requisitos funcionales V1 — Calendario y citas.

A.2.3. Mensajería

ID	Requisito	Actor
RF-12	Ofrecer un chat de mensajería directa entre nutricionista y cliente, con historial completo de conversaciones.	N/C
RF-13	Permitir adjuntar archivos (documentos, imágenes, PDFs) en los mensajes.	N/C
RF-43	Iniciar una videollamada con el interlocutor directamente desde el chat, sin salir de la plataforma.	N/C
RF-44	Iniciar una llamada de voz con el interlocutor desde el chat.	N/C
RF-61	Ofrecer al cliente un canal de chat con otros clientes del mismo nutricionista, manteniendo el aislamiento entre consultas de distintos profesionales.	C

Tabla A.3: Requisitos funcionales V1 — Mensajería.

A.2.4. Formularios personalizables

ID	Requisito	Actor
RF-14	Crear formularios personalizados mediante un editor visual, con múltiples tipos de campo (texto, número, escala, selección múltiple).	N
RF-15	Gestionar una biblioteca de plantillas de formularios predefinidos (primera consulta, seguimiento, valoración).	N
RF-16	Asignar formularios a clientes de forma manual o programada (semanal, mensual, tras cada cita).	N
RF-17	Ver formularios pendientes, completarlos y consultar el historial de respuestas anteriores.	C

Tabla A.4: Requisitos funcionales V1 — Formularios personalizables.

A.2.5. Sistema de alertas inteligentes (*innovación clave V1*)

Este bloque constituye una de las principales innovaciones de NutriApp en su versión original. Ninguno de los competidores analizados ofrece un sistema de alertas configurables vinculado a las respuestas de formularios. Su objetivo es que el nutricionista pueda actuar de forma proactiva ante problemas de sus clientes, en lugar de enterarse solo cuando el cliente se lo comunica.

A.2.6. Planes nutricionales

A.2.7. Lista de la compra inteligente (*innovación clave V1*)

Otra innovación diferenciadora de NutriApp en su planteamiento original. Los competidores que ofrecen listas de la compra las generan de forma estática; ninguno incorpora un sistema de sustituciones inteligentes basado en equivalencia nutricional.

ID	Requisito	Actor
RF-18	Definir reglas de alerta condicionales por formulario (ejemplo: “si comidas falladas > 2, generar alerta crítica”).	N
RF-19	Evaluar automáticamente las respuestas de formularios contra las reglas configuradas y generar alertas con nivel de prioridad (crítica, media, informativa).	S
RF-20	Generar alertas automáticas cuando un cliente no realiza check-in durante un número configurable de días.	S
RF-21	Ver, gestionar y resolver alertas desde el dashboard, con indicadores de color por prioridad.	N
RF-22	Resolver alertas y registrar las acciones tomadas al respecto.	N
RF-23	Enviar un mensaje rápido al cliente directamente desde una alerta, con el contexto del caso precargado.	N

Tabla A.5: Requisitos funcionales V1 — Sistema de alertas inteligentes.

ID	Requisito	Actor
RF-24	Crear planes nutricionales completos organizados por día y por comida (desayuno, comida, cena, snacks).	N
RF-25	Gestionar una base de datos de alimentos con información nutricional (calorías, macronutrientes, categoría).	N
RF-26	Crear y gestionar recetas compuestas por alimentos de la base de datos, con instrucciones de preparación e información nutricional agregada.	N
RF-27	Definir sustituciones entre alimentos del mismo grupo nutricional con equivalencia de macronutrientes.	N
RF-28	Guardar planes nutricionales como plantillas reutilizables para aplicar a otros clientes.	N
RF-29	Consultar el plan nutricional actual con vista semanal, desglose por comidas, recetas e información nutricional.	C
RF-59	Sustituir una comida del plan por una opción equivalente pre-configurada por el nutricionista, manteniendo el equilibrio de macronutrientes del día.	C

Tabla A.6: Requisitos funcionales V1 — Planes nutricionales.

ID	Requisito	Actor
RF-30	Generar automáticamente una lista de la compra a partir del plan nutricional asignado, organizada por categorías de alimentos.	S
RF-31	Marcar ingredientes como “no disponible” y recibir sugerencias de sustitución del mismo grupo nutricional.	C
RF-32	Marcar ítems como comprados y exportar o compartir la lista.	C

Tabla A.7: Requisitos funcionales V1 — Lista de la compra inteligente.

A.2.8. Check-in diario y gamificación (*innovación clave V1*)

La tercera innovación clave del proyecto en su versión original. Los competidores analizados ofrecen, en el mejor de los casos, check-ins básicos sin mecánicas de gamificación ni conexión con un sistema de alertas.

ID	Requisito	Actor
RF-33	Registrar diariamente: agua consumida, suplementos tomados, comidas completadas del plan, ejercicio realizado y hábitos personalizados.	C
RF-34	Calcular y mostrar la racha de días consecutivos con check-in completado, incluyendo comparación con la mejor racha histórica.	S
RF-35	Configurar hábitos personalizados para cada cliente (horas de sueño, nivel de estrés, meditación, etc.).	N

Tabla A.8: Requisitos funcionales V1 — Check-in diario y gamificación.

A.2.9. Seguimiento y progreso

ID	Requisito	Actor
RF-36	Visualizar la evolución mediante gráficos de peso, medidas corporales y porcentaje de adherencia.	C
RF-37	Mostrar un calendario de cumplimiento tipo <i>heat map</i> (estilo contribuciones de GitHub) basado en los check-ins diarios.	S
RF-38	Subir y comparar fotos de progreso organizadas cronológicamente.	C

Tabla A.9: Requisitos funcionales V1 — Seguimiento y progreso.

A.2.10. Informes y análisis

ID	Requisito	Actor
RF-39	Visualizar un dashboard con métricas globales: clientes activos, citas del día y formularios pendientes.	N
RF-40	Exportar informes de clientes y métricas en formato PDF.	N

Tabla A.10: Requisitos funcionales V1 — Informes y análisis.

A.2.11. Base de datos de alimentos con reconocimiento inteligente (*innovación clave V1*)

Una de las fricciones principales detectadas durante la investigación es el tiempo que tanto el nutricionista como el cliente dedican a consultar información nutricional

sobre alimentos concretos. El cliente, sobre todo, suele abandonar el registro de su dieta porque implica buscar manualmente las calorías de cada producto que consume. La propuesta original de NutriApp resolvía este punto mediante una base de datos de alimentos accesible desde ambos paneles, integrada con una API abierta como Open Food Facts, y complementada con dos mecanismos de captura rápida: el escaneo de códigos de barras y el reconocimiento de alimentos por fotografía usando un modelo de inteligencia artificial.

ID	Requisito	Actor
RF-45	Consultar una base de datos de alimentos con buscador por nombre, alimentada por una fuente externa como Open Food Facts.	N/C
RF-46	Escanear el código de barras de un producto para obtener su información nutricional de forma automática.	C
RF-47	Identificar alimentos mediante el análisis de fotografías usando un modelo de reconocimiento de imagen basado en IA.	C
RF-48	Añadir alimentos personalizados a la base de datos (recetas propias o productos locales) con sus macronutrientes.	N
RF-49	Registrar un alimento consumido como parte del plan diario del cliente desde el propio buscador.	C
RF-50	Comparar los macronutrientes de un alimento con los objetivos nutricionales restantes del día antes de decidir si añadirlo al plan.	C

Tabla A.11: Requisitos funcionales V1 — Base de datos de alimentos.

A.2.12. Marketplace de recursos educativos (*innovación clave V1*)

El análisis de competencia reveló que algunas plataformas profesionales como Healthie incorporan un marketplace donde los nutricionistas pueden vender contenido educativo y los clientes pueden adquirirlo. La propuesta original de NutriApp recogía esta idea y la adaptaba a sus dos paneles: el nutricionista publica sus propios recursos (guías, libros, plantillas, cursos) y gestiona sus ventas, mientras que el cliente navega el catálogo general y dispone además de una sección destacada con los recursos publicados específicamente por su propio profesional. De esta forma, la plataforma ofrecía al nutricionista una vía adicional de monetización y al cliente acceso a contenido educativo curado por alguien en quien ya confía.

A.2.13. Automatización de procesos (*innovación clave V1*)

La automatización de tareas repetitivas es la forma más directa de reducir la carga administrativa del nutricionista. La propuesta original ampliaba el concepto más allá del envío programado de formularios: ofrecía un motor de reglas configurables con disparadores y acciones asociadas, de forma que el profesional podía modelar procedimientos típicos de su práctica (bienvenida a nuevos clientes, seguimiento tras

ID	Requisito	Actor
RF-51	Explorar un catálogo de recursos educativos (cursos, libros, plantillas, guías) filtrado por categoría y con valoraciones de otros usuarios.	N/C
RF-52	Publicar recursos propios en el marketplace asignándoles categoría, descripción, precio y materiales asociados.	N
RF-53	Gestionar los recursos publicados y el historial de ventas propias desde la sección <i>My Published</i> .	N
RF-54	Adquirir recursos del marketplace y acceder a ellos desde una biblioteca personal (<i>My Library</i>).	C
RF-55	Visualizar una sección destacada con los recursos publicados específicamente por el propio nutricionista del cliente.	C

Tabla A.12: Requisitos funcionales V1 — Marketplace de recursos educativos.

sesión, detección de baja adherencia, celebración de hitos) y dejar que el sistema los ejecutase sin intervención manual.

ID	Requisito	Actor
RF-41	Automatizar el envío de formularios programados (semanal, mensual, tras cada cita).	S
RF-42	Automatizar las notificaciones de nuevos mensajes, formularios asignados y eventos relevantes del sistema.	S
RF-56	Definir reglas de automatización con disparadores (nuevo cliente, baja adherencia sostenida, cita próxima, hito alcanzado) y acciones asociadas (enviar mensaje, asignar formulario, crear tarea interna).	N
RF-57	Activar, desactivar, editar o eliminar reglas de automatización de forma individual.	N
RF-58	Ejecutar automáticamente las acciones de las reglas activas cuando se cumplen sus disparadores.	S

Tabla A.13: Requisitos funcionales V1 — Automatización de procesos.

A.3. Dependencias funcionales (V1)

Los requisitos funcionales no actúan de forma aislada: muchos de ellos dependen de otros para conformar las funcionalidades completas de la plataforma. La tabla A.14 muestra estas relaciones de dependencia tal y como se formularon originalmente, permitiendo trazar cada funcionalidad hasta los requisitos que la hacen posible.

En el planteamiento original se destacaba que la combinación de formularios personalizables con reglas de alerta asociadas constituye una innovación en sí misma: la posibilidad de que el nutricionista defina, sobre cada formulario que crea, reglas condicionales que generen alertas automáticas al ser evaluadas. Ningún competidor analizado conecta estas dos funcionalidades de forma integrada. Esta combinación

Funcionalidad	Requisitos involucrados
Gestión integral de clientes	RF-01, RF-02, RF-03, RF-04, RF-05
Ficha detallada del cliente	RF-01, RF-03, RF-05, RF-19, RF-34, RF-60
Calendario multi-clínica	RF-06, RF-07, RF-08, RF-09, RF-10, RF-11
Mensajería profesional	RF-12, RF-13
Videollamadas y llamadas de voz	RF-12, RF-43, RF-44
Comunidad entre clientes	RF-12, RF-61
Formularios personalizables	RF-14, RF-15, RF-16, RF-17
Sistema de alertas inteligentes*	RF-14, RF-18, RF-19, RF-20, RF-21, RF-22, RF-23
Formularios con reglas de alerta*	RF-14, RF-15, RF-16, RF-18, RF-19
Planes nutricionales	RF-24, RF-25, RF-26, RF-27, RF-28, RF-29
Sustitución de comidas por equivalencias	RF-27, RF-59
Lista de compra inteligente*	RF-24, RF-25, RF-27, RF-30, RF-31, RF-32
Base de datos de alimentos con IA*	RF-25, RF-45, RF-46, RF-47, RF-48, RF-49, RF-50
Check-in diario gamificado*	RF-33, RF-34, RF-35, RF-37
Seguimiento y progreso	RF-04, RF-05, RF-33, RF-36, RF-37, RF-38
Informes y análisis	RF-05, RF-34, RF-36, RF-39, RF-40
Marketplace de recursos*	RF-51, RF-52, RF-53, RF-54, RF-55
Motor de automatizaciones*	RF-18, RF-41, RF-42, RF-56, RF-57, RF-58

Tabla A.14: Dependencias funcionales V1: funcionalidades y sus requisitos. Las funcionalidades marcadas con * son innovaciones clave del planteamiento original.

emergía de la intersección de los bloques de Formularios (RF-14 a RF-16) y Alertas (RF-18, RF-19), y era la que daba al sistema su capacidad de detección proactiva de problemas.

También se observaba que el bloque de seguimiento y progreso actuaba como punto de convergencia de varios sistemas: los datos del check-in diario (RF-33) alimentan el heat map (RF-37) y las métricas de adherencia (RF-36), mientras que las mediciones y fotos del cliente (RF-04) eran consultadas por el nutricionista (RF-05) para evaluar la evolución.

A.4. Funcionalidades resultantes (V1)

A partir de los requisitos V1 definidos y sus dependencias, las funcionalidades inicialmente propuestas para NutriApp se agrupaban en las siguientes áreas:

Gestión de clientes (RF-01 a RF-05). Perfiles completos, clasificación por categorías, filtros de búsqueda, notas privadas del nutricionista y acceso a la evolución del cliente. Sustituye la combinación de cuaderno físico + notas del móvil + plataforma de la clínica.

Calendario y citas (RF-06 a RF-11). Agenda con vistas múltiples, soporte multi-clínica con código de colores, configuración de disponibilidad, solicitud de citas por parte del cliente y recordatorios automáticos. Sustituye Google Calendar.

Mensajería profesional (RF-12, RF-13, RF-43, RF-44). Chat integrado con historial y adjuntos, vinculado al perfil de cada cliente, con botones de videollamada y llamada de voz directamente en la cabecera del chat. Sustituye tanto a WhatsApp como a las herramientas externas de videoconsulta.

Comunidad entre clientes (RF-12, RF-61). El cliente dispone de un canal de chat con otros clientes del mismo nutricionista, pensado como apoyo entre pares. El sistema garantiza el aislamiento entre consultas de profesionales distintos, por lo que un cliente nunca puede comunicarse con clientes de un nutricionista que no sea el suyo.

Ficha detallada del cliente (RF-60). El nutricionista cuenta con una vista dedicada por cada cliente que consolida en una sola pantalla la foto, los datos básicos, las notas privadas, el plan actual, la próxima sesión, las alertas recientes y el histórico de check-ins. Evita tener que saltar entre varias secciones para obtener una visión global del caso.

Formularios personalizables (RF-14 a RF-17). Editor visual con múltiples tipos de campo, biblioteca de plantillas, asignación manual o programada, y acceso del cliente a sus formularios. Sustituye Google Forms.

Sistema de alertas inteligentes (RF-18 a RF-23). *Innovación clave V1*. Reglas condicionales por formulario, evaluación automática de respuestas, alertas por inactividad, dashboard de gestión con prioridades por color y acción directa desde la alerta. Sin equivalente en la competencia.

Planes nutricionales (RF-24 a RF-29, RF-59). Constructor de dietas por día y comida, base de datos de alimentos, gestión de recetas, sistema de sustituciones por equivalencia nutricional, plantillas reutilizables y vista del plan para el cliente. Incluye la posibilidad de que el cliente cambie una comida concreta por otra opción equivalente preconfigurada por el nutricionista sin romper el equilibrio de macronutrientes del día.

Base de datos de alimentos con reconocimiento inteligente (RF-45 a RF-50). *Innovación clave V1*. Base de datos accesible desde ambos paneles, alimentada por una fuente externa como Open Food Facts, con tres vías de captura: búsqueda

por nombre, escaneo de código de barras y reconocimiento de alimentos por fotografía mediante un modelo de IA. El cliente puede comparar cualquier alimento con sus objetivos nutricionales restantes del día antes de añadirlo al plan, y el nutricionista puede ampliar la base con alimentos personalizados.

Lista de la compra inteligente (RF-30 a RF-32). *Innovación clave V1*. Generación automática desde el plan, sustituciones inteligentes al marcar ingredientes como no disponibles, y gestión de la lista por el cliente. Sin equivalente en la competencia.

Check-in diario gamificado (RF-33 a RF-35). *Innovación clave V1*. Registro diario de adherencia en múltiples categorías, hábitos personalizados configurados por el nutricionista, y rachas de cumplimiento con mecánica de gamificación. Sin equivalente en la competencia.

Seguimiento y progreso (RF-36 a RF-38). Gráficos de evolución, heat map de cumplimiento estilo GitHub y comparativa de fotos de progreso. Integra datos del check-in diario y las mediciones del cliente.

Informes y análisis (RF-39, RF-40). Dashboard de métricas globales para el nutricionista y exportación de informes en PDF.

Marketplace de recursos educativos (RF-51 a RF-55). *Innovación clave V1*. Catálogo de recursos formativos (cursos, libros, plantillas, guías) con filtros por categoría y valoraciones. El nutricionista puede publicar sus propios recursos y gestionar sus ventas, y el cliente dispone de una biblioteca personal con los recursos adquiridos además de una sección destacada con los contenidos publicados específicamente por su profesional.

Motor de automatizaciones (RF-41, RF-42, RF-56 a RF-58). *Innovación clave V1*. Motor de reglas configurables con disparadores (nuevo cliente, baja adherencia sostenida, cita próxima, hito alcanzado) y acciones asociadas (enviar mensaje, asignar formulario, crear tarea interna). Permite al nutricionista modelar procedimientos recurrentes de su práctica y delegar su ejecución en el sistema, ampliando la automatización básica de envío de formularios y notificaciones.

A.5. Nota final sobre el alcance definitivo

El alcance efectivamente implementado en el TFG corresponde al subconjunto descrito en el capítulo 3 tras la reformulación recogida en la sección 3.1. Los requisitos aquí documentados que no forman parte del alcance final (*marketplace*, motor de automatizaciones avanzado, formularios personalizables, sistema de alertas inteligentes en su versión original, lista de la compra inteligente, reconocimiento de alimentos por imagen, videollamadas, chat entre clientes, entre otros) se recogen como trabajo futuro con su correspondiente justificación de posposición.

A.6. Comparativa con la competencia en el planteamiento inicial

El planteamiento inicial se comparó con cuatro plataformas bajo trece criterios funcionales, y la tabla A.15 es la que acompañaba a la idea original. Además de Nutrium y Practice Better, analizadas en el capítulo 2, incluía a Healthie (Healthie, 2025), por su marketplace de recursos, y a Cronometer Pro (Cronometer Software Inc., 2025), por su base de alimentos y su escáner de código de barras. La columna de NutriApp recoge lo que la V1 prometía, no lo que se entrega: la mayoría de esas filas corresponden a los bloques eliminados en la reformulación (sección 3.1), y la comparativa vigente es la tabla 2.1.

Funcionalidad (V1)	NutriApp	Nutrium	P. Better	Healthie	Cron. Pro
Gestión de clientes	Sí	Sí	Sí	Sí	Parc.
Calendario y citas	Sí	Sí	Sí	Sí	No
Mensajería integrada	Sí	Sí	Sí	Sí	No
Videollamada y voz en chat	Sí	No	Parc.	Parc.	No
Formularios personalizables	Sí	Sí	Sí	Sí	No
Alertas configurables por regla	Sí	No	No	No	No
Planes nutricionales	Sí	Sí	Sí	Sí	No
Lista de compra inteligente	Sí	Parc.	No	No	No
Base de alimentos con buscador	Sí	Sí	No	No	Sí
Escaneo de código de barras	Sí	No	No	No	Sí
Reconocimiento por fotografía	Sí	No	No	No	No
Check-in diario gamificado	Sí	No	No	No	No
Marketplace de recursos	Sí	No	No	Sí	No
Motor de automatizaciones	Sí	No	Parc.	Parc.	No
Comunidad entre clientes	Sí	No	No	No	No
Disponible en castellano	Sí	Sí	No	No	No

Tabla A.15: Cobertura funcional prometida por la idea inicial (V1) frente a los competidores analizados en 2025.

Anexo B

Formalización completa del motor de generación

Este anexo recoge la formalización completa del modelo del capítulo 5: los umbrales con sus valores, las 15 reglas invariantes, los 16 tipos configurables con su formulación y el mapa de cada elemento formal a su función en el código. La notación es la del capítulo: $ha_consumido[f, d, m]$ y $gramos_alimento[f, d, m]$ son la presencia y los gramos del alimento f en la comida m del día d ; $nut_{d,n}$ es la suma escalada del nutriente n en el día d ; y $scale_target(v) = round(v \cdot s \cdot 100)$ lleva un valor real a la escala entera del modelo, con $s = 10$. Siguiendo el convenio habitual, una variable booleana dentro de un sumatorio aporta 1 si la condición se cumple y 0 en caso contrario.

B.1. Umbrales y pesos del modelo

Todos los umbrales son constantes con nombre en la configuración del solver. Los cinco campos de `ObjectiveBands` son las tolerancias de los objetivos blandos de energía y macronutrientes: lo que queda dentro de ellas no penaliza (sección B.6).

Los pesos por familia de la función objetivo son $w_{kcal} = 10$, $w_{protein} = 8$, $w_{carb} = 6$, $w_{fat} = 6$, $w_{no_repeat} = 5\,000$, $w_{variety} = 5\,000$, $w_{prefer} = 4\,000$ y $w_{spread} = 4\,000$. Las familias no nutricionales llevan incorporado un factor de 1 000 que compensa la escala entera: sus términos se cuentan en unidades de 1 (una aparición, un alimento sin usar) mientras que las desviaciones nutricionales se miden en unidades escaladas, donde una kcal son 1000 (capítulo 5). Sobre estos pesos, cada fila de restricción modula con su peso propio de 1 a 10 (sección B.6).

B.2. Variables y enlace

Dos familias de variables por cada terna alimento, día y comida, más una tercera para los alimentos que se sirven por piezas:

Constante	Valor	Papel
NUTRIENT_SCALE	10	escala entera de los valores nutricionales
MIN_GRAMS_PRESENT	10 g	suelo absoluto de presencia
MAX_ITEMS_PER_MEAL	4	tope de alimentos por comida (R2)
MAX_SAME_FOOD_PER_DAY	2	repeticiones de un alimento en el día (R6)
MIN_DISTINCT_PER_DAY	4	variedad mínima diaria (R7)
MIN_DISTINCT_PER_WEEK	10	variedad mínima semanal (R8)
MAX_APPEARANCES_PER_DAY_RATIO	0.6	tope de reparto por día de plan
FLOOR_KCAL	1200 / 1500	suelo calórico absoluto (mujeres / hombres, R3)
PROTEIN_G_PER_KG	0.8 – 2.2	rango humano de proteína en g/kg (R4)
DEFAULT_WEIGHT_KG	70	peso por defecto si falta la antropometría
FAT_MIN_PCT_ENERGY	15	mínimo de energía en grasa, en porcentaje (R5)
MIN_ITEMS_MAIN_MEAL	2	composición mínima de las principales (R12)
CONDIMENT_MAX_PER_DAY	2	tope diario de condimentos (R14)
SWEET_FRUIT_MAX_PER_MEAL	1	dulce y fruta por comida (R15)
FALLBACK_MIN_SERVING_G	20 g	mínimo de ración sin perfil
FALLBACK_MAX_SERVING_G	250 g	máximo de ración sin perfil
ObjectiveBands.kcal_day	50 kcal	banda diaria del objetivo calórico
ObjectiveBands.kcal_mean	25 kcal	banda de la media del plan, objetivo calórico
ObjectiveBands.macro_day_pct	5 %	banda diaria de los macros, sobre el objetivo
ObjectiveBands.macro_mean_pct	2,5 %	banda de la media de los macros, sobre el objetivo
ObjectiveBands.mean_factor	10	factor K del ancla de la media
SOLVE_TIME_LIMIT_S	90 s	límite de tiempo de resolución
SOLVE_RELATIVE_GAP	0.02	brecha relativa de parada
SOLVE_WORKERS	8 / 2	hilos de búsqueda (desarrollo / despliegue)

Tabla B.1: Umbrales del modelo y parámetros de resolución.

$$\text{ha_consumido}[f, d, m] \in \{0, 1\}, \quad \text{gramos_alimento}[f, d, m] \in \{0, 1, \dots, \max_f\}, \\ \forall f \in F, d \in D, m \in M$$

$$\begin{aligned} \text{ha_consumido}[f, d, m] = 1 &\Rightarrow \text{gramos_alimento}[f, d, m] \geq \min_f, \\ \text{ha_consumido}[f, d, m] = 0 &\Rightarrow \text{gramos_alimento}[f, d, m] = 0 \end{aligned}$$

donde los límites por aparición salen del perfil de ración del alimento, con el suelo absoluto de presencia por debajo:

$$\begin{aligned} \min_f &= \text{máx} \left(\text{round}(\min_serving_g(f)), \text{MIN_GRAMS_PRESENT} \right) \\ \max_f &= \text{round}(\max_serving_g(f)) \end{aligned}$$

Para los alimentos con gramos por unidad $gpu(f)$, la variable entera k cuantiza los gramos a medias piezas (k cuenta medias unidades: $k = 3$ son 1,5 piezas):

$$k[f, d, m] \in \left\{ 0, \dots, \left\lfloor \frac{2 \cdot \max_f}{gpu(f)} \right\rfloor \right\}, \quad 2 \cdot \text{gramos_alimento}[f, d, m] = k[f, d, m] \cdot gpu(f)$$

y los alimentos que solo admiten piezas enteras (huevo, yogur) usan la variante sin medias:

$$k[f, d, m] \in \left\{ 0, \dots, \left\lfloor \frac{\max_f}{gpu(f)} \right\rfloor \right\}, \quad \text{gramos_alimento}[f, d, m] = k[f, d, m] \cdot gpu(f)$$

La cuantización compone con el enlace: si el alimento no está presente, los gramos quedan a cero y fuerzan $k = 0$; si lo está, el perfil acota k a las piezas que caben entre $\min_f$ y $\max_f$.

B.3. Restricciones estructurales (R1 a R8)

Todas duras y aplicadas siempre.

R1. Presencia mínima por comida.

$$\sum_{f \in F} \text{ha_consumido}[f, d, m] \geq 1, \quad \forall d \in D, m \in M$$

R2. Cota de alimentos por comida.

$$\sum_{f \in F} \text{ha_consumido}[f, d, m] \leq \text{MAX_ITEMS_PER_MEAL}, \quad \forall d \in D, m \in M$$

R3. Suelo calórico diario. Con el basal de Mifflin-St Jeor sobre el peso w_c (kg), la altura h_c (cm) y la edad a_c :

$$\text{BMR}(c) = 10 w_c + 6,25 h_c - 5 a_c + \begin{cases} +5 & \text{en hombres} \\ -161 & \text{en mujeres} \end{cases}$$

$$\text{floor}(c) = \text{máx} (\text{BMR}(c), \text{FLOOR_KCAL}[\text{sexo}])$$

$$\sum_{m \in M} \sum_{f \in F} \text{kcal100}(f) \cdot \text{gramos_alimento}[f, d, m] \geq \text{scale_target}(\text{floor}(c)),$$

$$\forall d \in D$$

R4. Rango humano de proteína. Con $\text{prot_min}(c)$ y $\text{prot_max}(c)$ iguales a 0.8 y 2.2 veces el peso del cliente (o el peso por defecto si falta):

$$\text{scale_target}(\text{prot_min}(c)) \leq \text{nut}_{d,\text{prot}} \leq \text{scale_target}(\text{prot_max}(c)), \\ \forall d \in D$$

R5. Grasa mínima por energía. Partiendo de $\text{fat_g} \cdot 9 \geq (\text{pct}/100) \cdot \text{kcal}$ y despejando la división con la multiplicación cruzada:

$$\left(\sum_{m,f} \text{nut100}(f, \text{fat}) \cdot \text{gramos_alimento}[f, d, m] \right) \cdot 9 \cdot 100 \geq \\ 15 \cdot \left(\sum_{m,f} \text{kcal100}(f) \cdot \text{gramos_alimento}[f, d, m] \right), \quad \forall d \in D$$

R6. No repetir el mismo alimento dentro del día.

$$\sum_{m \in M} \text{ha_consumido}[f, d, m] \leq \text{MAX_SAME_FOOD_PER_DAY}, \\ \forall f \in F, d \in D$$

R7. Variedad mínima diaria. Con un indicador de uso diario:

$$\text{usado_dia}[f, d] = \max_{m \in M} \text{ha_consumido}[f, d, m], \quad \forall f \in F, d \in D \\ \sum_{f \in F} \text{usado_dia}[f, d] \geq \min(\text{MIN_DISTINCT_PER_DAY}, |F|), \quad \forall d \in D$$

R8. Variedad mínima semanal. Para cada ventana de 7 días, con un indicador de uso en la ventana:

$$\text{usado_sem}[f] = \max_{d \in \text{ventana}, m \in M} \text{ha_consumido}[f, d, m] \\ \sum_{f \in F} \text{usado_sem}[f] \geq \min(\text{MIN_DISTINCT_PER_WEEK}, |F|)$$

B.4. Restricciones de plausibilidad (R9 a R15)

Todas duras y sin literal de asunción: no forman parte del vocabulario que el profesional puede reducir, así que nunca aparecen en un núcleo de infactibilidad. Se usan los subconjuntos COND (alimentos con rol de condimento), DULCE (dulces), FRUTA (fruta) y, para cada alimento f , la lista blanca de franjas momentos(f) derivada de sus etiquetas de momento; $c(m)$ es la franja de la comida m y MAIN el conjunto de franjas principales (desayuno, comida, cena).

R9. Perfil de ración por aparición. Es el enlace presencia-gramos de la sección B.2: presente implica $\min_f \leq \text{gramos_alimento} \leq \max_f$.

R10. Cantidades por unidades. Es la cuantización por piezas de la sección B.2.

R11. Lista blanca de franjas.

$$\begin{aligned} \text{ha_consumido}[f, d, m] &= 0, \\ \forall f : \text{momentos}(f) &\neq \emptyset, \forall d \in D, \forall m : c(m) \notin \text{momentos}(f) \end{aligned}$$

R12. Composición mínima de las comidas principales.

$$\begin{aligned} \sum_{f \in F} \text{ha_consumido}[f, d, m] &\geq \min(\text{MIN_ITEMS_MAIN_MEAL}, |F|), \\ \forall d \in D, \forall m : c(m) &\in \text{MAIN} \end{aligned}$$

R13. El condimento nunca va solo.

$$\begin{aligned} \text{ha_consumido}[f, d, m] &\leq \sum_{f' \in F \setminus \text{COND}} \text{ha_consumido}[f', d, m], \\ \forall f \in \text{COND}, d \in D, m \in M \end{aligned}$$

Caso frontera: si el catálogo visible solo tuviera condimentos, la regla dejaría el problema sin solución por construcción; en ese caso degenerado se omite y se emite un aviso.

R14. Tope diario de condimentos.

$$\begin{aligned} \sum_{f \in \text{COND}} \sum_{m \in M} \text{ha_consumido}[f, d, m] &\leq \text{CONDIMENT_MAX_PER_DAY}, \\ \forall d \in D \end{aligned}$$

R15. Dulce y fruta como complemento.

$$\begin{aligned} \sum_{f \in \text{DULCE} \cup \text{FRUTA}} \text{ha_consumido}[f, d, m] &\leq \text{SWEET_FRUIT_MAX_PER_MEAL}, \\ \forall d \in D, m \in M \end{aligned}$$

B.5. Los 16 tipos configurables

Cada tipo se da con su naturaleza por defecto y su formulación. Las variantes duras acotan el espacio de soluciones; las blandas aportan un término a la función objetivo con el patrón de exceso y defecto de la sección B.6.

meals_per_day y plan_duration_days (estructurales). Fijan los conjuntos M y D y, con ellos, cuántas variables se crean. No generan restricción optimizable; llegan por la firma de la función.

kcal_target (blanda por defecto). Acerca la energía diaria a un valor v . Como blanda, penaliza con el patrón de exceso y defecto de la sección B.6 solo lo que

sobresale de dos bandas: $T_{\text{día}} = 50 \text{ kcal}$ por día y $T_{\text{media}} = 25 \text{ kcal}$ sobre la media del plan, esta con un factor $K = 10$ que la hace casi obligatoria:

$$\begin{aligned} \text{exceso_diario}_d &\geq \text{over}_d + \text{under}_d - T_{\text{día}}, & \text{exceso_diario}_d &\geq 0, & \forall d \in D \\ \sum_{d \in D} (\text{over}_d - \text{under}_d) &= \text{over}_S - \text{under}_S, & \text{exceso_media} &\geq 0 \\ \text{exceso_media} &\geq \text{over}_S + \text{under}_S - |D| \cdot T_{\text{media}} \\ \text{penalización} &= w_{\text{kcal}} \cdot \text{peso de la fila} \cdot \left(\sum_{d \in D} \text{exceso_diario}_d + K \cdot \text{exceso_media} \right) \end{aligned}$$

Como dura:

$$\text{nut}_{d,\text{kcal}} = \text{scale_target}(v), \quad \forall d \in D$$

macro_target (blanda por defecto). Igual que el objetivo calórico pero sobre un macronutriente (proteína, hidratos o grasa), con las bandas en porcentaje del objetivo: un 5 % por día y un 2,5 % en la media. El peso de familia se elige según el macro.

nutrient_min (configurable). Mínimo diario de un nutriente. Como dura, para todo día, $\text{nut}_{d,n} \geq \text{scale_target}(v)$. Como blanda, penaliza solo el defecto, con una variable por día:

$$\text{def}_d \geq \text{scale_target}(v) - \text{nut}_{d,n}, \quad \text{def}_d \geq 0$$

nutrient_max (configurable). Máximo diario de un nutriente. Como dura:

$$\text{nut}_{d,n} \leq \text{scale_target}(v), \quad \forall d \in D$$

Como blanda, penaliza solo el exceso:

$$\text{exc}_d \geq \text{nut}_{d,n} - \text{scale_target}(v), \quad \text{exc}_d \geq 0$$

Se pondera con el peso calórico, por ser el de referencia para límites no atados a un macro concreto.

nutrient_ratio (configurable). Acota el cociente entre dos nutrientes (a sobre b) a un valor v , representado con 3 decimales. Como dura, con cota máxima:

$$\text{nut}_{d,a} \cdot 1000 \leq \text{round}(v \cdot 1000) \cdot \text{nut}_{d,b}, \quad \forall d \in D$$

y con cota mínima la desigualdad se invierte. La variante blanda no se modela: cuando llega, el motor la omite con un aviso.

forbid_food (dura por defecto). Prohíbe un alimento concreto:

$$\text{ha_consumido}[f_0, d, m] = 0, \quad \forall d \in D, m \in M$$

forbid_tag (dura por defecto). Prohíbe todos los alimentos de una etiqueta; es el mecanismo de las alergias e intolerancias:

$$\text{ha_consumido}[f, d, m] = 0, \quad \forall f \in \text{miembros}(t), d \in D, m \in M$$

prefer_food (blanda). Bonifica cada aparición del alimento, opcionalmente solo en un tipo de comida:

$$\text{bonificación} += \sum_{d \in D} \sum_{m \in M_{\text{ctx}}} \text{ha_consumido}[f_0, d, m]$$

donde M_{ctx} son las comidas del tipo indicado, o todas si no se indica.

prefer_tag (blanda). Como la anterior, bonificando cada aparición de un alimento de la etiqueta.

meal_kcal_ratio (blanda por defecto). Reparte la energía diaria entre las comidas según porcentajes deseados. Para cada comida con porcentaje objetivo pct (escalado por 10 para conservar un decimal), la desviación se linealiza con la multiplicación cruzada:

$$\text{kcal}_{d,m} \cdot 1000 - \text{round}(pct \cdot 10) \cdot \text{nut}_{d,\text{kcal}} = \text{over}_{d,m} - \text{under}_{d,m}$$

y $\text{over} + \text{under}$ entra como penalización; $\text{kcal}_{d,m}$ es la energía escalada de la comida m del día d .

max_servings_per_period (configurable). Limita las apariciones de un alimento o de una etiqueta en ventanas de window_days días. Como dura, para cada ventana W :

$$\sum_{f \in \text{obj}} \sum_{d \in W} \sum_{m \in M} \text{ha_consumido}[f, d, m] \leq \text{cap}$$

Como blanda, penaliza el exceso sobre el tope en cada ventana.

no_repeat_food y no_repeat_tag (blandas por defecto). Impiden o penalizan que el objetivo (un alimento, o cualquier alimento de una etiqueta) se repita en menos de sep días. Con la presencia por día del objetivo:

$$\text{pres}_d = \max_{f \in \text{obj}, m \in M} \text{ha_consumido}[f, d, m]$$

la versión dura exige, en cada ventana W de sep días, $\sum_{d \in W} \text{pres}_d \leq 1$; la blanda penaliza el exceso sobre 1 en cada ventana.

forbid_combination (dura por defecto). Prohíbe que dos alimentos, o un alimento y una etiqueta, coincidan en la misma comida. Con indicadores de presencia de cada grupo en la comida (a y b , máximos de cada grupo):

$$a_{d,m} + b_{d,m} \leq 1, \quad \forall d \in D, m \in M$$

B.6. Función objetivo

La función completa que se minimiza:

$$\min \left(\sum_i \text{penalización}_i - \sum_j \text{bonificación}_j + w_{\text{variety}} \cdot P_{\text{variety}} + w_{\text{spread}} \cdot P_{\text{spread}} \right)$$

Cada desviación absoluta se linealiza con dos variables no negativas de exceso y defecto:

$$\begin{aligned} \text{over} &\geq 0, & \text{under} &\geq 0, & \text{suma} - \text{objetivo} &= \text{over} - \text{under} \\ & & & & |\text{suma} - \text{objetivo}| &= \text{over} + \text{under} \end{aligned}$$

con las cotas ajustadas al catálogo: el exceso como mucho a la cota superior del nutriente menos el objetivo, y el defecto como mucho al objetivo. La cota superior de cada nutriente por comida se apoya en R2: es la suma de las `MAX_ITEMS_PER_MEAL` mayores aportaciones individuales alcanzables (la ración máxima del alimento por su densidad escalada), y la cota diaria es esa suma por el número de comidas.

Los objetivos blandos de energía y macronutrientes no penalizan $\text{over} + \text{under}$ directamente: sobre ellos se aplican las dos bandas de tolerancia de la sección B.5, la diaria y la de la media del plan, y solo los excesos fuera de ellas entran en la función objetivo, el de la media multiplicado por K . Con las tolerancias a cero la formulación se reduce a la desviación absoluta total sin variables adicionales. Los mínimos y máximos de nutriente no llevan banda porque son umbrales de un solo lado.

El término de variedad penaliza cada alimento no usado en todo el plan, con $\text{usado}[f]$ el máximo de `ha_consumido` sobre el plan entero:

$$P_{\text{variety}} = \sum_{f \in F} (1 - \text{usado}[f])$$

El término de reparto penaliza el exceso de apariciones de un alimento sobre el $\text{tope cap} = \lceil \text{MAX_APPEARANCES_PER_DAY_RATIO} \cdot |D| \rceil$:

$$\text{exceso}[f] \geq \left(\sum_{d \in D} \sum_{m \in M} \text{ha_consumido}[f, d, m] \right) - \text{cap}, \quad \text{exceso}[f] \geq 0$$

$$P_{\text{spread}} = \sum_{f \in F} \text{exceso}[f]$$

El peso efectivo de cada término de restricción es el producto del peso de su familia por el peso de la fila:

$$\text{peso efectivo} = w_{\text{familia}} \cdot \text{peso de la fila}$$

Un objetivo calórico con peso de fila 7 pesa $10 \cdot 7 = 70$ por unidad escalada de desviación fuera de la banda diaria (70 000 por kcal, y 700 000 por cada kcal que la suma de las desviaciones del plan se salga de la banda de la media, por el factor K); una preferencia de familia con peso 4 pesa $4000 \cdot 4 = 16000$ por aparición. En unidades naturales son magnitudes comparables, por el factor de 1000 de la sección B.1. Dentro de las bandas la energía no cuesta nada, así que ahí deciden la estructura y las preferencias.

B.7. Mapa de restricciones a código

Correspondencia entre cada elemento formal y su ubicación en el solver, para que el modelo sea trazable en ambos sentidos. Se citan fichero y función, no números de línea.

Elemento formal	Código
Variables de decisión y su enlace	<code>model.build_base_model</code>
Perfil de ración $\min_f$, $\max_f$ (R9)	<code>model.serving_bounds</code>
Cuantización por unidades (R10)	<code>model.build_base_model</code> (variable k)
Sumas escaladas $\text{nut}_{d,n}$	<code>model.build_base_model</code> (bucle de nutrientes)
Escala entera	<code>model.scale_target</code> , <code>model.scaled_100</code>
R1 y R2 (presencia y tope por comida)	<code>model._structural_constraints</code>
R3 (suelo calórico, Mifflin)	<code>model.daily_kcal_floor</code> , <code>model.mifflin_bmr</code>
R4 y R5 (proteína y grasa)	<code>model._structural_constraints</code>
R6, R7 y R8 (repetición y variedad)	<code>model._structural_constraints</code>
R11 a R15 (plausibilidad)	<code>model._plausibility_constraints</code>
Franjas por número de comidas	<code>loader._SLOTS_BY_COUNT</code> , <code>loader.load_meal_type_codes</code>

Tabla B.2: Mapa a código: modelo y reglas invariantes.

Elemento formal	Código
Aplicación del catálogo y asunciones	<code>catalog.apply_constraints</code> , <code>catalog.hard_enforce</code>
kcal_target / macro_target	<code>catalog._h_kcal_target</code> , <code>_h_macro_target</code> , <code>_target_deviation</code> ; <code>config.ObjectiveBands</code>
nutrient_min / max / ratio	<code>catalog._h_nutrient_min</code> , <code>_h_nutrient_max</code> , <code>_h_nutrient_ratio</code>
forbid_food / forbid_tag	<code>catalog._h_forbid_food</code> , <code>_h_forbid_tag</code>
prefer_food / prefer_tag	<code>catalog._h_prefer_food</code> , <code>_h_prefer_tag</code>
meal_kcal_ratio	<code>catalog._h_meal_kcal_ratio</code>
$\text{max_servings_per_period}$	<code>catalog._h_max_servings</code>
no_repeat_food / no_repeat_tag	<code>catalog._h_no_repeat_food</code> , <code>_h_no_repeat_tag</code>
$\text{forbid_combination}$	<code>catalog._h_forbid_combination</code>
Linealización del valor absoluto	<code>catalog._target_deviation</code>
Función objetivo (suma ponderada)	<code>objective.set_objective</code>
P_{variety} y P_{spread}	<code>objective._variety_penalty</code> , <code>objective._spread_penalty</code>
Resolución (tiempo, brecha, hilos)	<code>result.solve</code> , <code>config.SOLVE_WORKERS</code>
Diagnóstico de infactibilidad	<code>result._extract_infeasible</code>

Tabla B.3: Mapa a código: catálogo clínico, objetivo y resolución.

Prompts del sistema

Este anexo reproduce los prompts de sistema que reciben los modelos de lenguaje del copiloto (capítulo 6), tal como están en el código en el momento de cerrar la memoria. Están en inglés porque es el idioma en el que los modelos locales pequeños siguen mejor las instrucciones; la entrada del nutricionista va en español y el vocabulario de salida (tipos y códigos) es el del catálogo. Las líneas largas se han partido para caber en la página, sin cambiar el texto. Cada llamada añade al final el mensaje del usuario, precedido de la etiqueta que se indica en cada caso.

C.1. El traductor

El prompt de sistema del traductor tiene tres partes que se concatenan: el rol con sus reglas, el catálogo cerrado que el modelo puede producir y siete ejemplos fijos. El mensaje del usuario llega como **Input:** <texto> seguido de **Output:**, con una línea previa **Client:** <datos> si hay contexto del cliente.

Rol y reglas

You extract dietary constraints from what a nutritionist writes about a client, in Spanish or English.

Turn the text into a JSON object {"constraints": [...]}, one entry per distinct instruction. Rules:

- Use only the constraint types and codes from the catalog below. Never invent a type or a code.
- Tags and nutrients travel by code; foods travel by name (target_food_name), in the language of the text. When a rule is about a specific food (a serving cap, no-repeat, forbid, prefer), you MUST set target_food_name to that food; do not leave the item out.
- Allergies and intolerances are forbid_tag and hard. Preferences ("prefiere", "es vegetariana") are prefer_tag or prefer_food and soft.
- Fill only the fields a type needs. Leave the rest null.
- Whenever the text gives a number (kcal, grams, mg, servings, days, a ratio like "a third" = 0.33), you MUST put it in "value". Never emit a numeric constraint without its "value". kcal_target, macro_target, nutrient_min, nutrient_max, nutrient_ratio, max_servings_per_period and

- no_repeat_food always need "value".
- Read numbers as written. Do not judge whether a value is clinically sensible; translate it faithfully.
- If the text says nothing translatable, return an empty list.

Catálogo cerrado

CONSTRAINT TYPES (use only these):

- kcal_target: daily energy goal in kcal (value=kcal). Usually soft.
- macro_target: daily grams goal for a macronutrient (target_nutrient_code in {protein_g,carb_g,fat_g}, value=grams). Usually soft.
- nutrient_min: at least value of a nutrient per day (target_nutrient_code, value).
- nutrient_max: at most value of a nutrient per day (target_nutrient_code, value).
- nutrient_ratio: bound the ratio numerator/denominator (target_nutrient_code=numerator, denominator_nutrient_code, value=bound, ratio_bound=min|max).
- forbid_food: a food never appears (target_food_name). Hard by default.
- prefer_food: favor a food (target_food_name, optional meal_type). Soft.
- forbid_tag: no food from a family/allergen/intolerance (target_tag_code). Hard by default; use for allergies and intolerances.
- prefer_tag: favor a food family (target_tag_code). Soft.
- forbid_combination: two items must not share a meal (target_food_name or target_tag_code, plus combine_with_food_name or combine_with_tag_code). Hard.
- meal_kcal_ratio: share of daily energy per meal (split: {meal_type_code: percent}, sum <= 100). Soft.
- max_servings_per_period: cap how often a food or family shows up in a window (target_food_name or target_tag_code, value=cap, window_days). Soft by default.
- no_repeat_food: minimum gap in days before a food repeats (target_food_name, value=days).
- no_repeat_tag: minimum gap in days before a food family repeats (target_tag_code, value=days).

TAG CODES (use only these, pick the closest):

- allergen: gluten, milk_allergen, eggs_allergen, fish_allergen, crustaceans, mollusks_allergen, tree_nuts, peanuts, soy
- intolerance: lactose, fodmap_high
- cultural: vegetarian, vegan, halal, no_pork
- clinical_marker: high_sodium, high_potassium, high_phosphorus, high_gi
- processing: nova_4

NUTRIENT CODES (use only these): energy_kcal, protein_g, carb_g, fat_g, sat_fat_g, fiber_g, sodium_mg, sugar_added_g, iron_mg, calcium_mg, vit_d_ug, vit_b12_ug

MEAL TYPE CODES: breakfast, mid_morning, lunch, snack, dinner, late_snack

Ejemplos

Los tres primeros ejemplos reproducen las restricciones de los clientes de la semilla inicial; los cuatro restantes cubren los tipos cuyos argumentos viajan en el contexto (reparto, proporción, tope de raciones, prohibición por nombre).

EXAMPLES:

Input: Quiere perder peso, ponle unas 1500 kcal al dia y es vegetariana.

```
Output: {"constraints": [
  {"type": "kcal_target", "priority": "soft", "weight": 7, "value": 1500},
  {"type": "prefer_tag", "priority": "soft", "weight": 4,
   "target_tag_code": "vegetarian"]}]}
```

Input: Busca ganar musculo, es alergico al cacahuete y quiero al menos 140 g de proteina al dia.

```
Output: {"constraints": [
  {"type": "forbid_tag", "priority": "hard", "weight": 10,
   "target_tag_code": "peanuts"},
  {"type": "nutrient_min", "priority": "soft", "weight": 6,
   "target_nutrient_code": "protein_g", "value": 140}]]}
```

Input: Es diabetica e intolerante a la lactosa, y no pase de 2000 mg de sodio.

```
Output: {"constraints": [
  {"type": "forbid_tag", "priority": "hard", "weight": 9,
   "target_tag_code": "lactose"},
  {"type": "nutrient_max", "priority": "soft", "weight": 7,
   "target_nutrient_code": "sodium_mg", "value": 2000}]]}
```

Input: El desayuno debe ser un 30% de las calorías y la comida un 40%.

```
Output: {"constraints": [
  {"type": "meal_kcal_ratio", "priority": "soft", "weight": 5,
   "split": {"breakfast": 30, "lunch": 40}]]}
```

Input: Que no repita el mismo pescado en 3 dias, como mucho salmon dos veces por semana.

```
Output: {"constraints": [
  {"type": "no_repeat_tag", "priority": "soft", "weight": 5,
   "target_tag_code": "fish_allergen", "value": 3},
  {"type": "max_servings_per_period", "priority": "soft", "weight": 5,
   "target_food_name": "salmon", "value": 2, "window_days": 7}]]}
```

Input: El arroz como mucho tres veces por semana y nada de ternera.

```
Output: {"constraints": [
  {"type": "max_servings_per_period", "priority": "soft", "weight": 5,
   "target_food_name": "arroz", "value": 3, "window_days": 7},
  {"type": "forbid_food", "priority": "hard", "weight": 8,
   "target_food_name": "ternera", "value": 0}]]}
```

Input: Que las grasas saturadas no superen un tercio de la grasa total.

```
Output: {"constraints": [
  {"type": "nutrient_ratio", "priority": "soft", "weight": 5,
   "target_nutrient_code": "sat_fat_g", "denominator_nutrient_code": "fat_g",
   "value": 0.33, "ratio_bound": "max"}]]}
```

Esquema de salida

La salida se restringe con un esquema JSON que Ollama aplica durante la generación. Cada restricción lleva **type**, **priority** (**hard** o **soft**) y **weight** (1 a 10), y después solo los campos que su tipo necesita:

En el esquema que se pasa al modelo, **value** se declara numérico y requerido, por el hallazgo de la sección 7.4.2; en el esquema con que se parsea la respuesta sigue siendo opcional, y la capa determinista rechaza lo que falte con un motivo legible.

Campo	Papel
value, value2	argumento numérico (kcal, gramos, mg, raciones, días o proporción)
target_tag_code	etiqueta objetivo, por código del catálogo
target_nutrient_code	nutriente objetivo, por código del catálogo
target_food_name	alimento objetivo, por nombre en el idioma del texto
denominator_nutrient_code, ratio_bound	denominador y sentido de una proporción entre nutrientes
meal_type, window_days, granularity	franja, ventana de días y granularidad de la restricción
split	reparto de energía por franja, en porcentajes
combine_with_food_name, combine_with_tag_code	segundo elemento de una combinación prohibida

Tabla C.1: Campos de cada restricción en el esquema de salida del traductor.

C.2. Las tres comprobaciones previas

Cada comprobación recibe su propio prompt y nada más: sin catálogo, sin contexto del cliente y sin ver las otras comprobaciones. El mensaje del usuario llega como `Nutritionist's message`: seguido del texto. Los tres prompts son abstractos a propósito, sin cifras de ejemplo, por el eco de los ejemplos descrito en la sección 7.4.2.

Ámbito

You screen messages that a nutritionist writes to a diet planning assistant. The assistant only handles dietary constraints and goals for one client: calorie targets, nutrient minimums or maximums, allergies, intolerances, diets like vegetarian or vegan, foods to avoid or prefer, how often a food may appear, and how meals are structured. Most messages are orders to the assistant, in the imperative and about the plan it is going to build: what to put in, what to leave out, what to limit, how often something may come back. Those are dietary instructions and they belong in scope. A message needs no figures, no clinical reason and no polite framing to be in scope, and a single short sentence is enough. The subject matter decides the verdict, never the wording, the length or the tone.

Classify the message:

- `in_scope`: it says something about what the client should or should not eat or reach, or about how the plan is put together.
- `out_of_scope`: its subject is something else entirely, such as another area of the practice, the software itself, or conversation away from food.

Messages may be in Spanish or English. When the subject is food or diet but the request reads as incomplete or odd, answer `in_scope`: later passes judge whether it carries enough detail.

Reply with JSON: verdict (`in_scope` or `out_of_scope`) and reason (one short English sentence).

Completitud

Read a nutritionist's dietary requests. A request is DANGLING when it needs a number or a food name that the message does not contain, for example asking for more of a nutrient without an amount, or banning an unnamed food. A request that states its number, or that needs no number (an allergy, a diet style, a named food ban), is fine.

Reply with JSON: analysis (one short sentence per request: fine or dangling and why), missing (the data each dangling request needs, empty when none), complete (true when no request is dangling).

Contradicción

You check a nutritionist's dietary instructions for internal contradictions: two requests in the same message that cannot both hold (forbidding a food or food group and also requiring it, or two different values for the same target). Requests about different things are not contradictions: a calorie target combined with a diet style or with nutrient goals is a normal message. When unsure, answer false.

Reply with JSON: analysis (one short English sentence going over the requests and whether any pair clashes), contradictory (boolean) and conflict (one short English sentence naming the two clashing requests, empty when there is none).

Esquemas de salida

Los esquemas de las tres comprobaciones marcan todos sus campos como requeridos, para que un modelo pequeño los rellene siempre. El de ámbito tiene **verdict** y **reason**. El de completitud declara **analysis**, después **missing** y por último **complete**; el de contradicción, **analysis**, **contradictory** y **conflict**. El orden no es casual: bajo salida estructurada el modelo genera los campos en el orden declarado, y escribir el análisis antes del veredicto es lo que le impide contestar por patrón. En el lado del parseo, una respuesta que no responda a la pregunta se lee como “sin problema encontrado” en vez de tumbar la tarea.

Evaluación experimental: tablas completas

Este anexo recoge el detalle por caso de los experimentos del capítulo 7 que en el cuerpo se resumen con medianas o con una figura, más las figuras secundarias del estudio de formatos y la lista de casos de los bancos. Todo se regenera desde el repositorio: cada experimento tiene su documento, sus resultados crudos en JSON y el script que produce tablas y figuras, en la carpeta `feedback-docs` del repositorio (subcarpetas `experimentos-motor`, `experimentos-objetivo`, `estudio-formatos` y `auditoria-planes-8e`), con los harness en `backend/scripts`.

D.1. Casos de los bancos

El banco del solver tiene ocho casos con veredicto esperado:

1. María, 3 días y 3 comidas, sin restricciones configurables. Factible.
2. María, 7 días y 5 comidas, objetivo calórico blando de 1500 kcal. Factible; cada día dentro de ± 50 kcal y la media dentro de ± 25 .
3. María, 7 días y 5 comidas, objetivo calórico y preferencia vegetariana. Factible, con predominio vegetariano y el mismo criterio de bandas.
4. John, 7 días y 5 comidas, alergia al cacahuete dura y mínimo de proteína blando. Factible.
5. Emma, 7 días y 5 comidas, intolerancia a la lactosa dura y máximo de sodio blando. Factible.
6. John, 3 días y 5 comidas, 900 kcal duras y 180 g de proteína duros. Infactible, con las dos filas en el núcleo de conflicto.
7. Mínimo de proteína de 200 g duro con casi todas las fuentes prohibidas, construido contra el catálogo real. Infactible.
8. Diseño: ejercita la no repetición por etiqueta, el tope de raciones por periodo, el reparto por comidas y la preferencia por alimento. Factible.

Sobre esos casos el banco comprueba además las garantías de las reglas base y de la plausibilidad (tope de apariciones por día, variedad mínima, raciones dentro del perfil de cada alimento, franjas, roles de condimento y dulce), hasta las 32 comprobaciones de la tabla 7.1.

La prueba de extremo a extremo contra el backend desplegado corre doce casos por la API, los seis primeros anclados a clientes reales con sus restricciones de la base de datos y los otros seis con perfiles sintéticos:

1. María, 3 días, sin restricciones. Factible.
2. María, 7 días, objetivo calórico blando. Factible, desviación menor del 5 %.
3. María, 7 días, objetivo calórico y preferencia vegetariana. Factible, predominio vegetariano.
4. John, 7 días, alergia dura y mínimo de proteína. Factible.
5. Emma, 7 días, intolerancia dura y máximo de sodio. Factible.
6. John, 3 días, 900 kcal duras y 180 g de proteína duros. Infactible.
7. Sintético, 7 días y 4 comidas, objetivo calórico y reparto por comidas. Factible.
8. Sintético, 7 días y 5 comidas, carne roja como mucho dos veces por semana, duro. Factible.
9. Sintético, 7 días y 3 comidas, no repetir un alimento en 4 días, blando. Factible.
10. Sintético, 7 días y 3 comidas, no repetir pescado en 2 días, blando. Factible.
11. Estilo del profesional por ámbito nutricionista. Factible y refleja el estilo.
12. Sintético, 7 días y 5 comidas, mínimo alto de calcio con las familias ricas en calcio prohibidas. Infactible.

A los doce se suman el flujo de firma de un plan y el rechazo de una petición sin token, hasta las 29 comprobaciones.

D.2. Magnitudes de la función objetivo por caso

Contribución ponderada realizada de cada familia del objetivo, en unidades del objetivo, antes del ajuste de pesos (sección 7.2.3). Un guion indica que el caso no lleva ese término.

Cociente Q por caso y familia, antes del ajuste (rango completo alcanzable de la familia dividido por el coste de una unidad natural de desviación del término nutricional dominante):

Caso	kcal	macro	nutriente	reparto comidas	prefer.	variedad	reparto plan
banco 2	0	–	–	–	–	0	0
banco 3	0	–	–	–	2 224	100	0
banco 4	–	–	0	–	–	35	4
banco 5	–	–	0	–	–	60	0
banco 8	–	–	–	33 036 100 000	96	195	4
David	119 000	–	–	–	–	215	4
Lucía	–	–	0	–	2 688	230	0
Sofía	–	–	0	–	–	45	0
Tomás	694 932 000	4 899 720	–	62 508 450 000	–	135	0
Carlos	51 100	–	–	–	–	220	0
Nadia	–	–	–	–	2 688	60	0

Tabla D.1: Contribución de cada familia al objetivo, por caso, antes del ajuste. El término de preferencias es negativo en el objetivo (premia), y aquí se da en valor absoluto.

Caso	Preferencias	Reparto del plan	Variedad
banco 2	–	0,0514	0,0071
banco 3	0,0320	0,0514	0,0071
banco 4	–	0,0750	0,0104
banco 5	–	0,0514	0,0071
banco 8	0,0000	0,0001	0,0000
David	–	0,0514	0,0071
Lucía	0,0560	0,0750	0,0104
Sofía	–	0,0900	0,0125
Tomás	–	0,0001	0,0000
Carlos	–	0,0514	0,0071
mediana	0,0320	0,0514	0,0071

Tabla D.2: Q por caso antes del ajuste de pesos. Tras multiplicar por 1 000 los pesos no nutricionales, cada valor se multiplica por 1 000 (medianas 32, 51 y 7).

Caso	Sin rotura	Cadena lexicográfica
banco 2	óptimo, 4,9 s	óptimo, 5,2 s
banco 3	óptimo, 22,4 s	óptimo, 23,3 s
banco 4	óptimo, 69,7 s	5 de 5 en el límite; objetivo de 5 000 a 14 000
banco 5	óptimo, 66,7 s	5 de 5 en el límite; objetivo de 60 000 a 85 000
David	límite, desviación 0,02 %	límite, desviación 0,56 %; objetivo 13 veces peor
Lucía	límite	límite, objetivo algo peor
Sofía	límite sin óptimo	3 de 5 óptimos a 79,7 s; objetivo de 110 000 a 45 000
Tomás	óptimo, 9,8 s	óptimo, 11,2 s
Carlos	límite	límite, objetivo de 495 300 a 338 400
Nadia	óptimo, 52,9 s	3 de 5 sin probar óptimo, 85,6 s

Tabla D.3: Cadena lexicográfica frente al motor sin rotura.

Caso	Sin orden	Orden escalar (energía diaria no creciente)
banco 2	óptimo, 4,9 s	óptimo, 3,2 s
banco 3	óptimo, 22,4 s	óptimo, 9,7 s
banco 4	óptimo, 69,7 s; objetivo 5 000	5 de 5 en el límite; objetivo 100 000
banco 5	óptimo, 66,7 s; objetivo 60 000	5 de 5 en el límite; objetivo 75 000
David	límite; objetivo 430 100	límite; objetivo 230 100
Lucía	límite; objetivo −2 414 000	límite; objetivo −2 453 000
Sofía	80,8 s, 1 de 5 óptimos	63,5 s, 5 de 5 óptimos; objetivo de 110 000 a 45 000
Tomás	óptimo, 9,8 s	óptimo, 15,3 s
Carlos	límite; objetivo 495 300	límite; objetivo 387 200
Nadia	óptimo, 52,9 s	óptimo, 69,8 s

Tabla D.4: Variante escalar frente al motor sin orden.

D.3. Rotura de simetría: valores por caso

Detalle de la sección 7.3.3: estado, tiempo medio y objetivo (menor es mejor) en cada brazo, con 5 repeticiones por caso.

D.4. Bandas de tolerancia: desviaciones con signo

Para cada caso afectado de la sección 7.2.4: media con signo del plan en las tres repeticiones, peor día y exceso fuera de la banda diaria y de la banda de la media.

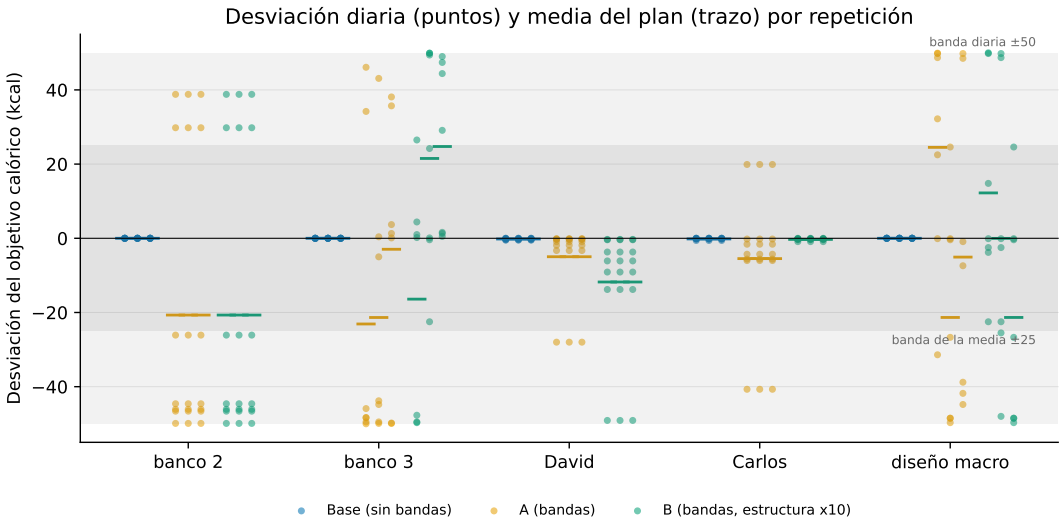


Figura D.1: Desviación diaria (puntos) y media del plan (trazo) por repetición en los casos afectados limpios. Las franjas grises son la banda diaria de ± 50 kcal y la banda de la media de ± 25 . Tomás, el caso de conflicto, queda fuera de la figura porque su desviación de $-1\,240$ kcal aplastaría la escala.

Caso	Brazo	Media con signo (kcal)	Peor día (kcal)	Fuera de banda
banco 2	base	0,0 / 0,0 / 0,0	0,0	0 / 0
banco 2	A y B	−20,7 en las tres	49,9	0 / 0
banco 3	base	0,0 / 0,0 / 0,0	0,0	0 / 0
banco 3	A	−23,1 / −21,4 / −3,0	50,0	0 / 0
banco 3	B	−16,4 / +21,5 / +24,7	49,9	0 / 0
David	base	−0,2 en las tres	0,5	0 / 0
David	A	−5,0 en las tres	28,0	0 / 0
David	B	−11,8 en las tres	49,1	0 / 0
Carlos	base	−0,2 en las tres	0,6	0 / 0
Carlos	A	−5,5 en las tres	40,7	0 / 0
Carlos	B	−0,3 en las tres	0,9	0 / 0
diseño macro (kcal)	A	+24,5 / −21,3 / −5,1	49,9	0 / 0
diseño macro (kcal)	B	+12,2 / 0,0 / −21,3	50,0	0 / 0
diseño macro (proteína, g)	A	+0,9 / +1,4 / +1,7	4,5	0 / 0
diseño macro (proteína, g)	B	+1,9 / +0,8 / +1,4	4,5	0 / 0

Tabla D.5: Desviaciones del objetivo por caso afectado, brazo y repetición. La última columna es el exceso fuera de la banda diaria y fuera de la banda de la media.

D.5. Estudio de formatos: detalle

Los cinco formatos del estudio de la sección 7.4.1, con el mismo mensaje redactado en cada uno:

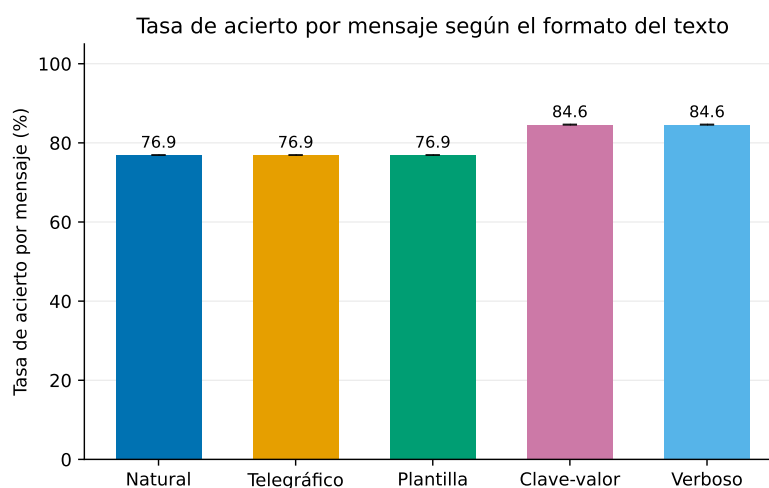


Figura D.2: Tasa de acierto por mensaje según el formato (media de 3 repeticiones, desviación cero).

Formato	Descripción	Mensaje 2
Natural	texto libre de un nutricionista, línea base	Busca ganar músculo, es alérgico al cacahuete y quiero al menos 140 g de proteína al día.
Telegráfico	lista de viñetas	- alergia: cacahuete / - proteína: mínimo 140 g/día / - objetivo: ganar músculo
Plantilla	campos con etiqueta legible	Objetivo: ganar músculo. Evitar: cacahuete (alergia). Mínimos: 140 g de proteína al día.
Clave-valor	pares casi de configuración	alergia=cacahuete; proteína_min=140g/dia; objetivo=ganar_musculo
Verboso	conversacional con contexto irrelevante de consulta	Este chico entrena fuerza cuatro días a la semana y me pregunta mucho por los batidos... quiero que llegue al menos a 140 g de proteína al día, y mucho cuidado porque es alérgico al cacahuete.

Tabla D.6: Los cinco formatos del estudio sobre el mismo mensaje.

Formato	Latencia media	Tokens de entrada	Tokens de salida
Natural	20,4 s (d.t. 8,8)	1 555	67
Telegráfico	20,5 s (d.t. 4,3)	1 550	63
Plantilla	17,5 s (d.t. 3,8)	1 555	63
Clave-valor	15,9 s (d.t. 2,7)	1 547	64
Verboso	16,0 s (d.t. 2,5)	1 588	64

Tabla D.7: Latencia y tokens por llamada al traductor, por formato, en el equipo local.

Formato	Superan las pasadas	Motivos de bloqueo
Natural	11 de 13	fuera de ámbito (mensajes 11 y 13)
Telegráfico	8 de 13	fuera de ámbito (mensajes 5, 7, 10, 11 y 13)
Plantilla	12 de 13	fuera de ámbito (mensaje 11)
Clave-valor	10 de 13	fuera de ámbito (7 y 13) e idioma no soportado (12)
Verboso	13 de 13	ninguno

Tabla D.8: Los formatos frente a las pasadas de validación previa (una pasada por par formato-mensaje).

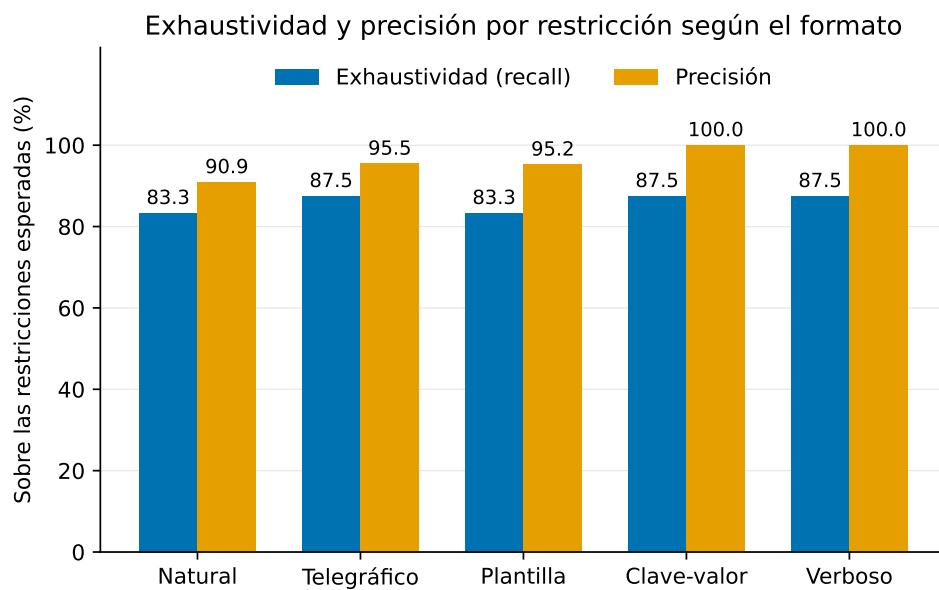


Figura D.3: Exhaustividad y precisión por restricción, por formato.

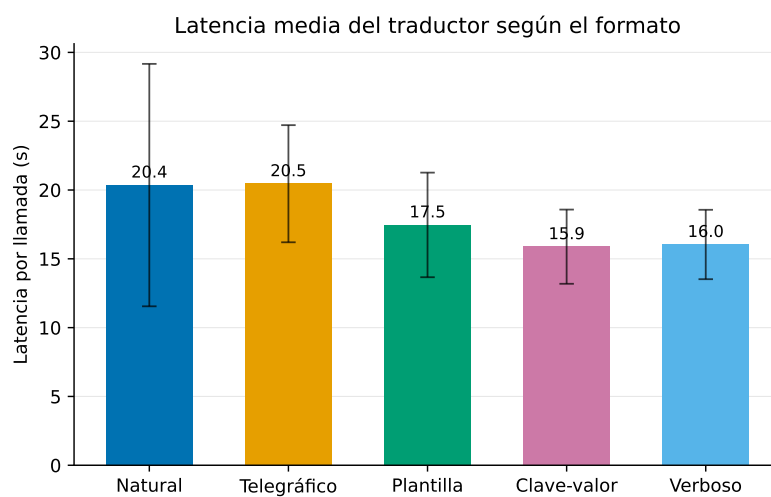


Figura D.4: Latencia por llamada al traductor, por formato.

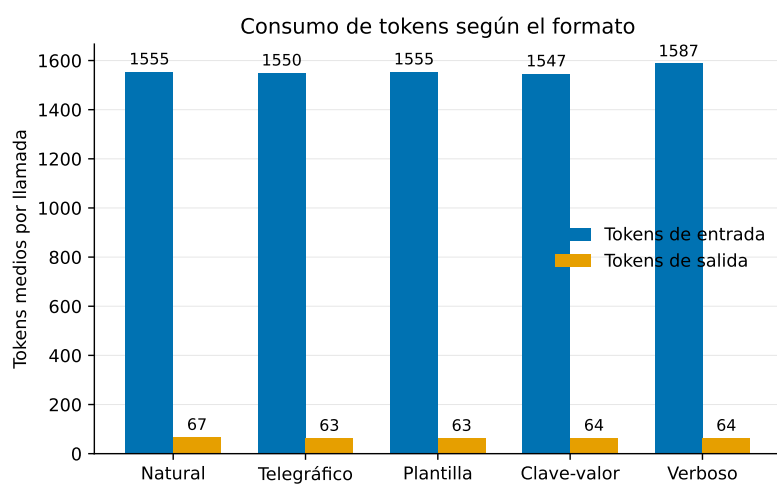


Figura D.5: Tokens de entrada y salida por llamada, por formato.

Declaración de uso de inteligencia artificial

Este anexo detalla el uso que he hecho de herramientas de inteligencia artificial generativa durante el trabajo, y complementa la declaración responsable sobre autoría y uso ético de herramientas de inteligencia artificial que la Universidad Complutense de Madrid pide firmar con la entrega. Recoge qué herramientas y modelos usé y en qué periodos, para qué tareas, y qué parte del trabajo se hizo sin ellas.

E.1. Herramientas y modelos

- **Claude Code (Anthropic).** Ha sido la herramienta principal (Anthropic, 2026a) desde enero de 2026 hasta la entrega, en septiembre de 2026. Es el asistente de programación de Anthropic para el terminal y el editor, y trabaja directamente sobre el repositorio del proyecto. En cada momento usé la versión de la familia Claude Opus disponible en la herramienta y, desde junio de 2026, también la familia Claude Fable. Con los identificadores de la documentación de Anthropic (Anthropic, 2026b), los modelos publicados en ese periodo fueron Claude Opus 4.5 (`claude-opus-4-5-20251101`, noviembre de 2025, el vigente en enero de 2026), Claude Opus 4.6 (`claude-opus-4-6`, febrero de 2026), Claude Opus 4.7 (`claude-opus-4-7`, abril de 2026), Claude Opus 4.8 (`claude-opus-4-8`, mayo de 2026), Claude Fable 5 (`claude-fable-5`, junio de 2026), Claude Opus 5 (`claude-opus-5`, julio de 2026) y Claude Fable 5.1 (`claude-fable-5-1`, septiembre de 2026). No conservo un registro de qué modelo atendió cada sesión, así que declaro el rango completo.
- **Figma desde Claude Code.** El prototipo navegable de Figma se construyó y se actualizó desde Claude Code a través del servidor MCP oficial de Figma (Figma, Inc., 2026b) (*Model Context Protocol*), que permite al asistente leer y modificar el fichero de diseño. Las pantallas del segundo prototipo y su actualización tras la auditoría entre diseño e implementación (capítulos 3 y 4) se hicieron así, sobre mis indicaciones y con mi revisión en Figma.
- **Gemini (Google).** Antes de Claude Code, en junio y julio de 2025 y en diciembre de 2025, usé la aplicación Gemini de Google con los modelos que

ofrecía entonces: en junio y julio de 2025 la familia Gemini 2.5 (Gemini 2.5 Pro y Gemini 2.5 Flash, publicados como versiones estables ese mismo junio) y en diciembre de 2025 la familia Gemini 3 (Gemini 3 Pro y Gemini 3 Flash, publicados en noviembre y diciembre de 2025) junto con la 2.5 (Google, 2026). No conservo registro de qué variante respondió a cada consulta.

No he usado ChatGPT ni ninguna otra herramienta de inteligencia artificial generativa.

E.2. Tareas en las que se usaron

Gemini lo usé para desarrollar la idea y como apoyo en los primeros pasos del proyecto, sin una tarea concreta.

Claude Code intervino como asistente en la escritura del código del repositorio, a partir de mis especificaciones y con mi revisión; en el diseño de las pantallas de Figma; en la preparación de los experimentos del capítulo 7 y de los scripts que producen sus tablas y figuras; en la redacción de esta memoria, incluida su traducción al inglés, sobre borradores que revisé y edité; y en la documentación interna del proyecto.

E.3. Autoría y verificación

Las decisiones de alcance, de arquitectura y de modelado son mías, tomadas en las reuniones con los directores del TFG y a partir de lo que pedían los nutricionistas consultados. Las entrevistas con profesionales y la prueba con usuarios las hice yo. Los datos y cifras de la evaluación están medidos sobre el sistema, no generados por ningún modelo. La bibliografía está verificada en su fuente. Y ninguna parte del sistema desplegado ni de esta memoria se ha incorporado sin mi revisión.